1 Introdução

[TODO]

1.1 Objetivos

[TODO]

1.1.1 Objetivo Geral

[TODO]

1.1.2 Objetivos Específicos

[TODO]

1.2 Organização da monografia

[TODO]

Este trabalho é composto por XX capítulos. Neste Capítulo 1, proporcionamos uma introdução, justificativa e delineamos os objetivos...

2 Fundamentação Teórica

Este capítulo apresenta os conceitos necessários para a compreensão do trabalho. Inicialmente, são introduzidos fundamentos de aprendizado profundo e redes neurais artificiais, seguidos pelo processo de treinamento dessas redes. Em seguida, discute-se o problema do consumo energético em modelos de *Deep Learning* e, por fim, são apresentadas as técnicas de compressão de modelos, com destaque para a poda neural (*pruning*), que é o foco deste trabalho.

A exposição é deliberadamente progressiva e não pressupõe familiaridade prévia com aprendizado de máquina. Cada conceito é introduzido primeiro por meio de uma analogia com situações cotidianas e só então formalizado, de modo que o leitor possa acompanhar o raciocínio antes de encontrar a notação matemática que o descreve.

2.1 Aprendizado Profundo e Redes Neurais

2.1.1 Aprendizado Profundo (Deep Learning)

O Aprendizado Profundo é um campo especializado de aprendizado de máquina (GOODFELLOW; BENGIO; COURVILLE, 2016; LECUN; BENGIO; HINTON, 2015), caracterizado pelo uso de redes neurais artificiais com múltiplas camadas, o que permite aprender e extrair informações complexas a partir de grandes volumes de dados.

2.1.2 Redes Neurais Artificiais

A rede neural artificial é um modelo computacional inspirado na estrutura do cérebro humano, que busca imitar a forma como neurônios se conectam e processam sinais para aprender a partir de dados, identificando padrões e tomando decisões. Um exemplo clássico é o perceptron multicamadas (MLP), um modelo composto por funções matemáticas que mapeiam valores de entrada em valores de saída (GOODFELLOW; BENGIO; COURVILLE, 2016). Em tarefas de visão computacional, por exemplo, milhões de imagens de um objeto podem ser processadas para que a rede aprenda suas características visuais e seja capaz de reconhecê-lo posteriormente.

Uma forma de tentar enteder a definição de uma RNN é, imagine uma comissão encarregada de decidir se um pedido de empréstimo deve ser aprovado. Nenhum membro decide sozinho, cada um examina um aspecto diferentee, como a renda declarada, o histórico de pagamentos, o tempo de emprego e então, após analisar todos os aspectos, emite um parecer. Esses pareceres não têm todos o mesmo valor, pois a comissão confia mais

em uns avaliadores do que em outros, e a decisão final combina todos eles, ponderados por essa confiança. Em uma rede neural, cada nó é um desses avaliadores e o peso de cada conexão é exatamente a confiança depositada naquele parecer. A diferença essencial é que a rede não recebe essas confianças prontas, ela as descobre sozinha, por tentativa e erro, ao longo do treinamento.

Uma rede neural profunda é composta por unidades (nós) interconectadas, organizadas em camadas (GOODFELLOW; BENGIO; COURVILLE, 2016; NIELSEN, 2015).

- **Nó (neurônio):** É uma unidade computacional que calcula uma soma ponderada das entradas mais um *bias* e passa o resultado através de uma função de ativação, produzindo uma saída.
- **Camada de Entrada (Input Layer):** É onde o processo é iniciado e os dados são introduzidos na rede. O número de nós nessa camada corresponde ao número de características dos dados de entrada.
- **Camadas Ocultas (Hidden Layers):** São camadas intermediárias entre a entrada e a saída. Cada nó de uma camada oculta está conectado a todos os nós das camadas anterior e seguinte. É nelas que ocorre a maior parte do processamento e do aprendizado; a saída de uma camada oculta é utilizada como entrada da próxima, e, a cada camada, a rede passa a detectar características mais complexas e abstratas. Redes com múltiplas camadas ocultas são as chamadas redes de *Deep Learning*.
- **Camada de Saída (Output Layer):** Camada final da rede, onde o resultado é produzido. A estrutura da camada de saída depende da tarefa: em regressão, há um único nó com função de ativação linear; em classificação binária, um único nó expressa resultados do tipo sim/não; em classificação multiclasse, há um nó para cada classe possível, e o de maior ativação corresponde à predição final.

Cada conexão entre nós possui um peso associado, que determina a sua importância relativa. Pesos altos amplificam o sinal de entrada, enquanto pesos próximos de zero tornam a conexão pouco influente. Os pesos são inicializados com valores aleatórios e ajustados durante o treinamento (GOODFELLOW; BENGIO; COURVILLE, 2016).

2.1.3 Bias em Redes Neurais

O *bias* é um parâmetro adicional que ajusta a saída de um neurônio. Trata-se de um valor numérico associado a cada neurônio, não ligado a nenhuma entrada específica, atuando como um coeficiente linear. Antes da aplicação da função de ativação, o *bias* é somado ao resultado da soma ponderada das entradas.

A principal função do *bias* é deslocar a função de ativação, permitindo que o modelo se ajuste melhor aos dados. Sem ele, a soma ponderada precisaria ser alta para que o neurônio fosse ativado; com o *bias*, um neurônio pode ser ativado mesmo com uma soma ponderada baixa, caso seu *bias* seja suficientemente alto (NIELSEN, 2015). Inversamente, um *bias* muito negativo dificulta a ativação do neurônio. O *bias* é um parâmetro aprendível: inicialmente é inicializado com zero ou valores próximos de zero e, durante o treinamento, é atualizado pelo mesmo processo de retropropagação e descida de gradiente utilizado para os pesos.

Retomando com a analogia da comissão de crédito, se os pesos representam o quanto se confia em cada avaliador, o *bias* representa o quanto o avaliador é exigente. Um membro gentil aprova o pedido mesmo diante de evidências modestas; um membro rigoroso só aprova quando as evidências são fortes. São duas coisas independentes — confiar muito em alguém não diz nada sobre o quanto essa pessoa é exigente — e é por isso que a rede precisa aprender os dois conjuntos de valores separadamente.

2.1.4 Função de Ativação

A função de ativação é uma função matemática aplicada ao resultado da soma ponderada das entradas de um neurônio (já acrescido do *bias*), determinando a saída do neurônio. Sua principal função é introduzir não linearidade no modelo: sem ela, a composição de diversas camadas resultaria em uma transformação puramente linear, equivalente a uma rede com uma única camada, o que limitaria drasticamente a capacidade expressiva do modelo (GOODFELLOW; BENGIO; COURVILLE, 2016).

A razão de existir dessa função merece um exemplo concreto, pois é ela que justifica o próprio empilhamento de camadas. Considere um produto de R$ 100,00 submetido a dois descontos sucessivos, um de 10% e outro de 20%: o primeiro leva o preço a R$ 90,00 e o segundo a R$ 72,00, exatamente o mesmo resultado que um único desconto de 28% produziria. Os dois passos colapsaram em um só, e o segundo nada acrescentou que um único desconto bem escolhido não fizesse sozinho. Encadear operações lineares nunca gera nada além de outra operação linear, e com redes neurais acontece o mesmo: sem função de ativação, uma rede de cem camadas calcularia exatamente aquilo que uma única camada já calcularia, e toda a profundidade seria desperdiçada.

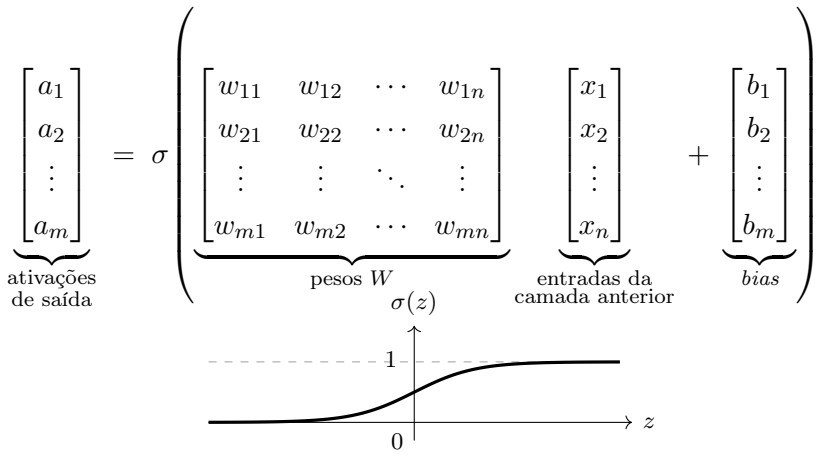
Suponha agora que a loja adote uma regra adicional, aplicada logo depois de cada desconto: nenhum produto pode ser vendido por menos de R$ 85,00. Essa regra é o análogo da função de ativação — ela não faz parte do desconto, mas incide sobre o resultado dele, antes que esse resultado siga para a etapa seguinte. Com ela, o encadeamento deixa de colapsar. O produto de R$ 100,00 cai para R$ 90,00 e depois para R$ 72,00, mas a regra o devolve a R$ 85,00, de modo que o desconto efetivo foi de 15%. Já um produto de R$ 200,00 percorre os mesmos dois descontos, chega a R$ 144,00 sem nunca esbarrar

no limite e tem desconto efetivo de 28%. A mesma sequência de operações passou a produzir resultados diferentes conforme a entrada, e nenhum desconto único reproduz esse comportamento. É esse rompimento do encadeamento que permite a cada camada acrescentar algo que a anterior não conseguia representar. A regra de preço mínimo, aliás, tem a mesma forma da ReLU, apresentada adiante nesta seção: ambas preservam os valores acima de um limiar e substituem os demais por um valor fixo.

A ativação de um neurônio ocorre em três etapas (NIELSEN, 2015):

- Soma ponderada: o neurônio multiplica o valor de ativação de cada nó da camada anterior pelo peso da respectiva conexão e soma todos os resultados.
- Adição do *bias*: ao resultado da soma ponderada é somado o valor do *bias*, que auxilia o neurônio a ativar-se mais ou menos facilidade.
- Função de ativação: o resultado final (soma ponderada acrescida do *bias*) é passado pela função de ativação, cuja saída é então transmitida aos nós da próxima camada.

Figura 1 – Representação matricial da propagação de um neurônio, da soma ponderada à função de ativação (Sigmoide).



a função de ativação comprime qualquer valor de z no intervalo $(0, 1)$

Fonte: Adaptado de Nielsen (2015).

Há diversas funções de ativação, cada uma com suas próprias características e aplicações.

- **ReLU (Rectified Linear Unit):** Função de ativação mais utilizada em redes neurais modernas, proposta para redes profundas por Nair e Hinton (2010) e popularizada por Krizhevsky, Sutskever e Hinton (2012). É uma função linear por partes, com inclinação 1 para valores positivos e 0 para valores negativos. É computacionalmente muito eficiente e não sofre do problema do desaparecimento do gradiente

para entradas positivas. Em contrapartida, pode sofrer do problema conhecido como *Dying ReLU*, no qual neurônios cujas entradas se tornam persistentemente negativas deixam de ser ativados e param de aprender.

- **Sigmoide:** Função que mapeia qualquer valor real para o intervalo $(0, 1)$, sendo útil em classificação binária, já que a saída pode ser interpretada como uma probabilidade. Caiu em desuso em camadas ocultas porque, para entradas com magnitude muito alta ou muito baixa, sua derivada se aproxima de zero, fenômeno conhecido como desaparecimento do gradiente (BENGIO; SIMARD; FRASCONI, 1994).

- **Tanh (Tangente Hiperbólica):** Semelhante à Sigmoide, porém mapeando os valores para o intervalo $(-1, 1)$, o que a torna centrada em zero e costuma favorecer a convergência. Também sofre do problema do desaparecimento do gradiente.

- **Softmax:** Diferentemente das demais, não é aplicada a neurônios individuais de camadas ocultas, mas sim ao conjunto dos neurônios da camada de saída em tarefas de classificação multiclasse. Transforma as saídas brutas da última camada em uma distribuição de probabilidades, com valores no intervalo $(0, 1)$ que somam 1. Na prática, converte pontuações sem escala definida em porcentagens comparáveis: o modelo deixa de responder "7,2 para gato e 3,1 para cachorro" e passa a responder "85% de chance de ser gato, 15% de ser cachorro".

Em cada camada, as ativações produzidas determinam as ativações da camada seguinte. De camada em camada, esses padrões tornam-se progressivamente mais complexos e abstratos, até que a camada de saída produz o resultado final. Assim, camadas iniciais aprendem características simples, enquanto camadas mais profundas combinam essas características para representar conceitos de alto nível (LECUN; BENGIO; HINTON, 2015).

2.2 Treinamento de Redes Neurais

2.2.1 Gradiente

No contexto de redes neurais, o gradiente corresponde ao vetor de derivadas parciais da função de perda em relação aos pesos da rede. Ele é o elemento central que permite a minimização da função de perda por meio do algoritmo da descida do gradiente, um método de primeira ordem que utiliza as informações do gradiente para atualizar os parâmetros da rede (GOODFELLOW; BENGIO; COURVILLE, 2016).

Formalmente, sendo L a função de perda e $\mathbf{w} = (w_1, w_2, \ldots, w_n)$ o conjunto dos n parâmetros da rede, o gradiente é o vetor que reúne a derivada parcial da perda em relação a cada um deles (GOODFELLOW; BENGIO; COURVILLE, 2016):

$$\nabla L(\mathbf{w}) = \left[\frac{\partial L}{\partial w_1},\ \frac{\partial L}{\partial w_2},\ \ldots,\ \frac{\partial L}{\partial w_n}\right] \tag{2.1}$$

Cada componente do vetor de gradiente é uma derivada parcial, calculada assumindo que as demais variáveis são mantidas constantes. A magnitude da derivada indica o quanto a função está variando; o seu sinal, a direção dessa variação: positivo, a função está aumentando; negativo, diminuindo.

Tomado como um todo, o vetor da Equação 2.1 aponta na direção em que a perda cresce mais rapidamente. É por isso que a descida do gradiente caminha no sentido oposto, $-\nabla L(\mathbf{w})$: seguir o gradiente invertido é o que aproxima a rede de um mínimo da função de perda.

Uma analogia comum é a de um caminhante que desce uma encosta em meio à neblina, enxergando apenas o solo ao seu redor: a cada passo, ele sente para qual lado o terreno mais desce (o oposto do gradiente) e avança nessa direção. Na descida do gradiente, a "encosta" é a função de perda e os "passos" são os ajustes nos pesos, repetidos até alcançar um ponto de mínimo.

2.2.2 Backpropagation

A retropropagação (*backpropagation*) é o algoritmo que permite o aprendizado das redes neurais, por possibilitar o cálculo da contribuição de cada parâmetro (peso ou *bias*) para o erro da predição (RUMELHART; HINTON; WILLIAMS, 1986). Uma vez que o erro é calculado na camada de saída, o algoritmo executa um fluxo invertido: da saída em direção à entrada, propagando o erro para as camadas anteriores.

A lógica é a mesma de uma apuração de responsabilidades em uma linha de produção. Sabe-se apenas que o produto final saiu defeituoso; para corrigir o processo, é preciso percorrer a linha de trás para frente e estimar o quanto cada etapa contribuiu para o defeito. Uma etapa que respondeu por boa parte do problema precisa de um ajuste grande; outra que quase não influiu, de um ajuste pequeno. O *backpropagation* faz esse rastreamento de forma matemática, atribuindo a cada peso da rede uma parcela de responsabilidade pelo erro.

Para saber como cada peso individual se correlaciona com o erro, o algoritmo calcula a derivada parcial da função de perda em relação a esse peso, utilizando a regra da cadeia multivariada para decompor o cálculo em derivadas intermediárias. Com todas as derivadas obtidas, um algoritmo de otimização atualiza cada peso na direção oposta ao gradiente:

$$w_{novo} = w_{antigo} - \eta \cdot \frac{\partial L}{\partial w}$$

em que η é a taxa de aprendizado, hiperparâmetro que controla o tamanho do ajuste aplicado a cada iteração.

Voltando ao caminhante na neblina, a taxa de aprendizado é o tamanho do passo que ele dá. Passos curtos demais tornam a descida segura, porém lenta a ponto de o treinamento não terminar em tempo viável, já passos longos demais fazem o caminhante descer mais rápido, mas aumenta o risco de se machucar. Encontrar um valor adequado é uma das decisões mais sensíveis do treinamento.

2.2.3 Hiperparâmetros

Hiperparâmetros são variáveis externas que definem tanto o algoritmo de aprendizado quanto a arquitetura da rede neural. Diferentemente dos parâmetros do modelo (pesos e *biases*), que são ajustados automaticamente pelo *backpropagation*, os hiperparâmetros são definidos antes do início do treinamento e controlam como a rede aprende a partir dos dados. Existem técnicas automatizadas para sua busca, como *grid search* (BERGSTRA; BENGIO, 2012) e otimização bayesiana (SNOEK; LAROCHELLE; ADAMS, 2012), mas o ajuste manual baseado em experiência ainda é amplamente empregado.

A distinção entre parâmetros e hiperparâmetros é a mesma que separa, em uma cozinha, as escolhas do cozinheiro do resultado do cozimento. A temperatura do forno e o tempo de preparo são definidos antes de o prato ir ao fogo e não se alteram sozinhos: são os hiperparâmetros. O ponto da massa e a cor da crosta, por outro lado, são produzidos pelo processo: são os parâmetros. Errar a temperatura não impede o prato de assar, mas compromete o resultado e, do mesmo modo, hiperparâmetros mal escolhidos não impedem o treinamento de ocorrer, apenas fazem com que ele chegue a um modelo pior.

Em geral, os hiperparâmetros podem ser agrupados em duas categorias:

- **Hiperparâmetros de arquitetura:** definem a estrutura da rede, como o número de camadas ocultas, o número de neurônios em cada camada e o tipo de função de ativação utilizada.
- **Hiperparâmetros de treinamento:** controlam como o modelo se ajusta aos dados, como a taxa de aprendizado, o número de épocas, o tamanho do lote (*batch size*) e a escolha do algoritmo de otimização.

2.2.4 Passo a passo do treinamento

O treinamento de uma rede neural supervisionada requer uma grande base de dados rotulada, isto é, cada exemplo de entrada deve estar associado a uma saída correta. Essa base de dados é tradicionalmente dividida em três subconjuntos (GOODFELLOW; BENGIO; COURVILLE, 2016):

- **Conjunto de treinamento**, usado para ajustar os pesos e *biases*;

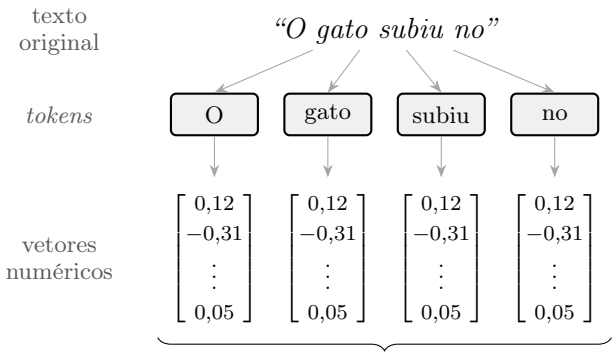
- **Conjunto de validação**, usado para monitorar o desempenho durante o treinamento e ajustar hiperparâmetros;
- **Conjunto de teste**, usado para avaliar o desempenho final do modelo de forma imparcial.

Além disso, é necessário definir a arquitetura da rede (camadas, neurônios, funções de ativação) e os hiperparâmetros de treinamento. O treinamento ocorre ao longo de diversas iterações, nas quais cada passagem completa pelo conjunto de treinamento é chamada de época (*epoch*); em cada época, os dados são processados em pequenos lotes (*batches*) (GOODFELLOW; BENGIO; COURVILLE, 2016).

A analogia mais direta é a de um estudante diante de um livro. Uma época equivale a percorrer o livro inteiro uma vez; os lotes são os trechos em que essa leitura é dividida, com uma pausa para revisão ao fim de cada um. E, assim como raramente basta uma única leitura para fixar o conteúdo, a rede precisa percorrer o conjunto de treinamento diversas vezes até que os ajustes se estabilizem.

Como exemplo, considere um modelo de linguagem que recebe a frase "O gato subiu no". Antes de qualquer cálculo, o texto precisa virar número: ele é fragmentado em unidades menores, chamadas *tokens*, e cada *token* é convertido em um vetor de valores numéricos. São esses vetores que alimentam a camada de entrada — cada posição do vetor corresponde a um nó (JURAFSKY; MARTIN, 2025). A Figura 2 ilustra esse percurso, da frase aos nós da camada de entrada.

Figura 2 – Da frase aos nós da camada de entrada.



Fonte: .

O ciclo de treinamento pode ser resumido nos seguintes passos (GOODFELLOW; BENGIO; COURVILLE, 2016):

- **Passo 1 – Inicialização dos parâmetros:** pesos são inicializados com valores aleatórios baixos e *biases* com zero. Essa quebra de simetria permite que diferentes neurônios aprendam características distintas.

- **Passo 2 – *Forward pass*:** um lote de dados é inserido na camada de entrada e propagado através das camadas ocultas até a camada de saída, produzindo uma predição.
- **Passo 3 – Cálculo da perda:** a predição é comparada com o rótulo verdadeiro por meio de uma função de perda, que quantifica numericamente o quão distante a predição está do valor correto.
- **Passo 4 – *Backpropagation*:** o erro é propagado de volta pela rede, calculando-se o gradiente da função de perda em relação a cada peso e *bias*.
- **Passo 5 – Atualização dos parâmetros:** um otimizador atualiza pesos e *biases* na direção oposta ao gradiente, reduzindo a função de perda.
- **Passo 6 – Repetição:** os passos 2 a 5 são repetidos para cada lote e ao longo de um número definido de épocas.

O treinamento é interrompido quando o modelo atinge uma precisão satisfatória ou quando a função de perda deixa de diminuir significativamente.

2.3 Arquitetura Transformer

A arquitetura *Transformer*, proposta por Vaswani et al. (2017) no trabalho *Attention is All You Need*, marcou uma mudança profunda na forma como modelos de aprendizado profundo processam sequências de texto. Antes dessa proposta, o campo era dominado por arquiteturas que processavam a sequência elemento por elemento, palavra por palavra. O *Transformer* rompeu com essa abordagem ao introduzir um mecanismo capaz de considerar toda a sequência de uma só vez. Esta seção apresenta, de forma progressiva, os conceitos necessários para compreender a arquitetura utilizada neste trabalho: as limitações das abordagens anteriores, o mecanismo de atenção, a operação de autoatenção e sua versão multi-cabeça, a estrutura de cada bloco da rede e, finalmente, o modelo GPT-2, que é o objeto central dos experimentos de poda.

2.3.1 O Problema que o Transformer Resolve

Toda arquitetura nasce como resposta a uma limitação da anterior, e o *Transformer* não é exceção. Compreender qual era essa limitação é o que transforma o mecanismo de atenção em uma solução, e não em uma escolha arbitrária.

Até 2017, o processamento de texto era dominado pelas redes neurais recorrentes (*Recurrent Neural Networks*, RNNs). Elas leem a frase palavra por palavra, na ordem em que aparece, carregando consigo um resumo do que já foi lido: a cada nova palavra, esse

resumo é atualizado. É, em essência, o modo como uma pessoa lê um texto em voz alta, guardando na memória o que veio antes. Dessa mecânica decorrem os dois problemas que motivaram o *Transformer*.

O primeiro é o esquecimento. Considere o trecho "A Inglaterra é minha terra natal. Passei toda a minha vida lá. Mudei-me para a Espanha há dois dias. Eu falo apenas uma língua, que é o ___". Preencher a lacuna com "inglês" exige recuperar uma palavra que apareceu muito antes. Como o resumo é reescrito a cada passo, a informação antiga vai se diluindo, e a influência de "Inglaterra" chega enfraquecida ao final. Tecnicamente, o fenômeno se manifesta durante o treinamento como gradiente desaparecente (*vanishing gradient*): os gradientes são multiplicados repetidamente pelas matrizes de peso e, quando esses valores são pequenos, encolhem até se aproximarem de zero (BENGIO; SIMARD; FRASCONI, 1994). É a limitação conhecida como incapacidade de lidar com dependências de longo alcance. Arquiteturas posteriores, como as redes LSTM (*Long Short-Term Memory*), atenuaram o esquecimento por meio de mecanismos que regulam o que deve ser mantido ou descartado (HOCHREITER; SCHMIDHUBER, 1997), mas não resolveram o segundo problema.

O segundo é a lentidão, como cada palavra só pode ser processada depois da anterior, a leitura é obrigatoriamente sequencial: não há como dividir o trabalho entre vários processadores. Isso desperdiça justamente a principal característica das unidades de processamento gráfico (GPUs) modernas, que é executar milhares de operações ao mesmo tempo.

Foi para atacar essas duas limitações simultaneamente que se desenvolveu o mecanismo de atenção.

2.3.2 Mecanismo de Atenção

O passo seguinte dessa evolução foi o mecanismo de atenção (*attention mechanism*), introduzido por Bahdanau, Cho e Bengio (2015) no contexto da tradução automática. Em vez de comprimir toda a frase em um único estado oculto, a atenção permite que o modelo "olhe" de volta e foque seletivamente em partes específicas da entrada. No exemplo anterior, ela pode consultar diretamente o termo "Inglaterra", independentemente de quão distante ele esteja da lacuna, eliminando o risco de o gradiente desaparecer pelo caminho.

Para operacionalizar essa ideia, o mecanismo utiliza três conceitos: a consulta (*query*), a chave (*key*) e o valor (*value*). Uma analogia útil é a de uma biblioteca: cada livro tem uma etiqueta na lombada (a *chave*) que descreve seu conteúdo; quando um leitor chega com uma pergunta (a *consulta*), ela é comparada com cada etiqueta para decidir quais livros são mais relevantes; o conteúdo desses livros (os *valores*) é então com-

binado, com os mais relevantes contribuindo mais para a resposta. No *Transformer*, esse processo ocorre com números: a relevância entre uma consulta e cada chave é calculada matematicamente, e os valores são somados com pesos proporcionais a essa relevância.

A inovação central de Vaswani et al. (2017), em 2017, foi construir uma arquitetura inteira baseada exclusivamente nesse mecanismo, dispensando por completo a recorrência. Com isso, todas as posições de uma sequência se comunicam diretamente em uma única operação, sem precisar "passar" por palavras intermediárias. Isso resolve as duas limitações das RNNs de uma só vez: as dependências de longo alcance tornam-se tão fáceis de capturar quanto as de curto alcance, e o cálculo passa a ser paralelizável, tornando as arquiteturas baseadas em RNN e LSTM obsoletas para muitas tarefas modernas.

2.3.3 Autoatenção e Atenção Multi-Cabeça

A variante do mecanismo de atenção utilizada no *Transformer* é chamada de autoatenção (*self-attention*). O prefixo "auto" indica que a consulta, a chave e o valor são todos derivados da mesma sequência de entrada. Ou seja, ao processar uma frase, cada palavra pergunta a si mesma: "quais outras palavras desta frase são relevantes para minha interpretação?". Exemplo: "O gato que estava no telhado dormiu". Ao processar a palavra "dormiu", o mecanismo de autoatenção calcula um peso para cada palavra da frase. A palavra "gato" recebe peso alto, pois é o sujeito do verbo; palavras como "telhado" ou "no" recebem pesos baixos, pois são menos relevantes para determinar quem dormiu.

Na prática, esses pesos são calculados da seguinte forma, a partir de cada representação de entrada, três matrizes de pesos aprendíveis durante o treinamento transformam cada representação em três vetores: o vetor de consulta Q (*query*), o vetor de chave K (*key*) e o vetor de valor V (*value*). O grau de compatibilidade entre uma consulta e cada chave é calculado pelo produto escalar entre eles. Esses valores brutos de compatibilidade passam pela função *softmax*, que os converte em uma distribuição de probabilidades, isto é, um conjunto de números entre 0 e 1 que somam exatamente 1. Esses pesos normalizados determinam com que intensidade cada valor V contribui para o resultado final, que é obtido pela soma ponderada dos valores.

Essa operação completa é denominada *Scaled Dot-Product Attention* (atenção por produto escalar escalado) e é definida formalmente por (VASWANI et al., 2017):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Em termos intuitivos, o produto $QK^\top$ mede a compatibilidade entre todas as consultas e todas as chaves de uma só vez; a divisão por $\sqrt{d_k}$ (a dimensão das chaves) é um fator de escala que estabiliza o treinamento; a *softmax* converte essas pontuações em pesos

que somam 1; e a multiplicação por V produz, para cada posição, uma média ponderada dos valores, isto é, sua representação já contextualizada pelo restante da sequência.

Uma única operação de atenção, porém, captura apenas um tipo de relação entre as palavras de cada vez. Para superar essa limitação, Vaswani et al. (2017) propõem a atenção multi-cabeça (*multi-head attention*). A analogia é a seguinte: se a atenção simples é como um único leitor relendo a frase em busca de relações gramaticais, a atenção multi-cabeça é como h leitores independentes, cada um procurando um tipo diferente de relação. Um leitor pode focar em concordância verbal ("gato"/"dormiu"), outro em relações de posse ("telhado do vizinho"), outro em referências pronominais, e assim por diante. O GPT-2 *small*, por exemplo, utiliza 12 cabeças de atenção por bloco.

Na prática, cada uma das h cabeças executa a operação *Scaled Dot-Product Attention* de forma independente, com suas próprias matrizes de projeção aprendíveis para Q, K e V. As saídas das h cabeças são depois concatenadas e projetadas por uma matriz linear aprendível, produzindo a representação final de cada posição. Desse modo, o modelo pode simultaneamente capturar relações em diferentes subespaços semânticos e sintáticos, sem custo adicional expressivo em relação à atenção simples.

Sob a perspectiva da compressão de modelos, as cabeças de atenção constituem unidades podáveis naturais para a poda estruturada: estudos da literatura demonstram que nem todas as cabeças são igualmente úteis, e que algumas podem ser removidas inteiramente sem degradação significativa da qualidade das predições (MICHEL; LEVY; NEUBIG, 2019).

2.3.4 Estrutura do Bloco Transformer

Para entender como o *Transformer* processa informação em profundidade, é útil pensar em cada bloco como uma "estação de trabalho" em uma linha de montagem. Na primeira bancada, as palavras trocam informações entre si por meio da atenção multi-cabeça: cada palavra atualiza sua representação consultando todas as demais. Na segunda bancada, cada palavra refina individualmente sua própria representação, de forma independente das outras, por meio de uma pequena rede totalmente conectada. Esse par de operações se repete em cada bloco, e os blocos são empilhados, fazendo com que as representações das palavras se tornem progressivamente mais ricas e abstratas a cada andar.

O primeiro componente de cada bloco é a atenção multi-cabeça, já apresentada na seção anterior, responsável por integrar informação contextual de toda a sequência. O segundo é uma rede *feedforward* totalmente conectada — um MLP (*multilayer perceptron*) aplicado de forma idêntica e independente a cada posição. Esse MLP é composto por duas transformações lineares com uma função de ativação não linear entre elas: a função

GELU (*Gaussian Error Linear Unit*) (HENDRYCKS; GIMPEL, 2016) nos modelos GPT, ou ReLU em outras variantes. Diferentemente da ReLU (NAIR; HINTON, 2010), que simplesmente zera valores negativos, a GELU faz uma transição suave em torno da origem. Na prática, essa suavidade evita o problema dos "neurônios mortos" (*dying ReLU*), em que unidades deixam de aprender por receberem gradiente nulo, e por isso Radford et al. (2019) a adotaram como ativação padrão na família GPT. A dimensão intermediária desse MLP é tipicamente quatro vezes maior do que a dimensão do modelo — no GPT-2 *small*, a dimensão do modelo é 768 e a dimensão intermediária é 3.072 — o que confere ao bloco grande capacidade de representação antes de comprimir novamente a informação.

2.3.5 Modelos de Linguagem de Grande Escala e o GPT-2

Modelos de linguagem de grande escala (*Large Language Models*, LLMs) são redes neurais profundas, tipicamente baseadas na arquitetura *Transformer*, treinadas em grandes volumes de texto com o objetivo de aprender a prever palavras. A tarefa de pré-treinamento é simples de enunciar: dado um trecho de texto, o modelo deve prever qual token vem a seguir. Um *token* é a unidade mínima de texto que o modelo processa — pode ser uma palavra, parte de uma palavra, um sinal de pontuação ou um espaço. Dada a sequência "O aluno abriu o", um LLM bem treinado atribuiria alta probabilidade a tokens como "livro", "caderno" ou "computador", e baixa probabilidade a tokens como "foguete" ou "plutônio". Repetida bilhões de vezes sobre textos variados, essa tarefa aparentemente simples força o modelo a aprender gramática, fatos sobre o mundo, raciocínio básico e muito mais (RADFORD et al., 2019).

Para que o texto possa ser processado numericamente, ele precisa primeiro ser convertido em tokens e depois em vetores numéricos. Essa conversão ocorre em duas etapas. A primeira é a tokenização, realizada por algoritmos como o BPE (*Byte Pair Encoding*), que divide o texto em unidades de comprimento variável de forma a equilibrar cobertura e eficiência. Por exemplo, a palavra "inconstitucional" pode ser dividida em tokens como "in", "constitu" e "cional". O GPT-2 possui um vocabulário de aproximadamente 50.257 tokens. A segunda etapa converte cada token em um vetor numérico denso por meio de uma camada de *embedding*: o modelo aprende, durante o treinamento, uma representação vetorial para cada token do vocabulário, de modo que tokens semanticamente próximos tendam a ter representações próximas no espaço vetorial.

O GPT-2 (*Generative Pre-trained Transformer 2*), proposto por Radford et al. (2019) pela OpenAI em 2019, é um modelo autorregressivo baseado exclusivamente no componente decodificador do *Transformer* original. Ele foi pré-treinado em uma base de dados de aproximadamente 40 GB de texto coletado da internet, abrangendo uma grande diversidade de domínios e estilos. A variante GPT-2 *small*, utilizada neste trabalho, possui 12 blocos *Transformer* empilhados, 12 cabeças de atenção por bloco, dimensão de modelo

igual a 768 e aproximadamente 124 milhões de parâmetros treináveis. Ao longo deste trabalho, cada um desses blocos é denominado **camada**, e as camadas são numeradas de 0 a 11, na ordem em que processam a entrada, conforme a indexação adotada na implementação; é nessa unidade que se definem a análise de sensibilidade por camada e as taxas de poda por camada. Apesar de ser a menor variante da família GPT-2, esse número de parâmetros é cerca de cinco vezes maior do que o de uma ResNet-50 (25 milhões de parâmetros), arquitetura de referência nos estudos clássicos de poda — e as variantes maiores da própria família GPT-2 chegam a 1,5 bilhão de parâmetros, o que ilustra a escala característica dos modelos baseados em *Transformer*.

O crescimento em escala dos LLMs ao longo dos anos seguintes foi vertiginoso: o GPT-3, publicado em 2020, possui 175 bilhões de parâmetros; o PaLM, do Google, chega a 540 bilhões. Mesmo modelos de médio porte como o GPT-2 impõem custos energéticos e computacionais consideráveis em ambientes de produção, onde a inferência é executada milhares ou milhões de vezes por dia. A poda estruturada surge, nesse contexto, como uma estratégia concreta para reduzir esses custos: a remoção de cabeças de atenção redundantes e de neurônios MLP de baixa importância pode diminuir substancialmente a contagem de operações por inferência, com impacto controlado sobre a qualidade das predições (PATTERSON et al., 2021).

2.4 Consumo Energético em Redes Neurais

O avanço de técnicas e *hardware* voltados ao treinamento de redes neurais, motivado pela busca por maior precisão, trouxe consigo um consumo de energia considerável e, consequentemente, uma significativa emissão de dióxido de carbono. Strubell, Ganesh e McCallum (2019) relatam, por exemplo, que o treinamento de um modelo *transformer* com busca de arquitetura neural pode emitir cerca de 626.155 libras de CO_2, enquanto a emissão média de um automóvel ao longo de toda a sua vida útil é de aproximadamente 126.000 libras — cerca de cinco vezes menos.

Esse custo energético concentra-se em duas etapas do ciclo de vida de um modelo: o treinamento e a inferência.

No treinamento, o modelo é alimentado com grandes volumes de dados e ajustado iterativamente pelo *backpropagation*. Esse processo é computacionalmente intenso e geralmente requer *hardware* especializado (GPUs ou TPUs), muitas vezes organizado em *clusters* com centenas ou milhares de aceleradores (PATTERSON et al., 2021; SCHWARTZ et al., 2020).

Na inferência, o modelo já treinado é utilizado para gerar predições. O custo de uma única predição é relativamente baixo; contudo, em contextos de larga escala — como os modelos de IA generativa servindo milhões de requisições diárias — o custo energético

agregado é expressivo (LUCCIONI; VIGUIER; LIGOZAT, 2022).

A diferença entre as duas etapas é de natureza, não apenas de grandeza. O treinamento é um gasto concentrado e único, comparável ao custo de construir uma ponte; a inferência é um gasto pequeno, porém perpétuo, comparável ao custo de mantê-la aberta ao tráfego. Uma ponte muito usada acaba custando, ao longo de décadas, mais em manutenção do que custou para ser erguida — e o mesmo ocorre com um modelo consultado milhões de vezes por dia. É por essa razão que este trabalho concentra-se na eficiência da inferência.

2.5 Compressão de Modelos

Com o crescimento do *Deep Learning*, os modelos de IA vêm dobrando em tamanho a cada poucos anos (SEVILLA et al., 2022). Modelos de alta performance costumam ser otimizados exclusivamente para acurácia, resultando em redes com grande quantidade de parâmetros, que ocupam muito espaço de armazenamento e demandam um elevado número de operações em ponto flutuante (FLOPs) para serem executadas. Isso é particularmente problemático em dispositivos com capacidade limitada, como *smartphones* e sistemas embarcados, e também em contextos de uso em larga escala, nos quais o custo energético se torna significativo.

A compressão de modelos é um campo que reúne técnicas voltadas a atenuar esses problemas, reduzindo o tamanho dos modelos e/ou a quantidade de operações necessárias para sua execução, preservando, tanto quanto possível, a acurácia. As redes neurais modernas possuem alta redundância de parâmetros (DENIL et al., 2013), o que torna viável remover ou simplificar aqueles com baixa contribuição ao resultado final.

Essa redundância não é um defeito, mas uma consequência do modo como esses modelos são construídos. Como não se sabe de antemão quantos parâmetros a tarefa exige, projeta-se a rede com folga generosa, do mesmo modo que se dimensiona uma instalação elétrica para uma carga muito acima da que será efetivamente usada. A folga facilita o treinamento, mas permanece no modelo depois que ele já aprendeu — e é justamente essa sobra que as técnicas de compressão procuram eliminar.

As quatro principais famílias de técnicas de compressão são:

- **Quantização (Quantization):** reduz o número de *bits* utilizados para representar cada peso do modelo. Em vez de pontos flutuantes de 32 bits, os parâmetros passam a ser representados com precisão reduzida (16 ou 8 bits, por exemplo), o que diminui o tamanho do modelo e pode acelerar operações em *hardware* que suporta aritmética de menor precisão (JACOB et al., 2018). O equivalente cotidiano é anotar preços arredondados: registrar R$ 20 em vez de R$ 19,99 ocupa menos espaço e agiliza as

contas, e na maioria das situações a diferença é irrelevante.

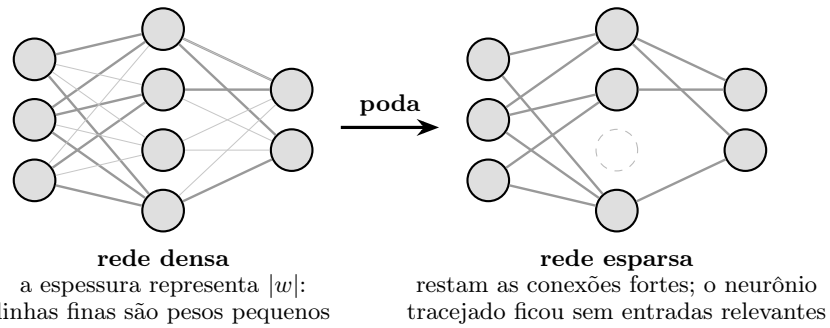
- **Destilação de Conhecimento (Knowledge Distillation):** proposta por Hinton, Vinyals e Dean (2015), é uma metodologia de treinamento em que um modelo menor (*aluno*) aprende a partir de um modelo maior e mais preciso (*professor*). O aluno é treinado não apenas para acertar os rótulos verdadeiros, mas também para imitar a distribuição de saída do professor, resultando em um modelo compacto que, frequentemente, alcança acurácia superior à que teria se fosse treinado do zero.

- **Fatoração de Matrizes (Matrix Factorization):** matrizes de pesos de grandes dimensões são decompostas em produto de matrizes menores cuja multiplicação aproxima a matriz original, reduzindo o custo computacional (DENTON et al., 2014).

- **Poda Neural (Neural Pruning):** processo de remoção de pesos — ou de estruturas maiores, como filtros — que contribuem pouco para a saída do modelo. É a técnica central deste trabalho e será detalhada na Seção a seguir.

A Figura 3 ilustra o processo de poda: a rede original, densa, tem conexões ou neurônios removidos e torna-se esparsa.

Figura 3 – Ilustração do processo de poda.



Fonte: Adaptado de Han et al. (2015).

2.6 Poda Neural (Pruning)

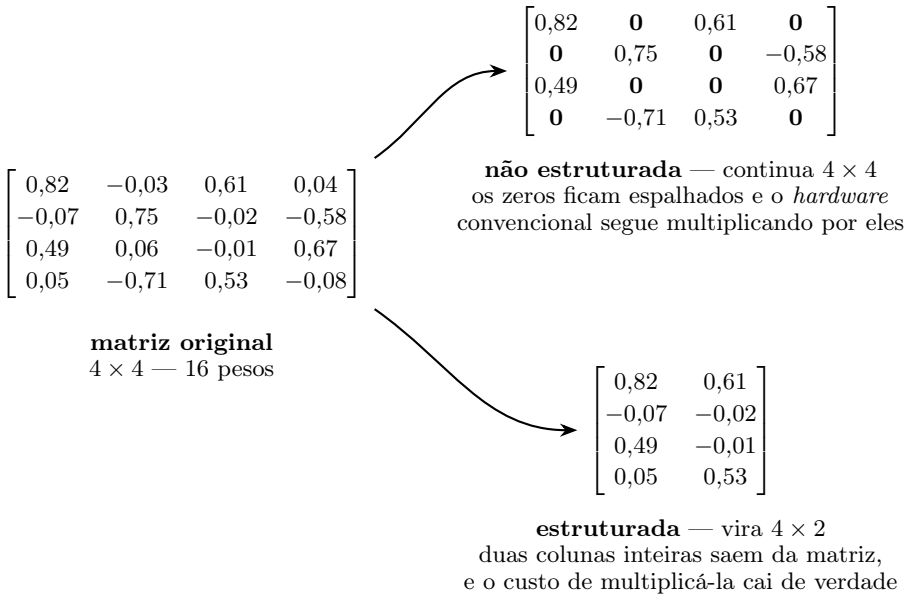
A poda neural é uma técnica de compressão que visa reduzir o tamanho do modelo e o custo computacional da inferência por meio da remoção de parâmetros que pouco contribuem para o resultado final. O processo mais comum parte de um modelo previamente treinado, cujo foco inicial foi exclusivamente a acurácia, e aplica a poda como uma etapa posterior (HAN et al., 2015; BLALOCK et al., 2020).

2.6.1 Tipos de Poda

Na literatura, duas categorias principais de poda são consideradas (BLALOCK et al., 2020):

- **Poda não estruturada:** remoção de pesos individuais, independentemente da sua posição na matriz de pesos. Esta abordagem oferece maior flexibilidade e, geralmente, taxas de compressão elevadas com perda reduzida de acurácia. Entretanto, do ponto de vista do custo de inferência, a poda não estruturada apresenta uma limitação importante: processadores e GPUs convencionais não são eficientes em pular multiplicações por zero espalhadas de forma aleatória na matriz de pesos. Assim, embora o modelo tenha muitos zeros, as operações ainda são executadas, e o ganho prático de velocidade e energia tende a ser marginal, a menos que se utilize *hardware* especializado ou formatos de armazenamento esparso.
- **Poda estruturada:** remoção de estruturas inteiras, como neurônios, filtros ou canais convolucionais, o que implica a remoção completa de detectores de características. Como o resultado é uma matriz menor e ainda densa, há um ganho direto de velocidade e eficiência em *hardware* convencional. O custo dessa abordagem é uma potencial perda maior de acurácia para uma mesma taxa de compressão.

Figura 4 – As duas categorias de poda aplicadas à mesma matriz de pesos.



Na Figura 4, as duas categorias são aplicadas à mesma matriz de pesos. Na poda não estruturada, os pesos removidos são apenas zerados, e a matriz conserva suas dimensões; na poda estruturada, colunas inteiras são eliminadas, e a matriz encolhe de fato.

A distinção entre as duas categorias é o que separa um ganho real de um ganho apenas contábil, e uma analogia com um livro a torna evidente. A poda não estruturada equivale a apagar palavras espalhadas por todas as páginas: o texto fica mais enxuto, mas o livro continua com o mesmo número de páginas, e quem for lê-lo ainda terá de folhear todas elas — inclusive as partes apagadas. A poda estruturada equivale a arrancar capítulos inteiros: o livro encolhe de fato, pesa menos e é percorrido em menos tempo. Como o *hardware* convencional não sabe pular um espaço em branco no meio de uma multiplicação, apenas a segunda forma se converte em redução efetiva de tempo e de energia. Essa diferença é retomada no Capítulo 6, onde é verificada experimentalmente.

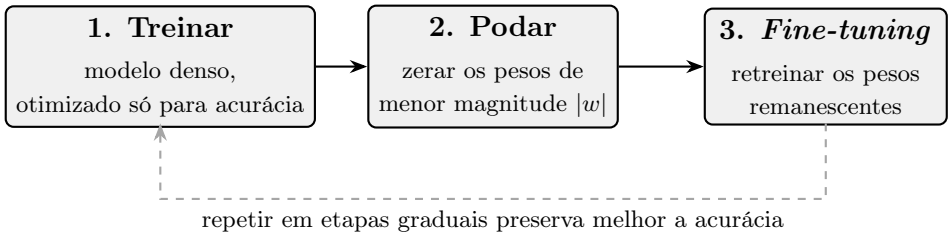
Cabe mencionar também a chamada esparsidade estruturada (por exemplo, padrões 2:4), em que o algoritmo é forçado a zerar, a cada bloco de quatro pesos, exatamente dois. Como o *hardware* sabe previamente quantos e em que formato os zeros aparecem, a matriz pode ser comprimida de maneira determinística, permitindo acelerações reais em GPUs modernas que suportam esse padrão (NVIDIA Corporation, 2020).

Assim, a poda estruturada é a que apresenta benefícios mais imediatos em termos de FLOPs e consumo de energia em *hardware* convencional (LI et al., 2017), razão pela qual ela recebe destaque neste trabalho.

2.6.2 Poda por Magnitude

A técnica mais simples e amplamente estudada é a poda por magnitude. Nela, considera-se que pesos cujos valores absolutos são próximos de zero contribuem pouco para a saída do modelo e, portanto, podem ser removidos. Define-se um limiar (ou uma taxa de esparsidade desejada) e todos os pesos abaixo desse limiar são zerados, criando redes esparsas (HAN et al., 2015). A Figura 5 resume o *pipeline* clássico do método: treinamento do modelo denso, remoção dos pesos de menor magnitude e retreinamento (*fine-tuning*) dos pesos remanescentes.

Figura 5 – *Pipeline* clássico de poda por magnitude.



Fonte: Adaptado de Han et al. (2015).

2.6.3 Sensibilidade e a Distribuição da Poda entre Camadas

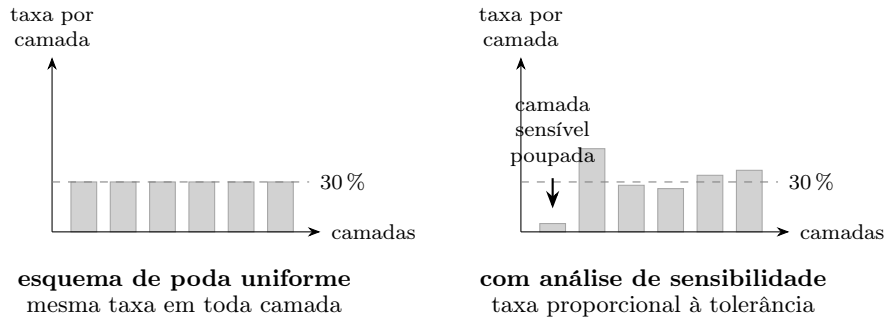
A poda por magnitude responde a uma pergunta: *quais* pesos remover. Ela deixa intocada uma segunda pergunta, logicamente anterior e menos discutida: *quanto* remover de cada parte da rede. Um modelo com doze camadas podado a uma taxa de 30 % admite infinitas distribuições dessa poda — 30 % em cada camada, 60 % na metade delas e nada na outra metade, ou qualquer combinação intermediária. Todas removem a mesma quantidade de parâmetros e produzem modelos de custo praticamente idêntico, mas não produzem modelos de qualidade idêntica.

A analogia é a de uma empresa que precisa cortar 30 % das despesas. O corte linear — 30 % em todos os setores — é o mais simples de justificar e o que exige menos informação, mas trata a contabilidade e a produção como igualmente dispensáveis. Um corte melhor exige saber de antemão qual setor a empresa suporta perder e qual a paralisa. Essa informação não está na despesa de cada setor: um setor caro pode ser supérfluo, e um setor barato pode ser crítico. Ela só aparece quando se pergunta o que acontece com a empresa na ausência de cada setor.

É exatamente essa a definição de **sensibilidade** adotada neste trabalho: a sensibilidade de uma camada é o quanto o desempenho do modelo se degrada quando aquela camada, e somente ela, é podada. Trata-se de uma medida empírica e direta de importância, obtida por ablação, e não de um *proxy*. A distinção em relação à magnitude é central: a magnitude é uma propriedade dos pesos, disponível sem custo algum, mas apenas correlacionada com a importância; a sensibilidade é uma propriedade medida do comportamento do modelo, cara de obter, porém direta.

A Figura 6 ilustra, de forma esquemática, duas formas de atingir a mesma taxa de poda, de 30%, nas camadas de uma rede. À esquerda, o esquema de poda uniforme ignora qualquer medida de importância; à direita, o esquema de poda com análise de sensibilidade poupa as camadas mais sensíveis e compensa nas mais tolerantes à poda. Ambos removem a mesma fração total de parâmetros: a linha tracejada indica a taxa de poda do modelo, que é a média das taxas por camada nos dois casos.

Figura 6 – Duas formas de atingir a mesma taxa de poda nas camadas de uma rede.



esquema de poda uniforme
mesma taxa em toda camada

com análise de sensibilidade
taxa proporcional à tolerância

A ideia de medir importância por degradação não é nova. Ela remonta ao *Optimal*

Brain Damage de LeCun, Denker e Solla (1990), que estima, por aproximação de segunda ordem, o aumento de erro provocado pela remoção de cada peso, e ordena os parâmetros por essa estimativa em vez de por magnitude. A limitação do método é o custo: a aproximação exige informação de curvatura, cujo cálculo é proibitivo em redes com centenas de milhões de parâmetros. A alternativa adotada aqui troca a estimativa analítica por medição direta e reduz drasticamente a granularidade — em vez de estimar a importância de cada peso individualmente, mede-se a de cada camada. Com doze camadas, o perfil completo custa doze avaliações do modelo, sem nenhum treinamento envolvido.

Há evidência consolidada de que essa desigualdade entre partes da rede existe e é substancial. Denil et al. (2013) mostram que uma fração pequena dos parâmetros de uma rede é suficiente para predizer todos os demais, o que indica redundância distribuída de forma desigual. Em *Transformers* especificamente, Michel, Levy e Neubig (2019) verificam que a maioria das cabeças de atenção pode ser removida após o treinamento com perda pequena de desempenho, enquanto algumas poucas são indispensáveis — e que quais delas são indispensáveis não é previsível a partir de propriedades dos pesos. Se a importância fosse aproximadamente uniforme, a distribuição da poda entre as camadas seria irrelevante e a poda uniforme seria ótima por construção; é precisamente porque não é uniforme que existe espaço para distribuir melhor a poda entre as camadas.

Duas ressalvas conceituais acompanham o método e delimitam o que ele pode oferecer.

A primeira é que a sensibilidade medida por ablação de uma camada por vez não captura interações entre camadas. A degradação causada por podar duas camadas simultaneamente não é, em geral, a soma das degradações individuais, de modo que o perfil deve ser tratado como uma **ordenação relativa de tolerância à poda**, e não como um modelo capaz de prever a qualidade de uma configuração completa. A segunda é que a sensibilidade não é uma propriedade absoluta da camada: ela é medida em relação a uma estratégia de poda específica, porque remover pesos de menor magnitude e remover cabeças de atenção inteiras são operações diferentes, que danificam a camada de maneiras diferentes. O perfil precisa, portanto, ser levantado uma vez para cada estratégia.

O procedimento que converte esse perfil em taxas de poda por camada é detalhado na Seção 5.3, e ambas as ressalvas acima são verificadas experimentalmente no Capítulo 6.

2.6.4 Pipeline Geral da Poda

De forma geral, a poda pode ser dividida nos seguintes passos (BLALOCK et al., 2020):

- **Passo 1 – Definição de critério e taxa de poda:** partindo de um modelo já treinado, define-se o critério de poda dos parâmetros (magnitude absoluta, norma

L_1 de filtros, sensibilidade, entre outros) e a taxa de esparsidade desejada.

- **Passo 2 – Identificação dos parâmetros a podar:** com base no critério escolhido, o algoritmo classifica os parâmetros por importância e seleciona os menos relevantes.
- **Passo 3 – Execução da poda:** os parâmetros selecionados são zerados (no caso da poda não estruturada) ou fisicamente removidos (no caso da poda estruturada), gerando um modelo esparso ou uma nova arquitetura reduzida.
- **Passo 4 – Ajuste fino (*fine-tuning*):** como a remoção tende a deteriorar a acurácia, o modelo podado é retreinado com uma taxa de aprendizado reduzida, para que os parâmetros remanescentes compensem a ausência dos que foram removidos.
- **Passo 5 – Iteração:** a poda pode ser aplicada de forma iterativa, em pequenas etapas intercaladas por *fine-tuning*, o que costuma preservar melhor a acurácia do que uma poda agressiva em um único passo.

2.6.5 Quando a Poda é Aplicada

Tradicionalmente, a poda é aplicada após o treinamento inicial do modelo, seguida de um ajuste fino. Métodos mais recentes aplicam a poda de forma gradual durante o próprio treinamento, intercalando passos de treino e poda, o que permite à rede adaptar-se progressivamente à nova estrutura (ZHU; GUPTA, 2017).

Uma linha de pesquisa mais recente investiga a possibilidade de podar a rede antes do treinamento, identificando sub-redes inerentemente promissoras dentro de uma rede densa inicializada aleatoriamente. Essa abordagem é impulsionada pela chamada *Lottery Ticket Hypothesis* (FRANKLE; CARBIN, 2019), segundo a qual, dentro de uma rede grande inicializada aleatoriamente, existe uma sub-rede pequena (o "bilhete premiado") que, quando treinada isoladamente com a mesma inicialização, atinge acurácia comparável à da rede completa.

3 Trabalhos Relacionados

Este capítulo apresenta trabalhos da literatura diretamente relacionados ao tema desta pesquisa, com foco em estratégias de poda neural e, em especial, naquelas que visam a redução do consumo energético de modelos de *Deep Learning*. Para cada trabalho, são descritos os objetivos principais, a metodologia adotada e os principais resultados obtidos.

3.1 Learning both Weights and Connections for Efficient Neural Networks

Han et al. (2015) propõem reduzir o número de parâmetros de redes neurais profundas sem perda de acurácia, a fim de diminuir a memória e a energia exigidas na inferência. A motivação é energética: segundo os dados apresentados pelos autores, ler um peso de 32 bits da memória externa (DRAM) custa 640 pJ, cerca de três ordens de grandeza a mais do que uma operação aritmética simples, e uma rede que não cabe na memória interna do processador paga esse custo a cada inferência.

O método tem três passos. Primeiro, a rede é treinada normalmente, não para obter os pesos finais, mas para identificar quais conexões são importantes. Em seguida, removem-se as conexões cujos pesos têm valor absoluto abaixo de um limiar, proporcional ao desvio-padrão dos pesos de cada camada, o que torna a rede esparsa. Por fim, a rede é retreinada para que as conexões remanescentes compensem as removidas; poda e retreinamento podem ser repetidos iterativamente. O retreinamento mostrou-se decisivo: sem ele, a acurácia começa a cair quando resta cerca de um terço das conexões; com ele, apenas quando resta um décimo. Os autores medem ainda a sensibilidade de cada camada, podando uma de cada vez e observando a queda de acurácia. As camadas convolucionais revelaram-se mais sensíveis que as totalmente conectadas, e a primeira camada convolucional, que recebe a imagem diretamente, a mais sensível de todas. Esse resultado orientou a escolha do limiar de cada camada: na AlexNet, a primeira camada convolucional reteve 84% dos pesos, e as duas primeiras totalmente conectadas, 9%.

Na base ImageNet, o método reduziu os parâmetros da AlexNet em 9×, de 61 milhões para 6,7 milhões, e os da VGG-16 em 13×, de 138 milhões para 10,3 milhões, sem perda de acurácia. O consumo de energia, embora motive o trabalho, não é medido: os resultados são expressos em parâmetros e operações, e os autores direcionam o método a *hardware* especializado em redes esparsas, dada a limitação do *hardware* de propósito geral na computação esparsa.

Este trabalho adota o critério de Han et al. (2015) como primeira estratégia de

poda (Seção 5.2.2), com três diferenças. A poda é aplicada a um *Transformer*, e não a redes convolucionais. A energia é medida, o que permitiu verificar que, em *hardware* de propósito geral, a poda por magnitude não reduz o consumo de forma detectável (Seção 6.2), como antecipavam os próprios autores. E a análise de sensibilidade por camada, que em Han et al. (2015) orienta os limiares sem que uma regra seja descrita, torna-se aqui uma regra explícita de definição das taxas por camada, comparada aos esquemas de poda sem análise de sensibilidade sob a mesma taxa de poda (Seção 4.3). Também no GPT-2 a camada mais sensível é a primeira (Seção 6.3.2).

3.2 Pruning Filters for Efficient ConvNets

Li et al. (2017) partem de uma limitação da poda por magnitude: ela remove sobretudo parâmetros das camadas totalmente conectadas, que concentram os parâmetros, mas pouco do custo computacional — na VGG-16, essas camadas reúnem 90% dos parâmetros e menos de 1% das operações de ponto flutuante (FLOPs) —, e a esparsidade irregular que produz nas camadas convolucionais só reduz o tempo de inferência com bibliotecas de convolução esparsa. O objetivo dos autores é reduzir o custo de inferência de redes convolucionais com uma poda que dispense essas bibliotecas.

O método é uma poda estruturada: em vez de pesos isolados, remove filtros convolucionais inteiros, junto com os mapas de características que eles produzem e com os núcleos correspondentes da camada seguinte. A importância de cada filtro é a norma L_1 dos seus pesos, e removem-se os de menor norma — critério que, nos experimentos dos autores, preservou mais acurácia do que remover filtros aleatórios na maior parte das camadas. Como a rede resultante permanece densa, apenas menor, ela é executada pelas bibliotecas usuais de multiplicação de matrizes densas. A sensibilidade de cada camada é medida podando-a isoladamente e avaliando a acurácia no conjunto de validação; nas camadas sensíveis, poda-se uma fração menor de filtros, ou nenhum. Para evitar um hiperparâmetro por camada, as camadas de um mesmo estágio da rede recebem a mesma taxa, e todas são podadas de uma só vez, seguidas de um único retreinamento.

Na base CIFAR-10, o método reduziu os FLOPs da VGG-16 em 34,2% e seus parâmetros em 64%, e os FLOPs da ResNet-110 em 38,6%, com erro de teste próximo ao da rede original após o retreinamento: 6,60% contra 6,75% na VGG-16 e 6,70% contra 6,47% na ResNet-110. A redução de FLOPs converteu-se em redução semelhante do tempo de inferência, de 40,7% na VGG-16. Os experimentos mostraram ainda que camadas sensíveis são difíceis de recuperar pelo retreinamento e que excluí-las da poda melhora o resultado. O consumo de energia não é medido.

Este trabalho adapta o método de Li et al. (2017) ao *Transformer* como segunda estratégia de poda: as unidades estruturais passam a ser as cabeças de atenção e os

neurônios do MLP, ordenados pela norma L_1 (Seção 5.2.3). Como em Li et al. (2017), a sensibilidade de cada camada é medida podando-a isoladamente; o perfil obtido, porém, define as taxas por camada por meio de uma regra explícita, que preserva a taxa de poda do modelo e substitui a escolha empírica por estágio. Além disso, o custo é medido também em energia, e a poda é avaliada tanto sem retreinamento quanto após o *fine-tuning* de recuperação.

3.3 Designing Energy-Efficient Convolutional Neural Networks using Energy-Aware Pruning

Yang, Chen e Sze (2017) propõem um algoritmo de poda para redes neurais convolucionais (CNNs) cujo foco é minimizar o consumo real de energia, e não apenas o número de parâmetros ou de operações, preservando a acurácia. Os autores consideram especialmente o custo computacional associado a operações de multiplicação e acumulação (MAC) e o custo de acesso à memória em dispositivos periféricos, como *smartphones*.

O método proposto aplica poda camada por camada, priorizando aquelas cujo consumo energético estimado é maior. Para isso, os autores desenvolvem um *framework* de estimativa que ordena as camadas pela combinação de custo computacional (número de MACs) e custo de acesso à memória. Em cada camada podada, parte dos pesos é removida; em seguida, alguns pesos são restaurados para minimizar o erro no mapa de características de saída; por fim, os pesos remanescentes são ajustados localmente por meio de uma solução de mínimos quadrados. Após o processamento de todas as camadas, a rede passa por um *fine-tuning* global utilizando *backpropagation*.

Os autores reportam redução de 3,7× no consumo de energia da AlexNet e 1,6× no da GoogLeNet, com perda de acurácia inferior a 1%. O trabalho também demonstra empiricamente que as camadas convolucionais são as que mais consomem energia, principalmente devido ao volume de acessos à memória, e que a simples redução do número de classes tem impacto limitado na redução do consumo energético, uma vez que as camadas convolucionais não são significativamente afetadas.

A fazer: adicionar resenhas de outros trabalhos fundamentais para esta pesquisa. Sugestões: Frankle e Carbin (2019) (Lottery Ticket Hypothesis); Blalock et al. (2020) (revisão crítica de métodos de poda); Liu et al. (2019) (*Rethinking the Value of Network Pruning*). Para cada um, seguir o mesmo padrão de resenha: objetivo, método, resultados principais.

4 Metodologia

Este capítulo apresenta a metodologia adotada neste trabalho. A Seção 4.1 descreve os procedimentos utilizados para selecionar o modelo base, as bases de dados e as métricas empregadas na avaliação dos modelos. A Seção 4.2 apresenta as estratégias clássicas de poda aplicadas ao modelo base. A Seção 4.3 descreve a estratégia de poda otimizada proposta neste trabalho, baseada na definição de taxas de poda variáveis para as diferentes camadas do modelo. Por fim, a Seção 4.4 apresenta a estratégia de retreinamento dos modelos podados, com o objetivo de avaliar em que medida a degradação de desempenho decorrente da poda pode ser revertida.

Este capítulo se limita à descrição metodológica das estratégias adotadas, apresentando os procedimentos gerais e os critérios que orientam as decisões. As escolhas específicas de implementação, incluindo o modelo base, as bases de dados, as configurações de poda e os parâmetros de retreinamento, são detalhadas no Capítulo 5, que segue a mesma estrutura deste capítulo. Essa divisão tem como finalidade distinguir as decisões metodológicas, que definem os procedimentos e critérios da investigação, das decisões de implementação, que especificam como esses procedimentos são concretizados.

A metodologia foi definida para responder a duas perguntas de natureza empírica e comparativa. Não se pretende demonstrar que a poda comprime modelos, resultado já consolidado pela literatura (HAN et al., 2015; LI et al., 2017; BLALOCK et al., 2020), mas responder, sob condições controladas e reprodutíveis, às seguintes questões:

(A) Em que medida a redução *teórica* de parâmetros se converte em redução *real* de tempo de inferência e de consumo de energia, e como essa conversão depende da granularidade da poda?

(B) Um esquema de poda, isto é, um conjunto de taxas de poda por camada, definido por uma análise de sensibilidade por camada melhora o compromisso entre compressão e qualidade em relação aos esquemas de poda sem análise de sensibilidade?

Responder a essas duas perguntas exige que modelos comprimidos por estratégias distintas sejam avaliados exatamente pelo mesmo conjunto de métricas, na mesma base de dados e no mesmo *hardware*. Essa simetria de avaliação é condição necessária para que qualquer comparação seja interpretável e orienta todas as etapas descritas a seguir.

4.1 Modelo Base, Base de Dados e Métricas

Esta seção apresenta os procedimentos adotados para a seleção do modelo base, das bases de dados e das métricas utilizadas na avaliação dos modelos testados. O modelo base fornece os valores iniciais das métricas e constitui o ponto de partida para a geração dos modelos podados. As bases de dados são utilizadas para avaliar o desempenho dos modelos, enquanto as métricas permitem quantificar e comparar os resultados obtidos de acordo com os objetivos do trabalho.

4.1.1 Seleção do Modelo

Nesta primeira etapa, seleciona-se o modelo base (*baseline*), que é a versão original de um modelo de linguagem de grande escala (*Large Language Model* — LLM), antes de qualquer processo de poda. O modelo é executado e avaliado sobre uma base de dados, registrando-se métricas de desempenho preditivo, de custo computacional e de consumo de energia. Os valores obtidos constituem o referencial de comparação de todo o trabalho.

Para a seleção do modelo base, foram adotados os seguintes critérios:

- **C1 — Arquitetura:** ser um *Transformer* exclusivamente decodificador, treinado de forma autorregressiva para predizer o próximo *token*. Esse critério delimita a classe arquitetural de interesse, representada por modelos generativos de pesos abertos, como OPT e Llama 2 (ZHANG et al., 2022; TOUVRON et al., 2023). Além disso, essa arquitetura deve ser compatível com a métrica *perplexity*.
- **C2 — Disponibilidade de pesos pré-treinados:** disponibilização de pesos pré-treinados para *download* direto, sob licença que permita o uso acadêmico. Esse critério dispensa o pré-treinamento dos modelos, atividade fora do escopo deste trabalho devido aos custos computacionais e energéticos envolvidos. A dimensão desses custos é ilustrada pelo pré-treinamento da família GPT-2, que utilizou o WebText, uma base composta por mais de 8 milhões de documentos e aproximadamente 40 GB de texto (RADFORD et al., 2019), e apresentou elevado custo computacional e energético (STRUBELL; GANESH; MCCALLUM, 2019).
- **C3 — Viabilidade do *fine-tuning*:** a memória necessária para o *fine-tuning* completo do modelo deve ser compatível com a capacidade da GPU disponível. O treinamento com o otimizador Adam em precisão mista exige, como estimativa mínima, 16 bytes por parâmetro para armazenar pesos, gradientes e estados do otimizador (RAJBHANDARI et al., 2020). Além disso, as ativações também ocupam memória, o que torna essa estimativa um limite inferior. No ambiente adotado, a GPU NVIDIA Tesla T4, disponibilizada pela plataforma Kaggle, possui 16 GB de memória (NVIDIA Corporation, 2019). Assim, esse critério restringe os candidatos

a modelos com menos de aproximadamente 1 bilhão de parâmetros, considerando que a memória necessária para armazenar as ativações reduz esse limite.

- **C4 — Modelo original:** não ter sido submetido previamente a processos de compressão, como destilação, poda ou quantização. Esse critério permite atribuir à poda aplicada neste trabalho a degradação observada no modelo.
- **C5 — Resultado de referência:** possuir valores de *perplexity* publicados em artigo científico para a base de dados de avaliação adotada. Esses valores, apresentados pelos autores do modelo ou por trabalhos posteriores, permitem comparar o modelo base reproduzido com os resultados disponíveis na literatura.
- **C6 — Implementação disponível:** possuir arquitetura implementada nativamente na biblioteca `transformers` (WOLF et al., 2020), o que dispensa a implementação própria do modelo e permite utilizar a infraestrutura da biblioteca PyTorch.

Nota-se que o critério C5 depende da seleção da base de dados de avaliação. Além disso, a seleção da base de dados também depende do modelo, conforme descrito na Seção 4.1.2. Dessa forma, a seleção do modelo e da base de dados são interdependentes e devem ser realizadas em conjunto.

Quando mais de um candidato atende simultaneamente a todos os critérios, seleciona-se aquele com o menor número de parâmetros. Essa escolha considera o custo computacional das varreduras experimentais (Seção 4.2.2), que exigem numerosas avaliações completas do modelo e cujo custo aumenta com o tamanho do modelo. O resumo da aplicação desses critérios ao conjunto de modelos candidatos é apresentado na Tabela 2.

4.1.2 Seleção da Base de Dados

O modelo base e todos os modelos podados são avaliados sobre a mesma base de dados, composta pela coleção de textos utilizada para medir o desempenho dos modelos. A seleção dessa base considera os seguintes critérios:

- **D1 — Resultado de referência:** possuir valores de *perplexity* publicados em artigo científico para o modelo base avaliado na base de dados selecionada. Esse critério corresponde à exigência de C5 sob a perspectiva da base de dados.
- **D2 — Preservação das características do texto:** preservar caixa, pontuação, números, palavras raras e continuidade dos documentos. Esse critério considera que modelos com tokenização por subpalavras são pré-treinados sobre texto bruto e, portanto, devem ser avaliados em textos que preservem essas características, evitando alterações na distribuição dos dados.

- **D3 — Divisão oficial:** possuir partições oficiais de treino, validação e teste. Esse critério permite utilizar a partição de treino no *fine-tuning* e a partição de teste na avaliação, dispensando a criação de divisões arbitrárias.
- **D4 — Ausência de sobreposição com o pré-treinamento:** não conter dados utilizados no pré-treinamento do modelo, evitando que a *perplexity* reflita, ainda que parcialmente, a memorização dos textos utilizados durante o treinamento.

Observa-se que os critérios D1 e D4 dependem tanto da base de dados quanto do modelo, o que justifica a seleção conjunta. Em caso de empate na aplicação desses critérios, seleciona-se a base de dados com o menor número de *tokens* no conjunto de treino, pois essa quantidade influencia o custo computacional do *fine-tuning*. Um resumo dos candidatos é apresentado na Tabela 4.

4.1.3 Métricas de Avaliação

As métricas de avaliação são organizadas em três dimensões: desempenho preditivo, custo computacional e consumo de energia. Todas são obtidas pelo mesmo procedimento e sobre a mesma base de dados, tanto para o modelo base quanto para os modelos podados nas Etapas 2 a 4. Para cada métrica, são apresentados seu significado e o procedimento utilizado para obtê-la.

4.1.3.1 Desempenho preditivo

Como o modelo base é autorregressivo, adota-se a métrica *perplexity* para avaliar seu desempenho preditivo, por ser amplamente utilizada nessa classe de modelos. Essa métrica quantifica o grau de "surpresa" do modelo diante dos *tokens* do conjunto de avaliação e permite avaliar a qualidade de sua modelagem probabilística da linguagem (GOODFELLOW; BENGIO; COURVILLE, 2016). Valores menores de *perplexity* indicam melhor desempenho preditivo.

A adoção da *perplexity*, em vez de métricas de tarefas descendentes, como a acurácia em uma tarefa de classificação, deve-se ao fato de ser uma métrica intrínseca, calculada diretamente sobre a base de dados de avaliação, sem exigir o *fine-tuning* do modelo em uma tarefa específica. Esse procedimento evita que a comparação entre as configurações dependa de um treinamento adicional. O procedimento de cálculo é descrito no Capítulo 5.

4.1.3.2 Custo computacional

O custo computacional é avaliado por meio de quatro indicadores complementares, descritos a seguir:

- **Número de parâmetros remanescentes:** contagem dos elementos não nulos dos tensores de pesos, no caso da poda não estruturada, ou dos parâmetros existentes após a remoção de estruturas, no caso da poda estruturada;
- **Operações de ponto flutuante (FLOPs):** contagem das operações matemáticas necessárias para uma inferência completa, obtida por ferramentas de contagem, que percorrem os módulos do modelo durante uma inferência e acumulam as operações correspondentes. Essa automação, entretanto, é parcial, pois abrange apenas os tipos de módulo previstos no catálogo da ferramenta. Módulos não contemplados exigem regras de contagem próprias, caso contrário, são ignorados. O tratamento desses módulos no modelo adotado e as omissões remanescentes são detalhados no Capítulo 5;
- **Tempo de inferência:** tempo decorrido durante o processamento de uma entrada representativa, medido empiricamente em condições controladas de *hardware*. Para reduzir os efeitos da variabilidade natural da execução, são realizadas múltiplas medições, cujos resultados são expressos por meio da média e do desvio-padrão;
- **Tamanho do modelo:** memória necessária para armazenar os parâmetros, calculada pelo produto entre o número de parâmetros e a quantidade de *bytes* utilizada para representar cada um.

4.1.3.3 Consumo de energia

O consumo de energia corresponde à energia total consumida durante o intervalo de inferência, estimada por ferramentas de monitoramento do *hardware* (CPU e GPU), que integram leituras periódicas do consumo elétrico dos componentes. O resultado é expresso em Wh.

As emissões equivalentes de CO_2 não são adotadas como métrica, pois não acrescentam informação à comparação entre os modelos. Essas emissões resultam da multiplicação da energia consumida por um fator de intensidade carbônica, que indica quanto CO_2 é emitido para gerar cada kWh na rede elétrica do local de execução e não é controlado pelo experimento. Quando o fator é o mesmo, a redução relativa das emissões coincide com a redução relativa de energia: se a poda reduz o consumo em 10%, as emissões também diminuem 10%. Quando os fatores diferem, a comparação deixa de refletir o modelo: um mesmo consumo de energia resulta em emissões diferentes apenas porque a rede elétrica mudou.

4.1.3.4 Comparação entre Modelos Podados

Cada modelo podado corresponde a uma **configuração**, definida pela estratégia de poda, pelo esquema de poda (Seção 4.3) e pela taxa de poda. As configurações são

comparadas de forma pareada, isto é, sempre entre modelos obtidos com a mesma taxa de poda: o modelo podado a 30% com um esquema de poda, por exemplo, é comparado ao modelo podado a 30% com outro esquema, e nunca ao de outra taxa. As taxas avaliadas são dez, de 0% a 90%, em intervalos de 10 pontos percentuais (Seção 4.2.2). Em cada uma delas, consideram-se a *perplexity* e o consumo energético de cada esquema de poda em relação ao modelo base e aos demais esquemas. Essa comparação é sintetizada por três indicadores, seguindo a recomendação de considerar a eficiência como critério de avaliação, além do desempenho preditivo (SCHWARTZ et al., 2020), e de relatar sistematicamente o consumo energético (HENDERSON et al., 2020). A definição explícita desses indicadores também busca reduzir os problemas decorrentes da falta de padronização das métricas, que dificulta a comparação entre métodos de poda (BLALOCK et al., 2020).

Redução relativa de energia (ΔE): percentual de energia economizada pelo modelo podado em relação ao modelo base, calculado por:

$$\Delta E = \frac{E_0 - E}{E_0} \times 100 \ [\%], \tag{4.1}$$

em que E representa a energia consumida pelo modelo podado e E_0, a energia consumida pelo modelo base na mesma sessão de execução. Essa referência é necessária porque o consumo energético de um mesmo modelo pode variar entre sessões do ambiente de execução. Por essa razão, a comparação é realizada em termos relativos, e não por meio de valores absolutos.

Eficiência relativa (η): relaciona o desempenho preditivo à energia consumida. Como valores menores de *perplexity* indicam melhor desempenho, a razão entre as métricas é calculada de forma inversa em relação ao modelo base:

$$\eta = \frac{\mathrm{PPL}_0}{\mathrm{PPL}} \times \frac{E_0}{E}, \tag{4.2}$$

em que PPL e PPL_0 representam as *perplexities* do modelo podado e do modelo base, respectivamente. O modelo base apresenta $\eta = 1$, enquanto valores superiores a 1 indicam maior desempenho preditivo por unidade de energia. Por utilizar apenas razões, esse indicador mantém a comparação energética em termos relativos. Entretanto, quando a diferença de consumo energético está abaixo do limiar de detecção do protocolo (Seção 6.2), η passa a refletir predominantemente a variação da *perplexity* e variações abaixo desse limiar.

Ponto de eficiência (S_E^*): maior taxa de poda para a qual a *perplexity* não apresenta aumento superior a 5% em relação à do modelo base:

$$S_E^* = \max \left\{ s \mid \mathrm{PPL}(s) \leq 1{,}05 \cdot \mathrm{PPL}_0 \right\}, \tag{4.3}$$

em que s representa a taxa de poda. A comparação utiliza como referência o modelo base no mesmo regime: sem retreinamento, considera-se o modelo pré-treinado; após o

fine-tuning, considera-se o modelo base submetido ao mesmo retreinamento. O limite de 5% não foi ajustado em função dos valores obtidos.

Resumo por varredura: para comparar estratégias e esquemas de poda por um único valor, calcula-se a média dos resultados nas taxas de poda consideradas de cada varredura (Seção 4.2.2). Para a *perplexity*, utiliza-se a média geométrica, pois, nas avaliações sem retreinamento, seus valores variam por ordens de grandeza entre as taxas. Nesse cenário, a média aritmética seria fortemente influenciada pelos resultados das maiores taxas de poda, enquanto a média geométrica equivale à média da degradação logarítmica, utilizada na análise de sensibilidade. Para ΔE, utiliza-se a média aritmética, e η é calculado a partir das duas médias. São consideradas apenas as taxas até a maior em que nenhuma camada atinge o teto do esquema de poda com análise de sensibilidade (Seção 4.3). Acima desse limite, o esquema deixa de diferenciar as camadas, comprometendo a comparação entre os esquemas de poda. Essas taxas são reportadas, mas não entram no cálculo das médias. Esse resumo aplica-se às varreduras sem retreinamento, que abrangem todas as taxas de poda. As configurações submetidas ao *fine-tuning*, avaliadas em um número menor de taxas, são comparadas apenas de forma pareada, em cada taxa avaliada.

4.2 Poda sem Otimização

Na segunda etapa, aplicam-se estratégias clássicas de poda ao modelo base, sem o uso de técnicas de otimização. Cada estratégia é aplicada independentemente para cada taxa de poda, sempre a partir do modelo base original, e não de modelos previamente podados. Dessa forma, os resultados de cada configuração não dependem da aplicação das estratégias anteriores. Os modelos podados são avaliados pelas mesmas métricas e sobre a mesma base de dados descritos na Seção 4.1. As métricas de cada modelo podado, nesta e nas etapas seguintes, são comparadas às do modelo base, o que permite mensurar a perda de desempenho e o ganho de eficiência introduzidos pela compressão.

As estratégias de poda sem otimização buscam reduzir o número de parâmetros de um modelo pré-treinado por meio da remoção de pesos ou estruturas, selecionados por um critério de poda (Seção 2.6).

A **taxa de poda** corresponde à fração de elementos removidos do modelo (por exemplo, 30% ou 50%). O critério de poda determina como essas remoções são distribuídas entre as camadas, podendo ser aplicado de duas formas. No **esquema de poda uniforme**, a mesma taxa de poda é aplicada a todas as camadas, e o critério atua apenas localmente para selecionar os elementos removidos dentro de cada uma delas. No **esquema de poda não uniforme**, os elementos de todo o modelo são ordenados conjuntamente. Assim, a taxa de poda varia de uma camada para outra, concentrando as

remoções naquelas que possuem maior proporção de elementos de menor pontuação.

4.2.1 Parâmetros Podáveis e Não Podáveis

A taxa de poda não é absoluta: ela é relativa aos **parâmetros podáveis**, o conjunto de parâmetros que as estratégias de poda podem remover, e não ao total de parâmetros do modelo. Os demais, denominados **não podáveis**, permanecem intactos em todas as configurações, qualquer que seja a estratégia ou a taxa de poda.

Em modelos baseados na arquitetura *Transformer*, adotam-se como podáveis, por convenção consolidada na literatura de poda de LLMs (FRANTAR; ALISTARH, 2023; SUN et al., 2024), apenas as matrizes de peso das projeções lineares internas às camadas: as da atenção multi-cabeça, que produzem a consulta, a chave e o valor e projetam a saída das cabeças (Seção 2.3.3), e as das duas transformações lineares do MLP (Seção 2.3.4). É sobre essas matrizes que incidem as multiplicações de matriz das camadas, responsáveis pela maior parte do custo de inferência — no GPT-2 *small*, por exemplo, 87,0 dos 126,6 GFLOPs do modelo (Seção 5.1.3.2). Nelas estão também os elementos podáveis da definição de taxa de poda: os pesos individuais são valores dessas matrizes, e as estruturas, como as cabeças de atenção e os neurônios do MLP, são fatias delas.

Ficam fora do conjunto podável os demais parâmetros, cada grupo por uma razão própria:

- as matrizes de *embedding* de *token* e de posição (Seção 2.3.5). Seu uso não envolve multiplicação de matrizes: a cada *token*, o modelo apenas copia as linhas correspondentes ao *token* e à sua posição. Anular valores dessas matrizes reduziria o armazenamento, mas não o custo computacional, cuja redução é o objetivo deste trabalho. Elas tampouco contêm cabeças de atenção ou neurônios do MLP: remover uma linha eliminaria um *token* do vocabulário ou uma posição da sequência, e remover uma coluna alteraria a dimensão das representações, que é a dimensão de entrada e de saída de todas as camadas;
- a cabeça de linguagem, matriz que converte a representação entregue pela última camada em uma pontuação para cada *token* do vocabulário. Diferentemente dos *embeddings*, ela efetua multiplicação de matrizes, mas também não contém as unidades da poda estruturada: cada uma de suas linhas corresponde a um *token*. Além disso, em modelos que reutilizam na cabeça a matriz de *embedding* de *token*, como o GPT-2, um peso anulado na cabeça seria anulado também no *embedding*, e a poda alteraria simultaneamente a entrada e a saída do modelo, o que impediria atribuir seu efeito a uma única camada, premissa da análise de sensibilidade por camada (Seção 4.3);

- os vieses e os parâmetros de normalização de camada. São uma fração ínfima do modelo — menos de 0,1% no GPT-2 *small* — e efetuam apenas somas e produtos elemento a elemento, de custo desprezível diante das multiplicações de matriz: mesmo eliminados por inteiro, não reduziriam o custo de forma mensurável. O risco, em contrapartida, é alto: um parâmetro de normalização multiplica uma dimensão inteira da representação que entra na atenção ou no MLP, enquanto um peso de matriz é apenas uma entre centenas ou milhares de parcelas de uma soma. Kovaleva et al. (2021) mostraram que anular os parâmetros de normalização associados a uma única dimensão — menos de 0,0001% dos pesos do modelo — degrada significativamente o desempenho de modelos *Transformer* pré-treinados, efeito também identificado no GPT-2. Os vieses não são removidos isoladamente; na poda estruturada, o viés de uma estrutura removida sai junto com ela.

Como os parâmetros não podáveis permanecem no modelo, a fração do total de parâmetros removida é sempre menor que a taxa de poda. No GPT-2 *small*, por exemplo, os *embeddings* reúnem 31,6% dos 124,4 milhões de parâmetros, e o conjunto podável, cerca de 85 milhões, 68,3% do total. Uma taxa de poda de 50% remove, assim, cerca de 42,5 milhões de parâmetros, metade do conjunto podável, e não 62,2 milhões, metade do modelo: a redução no total de parâmetros é de 34%, e não de 50%. Por essa razão, a fração removida do conjunto podável e o número de parâmetros remanescentes no modelo inteiro (Seção 4.1.3) são reportados separadamente.

O conjunto podável é o mesmo para todas as estratégias de poda, o que mantém comparáveis as suas taxas: uma taxa de 50%, em qualquer estratégia, incide sobre as mesmas matrizes. A identificação dos parâmetros podáveis no modelo selecionado é apresentada no Capítulo 5 (Seção 5.2.2 e Figura 9).

4.2.2 Taxas de Poda Avaliadas

Cada estratégia de poda é avaliada, com cada esquema de poda, em dez taxas de poda igualmente espaçadas: 0%, 10%, 20% e assim por diante, até 90%. O conjunto das dez avaliações de uma mesma estratégia com um mesmo esquema de poda é denominado, ao longo do texto, **varredura**: as dez avaliações de uma estratégia com o esquema de poda uniforme, por exemplo, formam a varredura uniforme dessa estratégia. Os esquemas de poda com análise de sensibilidade da Etapa 3 (Seção 4.3) são avaliados nas mesmas taxas, o que permite comparar todos os esquemas de poda de uma estratégia de forma pareada, como prevê a Seção 4.1.3.

Avalia-se toda a faixa, de 10% a 90%, porque não se sabe de antemão a partir de qual taxa de poda cada estratégia passa a degradar o modelo de forma acentuada, e não há razão para supor que essa taxa seja a mesma em estratégias de granularidades

diferentes (Seção 2.6), que removem elementos de tamanhos muito diferentes. A poda de granularidade fina remove pesos individuais; a de granularidade grossa remove estruturas inteiras, cada uma formada por muitos pesos — no GPT-2 *small*, por exemplo, um neurônio do MLP reúne 1.536 pesos, e uma cabeça de atenção, 196.608 (Seção 5.2.3). Avaliar apenas parte da faixa, como somente as taxas baixas, poderia deixar de fora justamente a taxa em que a degradação de uma delas se acentua.

A taxa de 0% corresponde ao modelo base, sem nenhuma poda, e é incluída em cada varredura com duas funções. A primeira é de verificação: avaliado pelo mesmo procedimento das demais taxas, o modelo base deve reproduzir a *perplexity* medida na Etapa 1 (Seção 4.1), e uma divergência indicaria erro no procedimento de poda ou de avaliação, e não efeito da poda. A segunda é de referência: a energia medida a 0% é a energia E_0 do modelo base na mesma sessão de execução das demais taxas da varredura, contra a qual se calcula a redução relativa de energia (Equação 4.1).

Por fim, como estabelecido no início desta seção, cada taxa parte do modelo base original, e não do modelo podado na taxa anterior: a configuração de 30%, por exemplo, é obtida removendo 30% dos elementos podáveis do modelo base, e não acrescentando 20 pontos percentuais à poda da configuração de 10%. A aplicação dessas taxas às estratégias selecionadas é apresentada no Capítulo 5 (Seção 5.2.5).

4.2.3 Seleção das Estratégias

As estratégias desta etapa não constituem a contribuição principal deste trabalho. Elas são aplicadas segundo os critérios de poda propostos por seus respectivos autores, e os modelos resultantes servem de referência para avaliar a estratégia com otimização (Seção 4.3), que representa a contribuição central do trabalho.

A escolha das estratégias, entre as alternativas disponíveis na literatura, baseou-se em três critérios:

- **P1 — Critério de poda restrito aos pesos:** a seleção dos elementos a serem removidos não deve empregar gradientes ou dados de calibração. A incorporação de informações baseadas em dados introduziria um viés experimental, impossibilitando distinguir se o desempenho final decorre da eficácia da técnica de poda avaliada ou do simples acesso a esses dados. Neste trabalho, a informação dependente de dados entra na decisão de poda apenas pela análise de sensibilidade por camada (Seção 4.3), cuja contribuição se pretende medir.
- **P2 — Aplicação *one-shot*:** a poda é aplicada diretamente ao modelo pré-treinado, sem treinamento durante o processo. Dessa forma, o efeito da remoção de

parâmetros é avaliado separadamente daquele decorrente do *fine-tuning* de recuperação (Seção 4.4).

- **P3 — Cobertura das granularidades:** o conjunto de estratégias contempla duas granularidades de poda: fina, que remove parâmetros individuais, e grossa, que remove unidades arquiteturais inteiras.

O critério P3 se justifica pelas vantagens contrastantes oferecidas pelas duas granularidades (Figura 4). De um lado, a poda de granularidade fina geralmente alcança taxas de compressão elevadas com perda reduzida de desempenho preditivo (BLALOCK et al., 2020). Entretanto, os pesos removidos são apenas anulados: as matrizes conservam suas dimensões, e o *hardware* convencional continua a executar multiplicações por zero. Assim, a redução do número de parâmetros não se traduz necessariamente em redução do tempo de inferência ou do consumo de energia. Por sua vez, a poda de granularidade grossa produz matrizes menores e densas, reduzindo efetivamente o custo de inferência em *hardware* convencional (LI et al., 2017), mas pode provocar maior perda de desempenho preditivo para uma mesma taxa de compressão (BLALOCK et al., 2020). Portanto, enquanto a primeira tende a preservar melhor o desempenho preditivo, a segunda permite converter a compressão em economia efetiva de tempo e energia. Avaliar ambas sob as mesmas condições permite quantificar essa relação de compromisso (*trade-off*), que não poderia ser caracterizada pela análise isolada de cada granularidade e constitui uma das questões centrais deste trabalho, a pergunta (A), apresentada no início deste capítulo.

A aplicação desses critérios ao conjunto de estratégias candidatas, bem como a apresentação das estratégias selecionadas e dos detalhes de sua implementação, são descritas no Capítulo 5 (Tabela 6).

4.3 Poda com Otimização

Esta é a etapa central do trabalho. Para cada taxa de poda, é necessário determinar quanto remover de cada camada do modelo. Esta etapa investiga se essa decisão pode ser aprimorada ao medir a tolerância de cada camada à remoção de parâmetros, em vez de assumir uma distribuição uniforme ou previamente definida.

4.3.1 O Problema: Quanto Podar em Cada Camada

A fração de elementos removidos de uma camada corresponde à sua **taxa de poda por camada**. A taxa de poda do modelo, definida na Etapa 2, corresponde à média das taxas por camada, ponderada pelo número de elementos de cada camada quando seus tamanhos são diferentes. O conjunto de taxas de poda por camada utilizado para atingir determinada taxa de poda do modelo é denominado **esquema de poda**.

Uma mesma taxa de poda pode ser obtida por diferentes esquemas de poda. Em um modelo com quatro camadas de mesmo tamanho, aplicar uma taxa de poda de 30% a cada camada e distribuir as taxas em 20%, 50%, 10% e 40% nas camadas 0 a 3 são dois esquemas de poda distintos para a mesma taxa de poda do modelo: ambos removem o mesmo número de elementos, mas diferem quanto às camadas das quais eles são removidos. Como as camadas podem apresentar diferentes níveis de tolerância à remoção, esses esquemas podem produzir modelos com desempenhos distintos.

Denomina-se **sensível** a camada cuja poda provoca grande degradação do desempenho preditivo do modelo e **tolerante** à poda aquela cuja remoção de elementos provoca pouca degradação.

Os dois esquemas de poda apresentados na Seção 4.2 compartilham uma limitação: nenhum mede diretamente a sensibilidade de cada camada à poda. O esquema de poda uniforme desconsidera essas diferenças e trata as camadas como intercambiáveis. O esquema de poda não uniforme, por sua vez, infere a sensibilidade a partir dos pesos, fornecendo uma aproximação, mas não uma medida direta. A Figura 6 ilustra a diferença entre o esquema de poda uniforme e aquele orientado pela sensibilidade das camadas.

4.3.2 Hipótese e Desenho Experimental

A hipótese central deste trabalho é que a poda mais intensa das camadas tolerantes e menos intensa das camadas sensíveis resulte em menor perda de desempenho preditivo, para uma mesma taxa de poda, em comparação com os esquemas de poda uniforme e não uniforme. Para testá-la, mantém-se fixa a taxa de poda e varia-se apenas o esquema de poda. Os modelos resultantes são comparados segundo as métricas e o critério definidos na Seção 4.1.3. Como a taxa de poda permanece constante, o objetivo da técnica não é ampliar a compressão, mas minimizar a perda de desempenho preditivo associada à remoção de parâmetros.

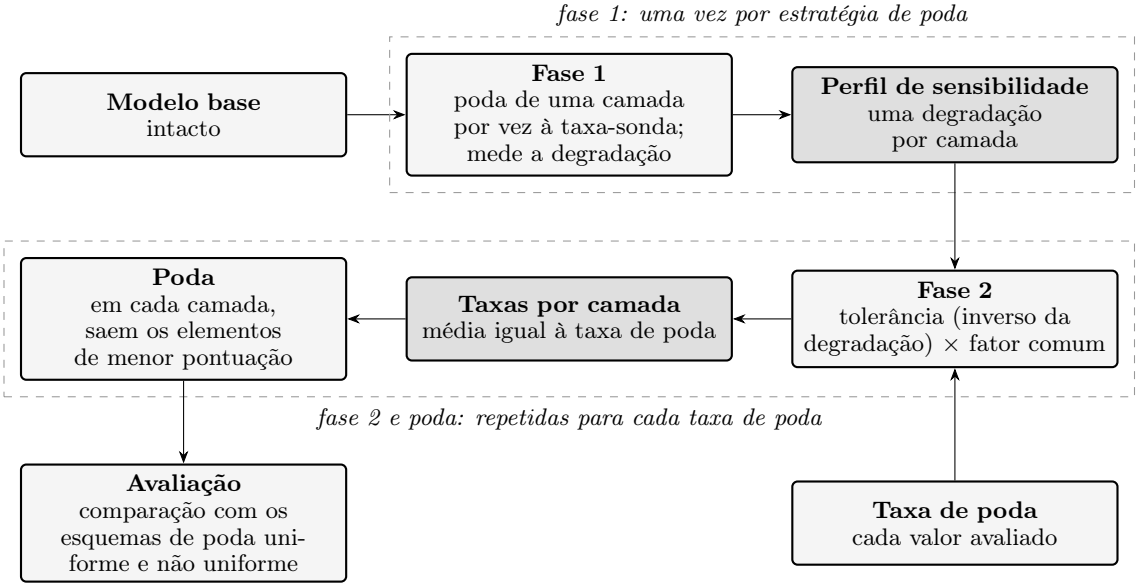
Mantidos constantes o modelo, o critério de poda, a taxa de poda e a ausência de treinamento, as diferenças de desempenho entre os modelos podem ser atribuídas à distribuição da poda entre as camadas.

4.3.3 Técnica Adotada: Análise de Sensibilidade por Camada

A técnica de otimização adotada neste trabalho é a **análise de sensibilidade por camada**, que consiste em medir a sensibilidade de cada camada à poda para orientar a distribuição das remoções, priorizando as camadas mais tolerantes e preservando as mais sensíveis. Essa análise produz o **perfil de sensibilidade**, que expressa a degradação do desempenho preditivo associada à poda de cada camada.

A técnica compreende duas fases (Figura 7). A primeira mede o perfil de sensibilidade e é executada uma única vez para cada estratégia de poda. A segunda utiliza esse perfil para determinar o esquema de poda e é repetida para cada taxa de poda avaliada, sem necessidade de realizar novas medições.

Figura 7 – Fluxo da técnica de análise de sensibilidade por camada.



Fonte: Elaboração própria.

Na Figura 7, a caixa tracejada superior corresponde à fase 1, executada uma vez por estratégia de poda; a inferior, à fase 2 e à poda, repetidas para cada taxa de poda a partir do mesmo perfil.

Para uma determinada estratégia e taxa de poda, o procedimento compreende cinco passos.

1. **Medir o perfil (fase 1).** Para cada camada, parte-se do modelo base e remove-se, exclusivamente daquela camada, uma fração dos elementos denominada **taxa-sonda**. Em seguida, mede-se a *perplexity* (Seção 4.1.3) e compara-se o resultado com o do modelo base. A **degradação** da camada corresponde ao aumento observado nessa métrica. Antes de avaliar a camada seguinte, o modelo é restaurado ao seu estado original. A taxa-sonda é mantida constante para todas as camadas, garantindo a comparabilidade das degradações. Essa taxa é utilizada exclusivamente para a medição e não gera modelos finais. O perfil de sensibilidade é medido uma única vez por estratégia e reutilizado em todas as taxas de poda.

   O valor da taxa-sonda é fixado antes da medição, segundo dois limites. Ele deve ser alto o bastante para que as degradações das camadas se diferenciem claramente entre si: com uma taxa-sonda muito baixa, cada camada seria pouco alterada, as degradações ficariam todas pequenas e próximas, e a ordenação das camadas

passaria a depender de diferenças mínimas. E não deve superar 50%, pois, acima desse valor, cada camada perde mais da metade de seus elementos, e a medição se aproxima da remoção da camada inteira, granularidade mais grossa que a das estratégias avaliadas. Como nenhum valor dentro desses limites é preferível de antemão, a estabilidade do perfil é verificada repetindo-se a medição a uma segunda taxa-sonda e comparando-se as ordenações das camadas obtidas nas duas medições: se a ordenação se mantém, ela reflete as camadas, e não a taxa-sonda escolhida.

2. **Converter degradação em tolerância.** A **tolerância à poda**, ou simplesmente tolerância, de uma camada é definida como o inverso de sua degradação: quanto menor a degradação provocada pela poda, maior sua tolerância. Esses valores determinam apenas as proporções entre as taxas de poda por camada, e não seus valores absolutos.

3. **Escalar à taxa de poda.** Todas as tolerâncias são multiplicadas por um mesmo fator, determinado de modo que a média ponderada das taxas resultantes corresponda à taxa de poda pretendida. Essas taxas definem o **esquema de poda com análise de sensibilidade**. Um teto limita a taxa de poda de cada camada, impedindo sua remoção integral. Quando uma camada atinge esse limite, o excedente é redistribuído entre as demais. A regra também admite um parâmetro que controla a intensidade com que as diferenças observadas no perfil de sensibilidade são refletidas no esquema de poda.

   Enquanto nenhuma camada atinge o teto, o fator é obtido diretamente, pela divisão da taxa de poda pretendida pela média ponderada das tolerâncias, já ajustadas pelo parâmetro de intensidade. Quando alguma camada atinge o teto, essa divisão deixa de valer, pois a taxa dessa camada fica fixada no teto e o excedente redistribuído altera a média; o fator é então determinado por tentativas sucessivas, ajustado até que a média ponderada das taxas resultantes coincida com a taxa de poda pretendida.

4. **Poda.** Em cada camada, o critério de poda da estratégia seleciona os elementos de menor pontuação para remoção, até atingir a taxa de poda definida para aquela camada. Assim, a análise de sensibilidade determina *quanto* podar em cada camada, enquanto a estratégia de poda determina *quais* elementos remover.

5. **Avaliar e comparar.** O modelo podado é avaliado segundo as métricas da Seção 4.1.3 e comparado aos modelos obtidos pelos esquemas de poda uniforme e não uniforme, mantendo-se a mesma taxa de poda.

Os passos 2 a 5 são repetidos para cada taxa de poda avaliada, enquanto o passo 1 é executado uma única vez por estratégia de poda.

Exemplo

A Tabela 1 ilustra um exemplo numérico dos passos 1 a 3 da análise de sensibilidade em um modelo hipotético de quatro camadas de mesmo tamanho, à taxa de poda de 30%. Os valores são ilustrativos, e não medidos: a tolerância é o inverso da degradação, e cada taxa do esquema de poda com análise de sensibilidade é a tolerância multiplicada por 0,10. A medida de degradação adotada é definida na Seção 5.3.1.

Tabela 1 – Exemplo numérico dos passos 1 a 3 da análise de sensibilidade em um modelo hipotético.

	Camada 0	**Camada 1**	**Camada 2**	**Camada 3**	**Média**
Degradação (passo 1)	0,50	0,20	1,00	0,25	—
Tolerância (passo 2)	2	5	1	4	3
Taxa de poda por camada, por esquema de poda (passo 3)					
Uniforme	30%	30%	30%	30%	30%
Com análise de sensibilidade	20%	50%	10%	40%	30%

O passo 1 consiste em podar cada camada de forma isolada, aplicando uma taxa-sonda, cujo valor é fixado pelos limites descritos no procedimento, e avaliando o impacto na *perplexity* em relação ao modelo base. Conforme ilustrado na primeira linha da tabela, supondo que a camada 2 sofra a maior degradação e a camada 1, a menor, conclui-se que a camada 2 possui alta sensibilidade à poda, ao passo que a camada 1 apresenta maior tolerância à poda.

No passo 2, a tolerância de cada camada é definida pelo inverso de sua respectiva degradação. Dessa forma, se a degradação da camada 1 é cinco vezes menor que a da camada 2, atribui-se a ela uma tolerância cinco vezes maior. Portanto, a distribuição das tolerâncias (2, 5, 1 e 4) estabelece que a taxa de poda efetiva da camada 1 deverá ser cinco vezes mais agressiva do que a da camada 2.

O passo 3 normaliza as tolerâncias para atingir a taxa de poda pretendida. Como a média das tolerâncias calculadas é 3, aplica-se o fator 0,10 para obter uma taxa de poda de 30%. Com isso, as taxas individuais das camadas são 20%, 50%, 10% e 40%, mantendo a mesma proporção do exemplo inicial. Para uma taxa de poda de 15%, o fator seria 0,05, resultando em taxas de 10%, 25%, 5% e 20%.

No passo 4, ocorre a remoção dos parâmetros. Em cada camada, são removidos os elementos de menor pontuação, segundo o critério de poda adotado, até atingir a respectiva taxa de poda. Assim, o perfil de sensibilidade determina a *distribuição* da poda entre as camadas, enquanto a taxa de poda estabelece o *volume total* de parâmetros removidos.

Embora os dois esquemas de poda apresentados na Tabela 1 apresentem a mesma média e, portanto, removam a mesma quantidade global de parâmetros, eles diferem

quanto à distribuição das remoções entre as camadas. De forma semelhante, o esquema de poda não uniforme da Etapa 2 estabelece taxas distintas para cada camada. Entretanto, essas taxas são determinadas pelas pontuações individuais dos pesos, e não por uma medida direta da degradação do desempenho do modelo. Como o perfil de sensibilidade se baseia na degradação empírica e cada estratégia de poda remove unidades de naturezas distintas, sejam pesos individuais ou estruturas inteiras, sua medição deve ser realizada separadamente para cada estratégia.

A aplicação dessa estratégia é detalhada na Seção 5.3.

4.4 *Fine-tuning* de Recuperação

As Etapas 2 e 3 adotam uma abordagem *one-shot*, na qual a poda é seguida imediatamente pela avaliação do modelo, sem retreinamento. Essa escolha metodológica permite mensurar o impacto direto da remoção de parâmetros sobre o desempenho preditivo, sem a interferência de eventuais efeitos de recuperação. Essa abordagem, entretanto, difere do procedimento convencional das estratégias clássicas de poda, que empregam o retreinamento para permitir que os pesos remanescentes se adaptem e compensem as perdas decorrentes da remoção de parâmetros (HAN et al., 2015; LI et al., 2017) (Seção 2.6). Por sua vez, a Etapa 4 incorpora o retreinamento ao escopo experimental, com o objetivo de avaliar a capacidade de recuperação do modelo diante da degradação provocada pela poda.

O *fine-tuning* de recuperação consiste em dar continuidade ao treinamento do modelo após a poda. Os parâmetros remanescentes são ajustados utilizando a partição de treino da base de dados de avaliação, com o mesmo objetivo do pré-treinamento — a predição do próximo *token* — e um número fixo de passos de otimização, igual para todas as configurações submetidas ao retreinamento. Ao final, o modelo é novamente avaliado na partição de teste, utilizando as mesmas métricas da Seção 4.1.3. A recuperação é quantificada pela comparação da *perplexity* do modelo podado antes e depois do retreinamento. Como o retreinamento pode alterar o desempenho do modelo mesmo sem poda, o modelo base é submetido ao mesmo retreinamento, e a *perplexity* de cada modelo podado retreinado é comparada também à do modelo base retreinado, referência adotada após o *fine-tuning* (Seção 4.1.3).

5 Desenvolvimento

Este capítulo descreve a implementação do experimento e as decisões técnicas decorrentes da aplicação da metodologia estabelecida no Capítulo 4, com o objetivo de responder às perguntas (A) e (B) formuladas naquele capítulo. O capítulo organiza-se segundo as quatro etapas metodológicas: escolha do modelo base e da base de dados (Seção 5.1), aplicação da poda sem otimização (Seção 5.2), aplicação da poda com otimização (Seção 5.3) e aplicação do *fine-tuning* de recuperação (Seção 5.4). As seções finais abordam aspectos comuns a todas as fases, incluindo o *framework* e as bibliotecas utilizadas, a arquitetura do *pipeline*, o ambiente computacional, a síntese das decisões de implementação e as limitações do estudo.

5.1 Modelo Base

Na primeira etapa, os critérios da Seção 4.1 foram aplicados para selecionar, em conjunto, o modelo base e a base de dados de avaliação, e definiram-se a implementação e os parâmetros das métricas de avaliação. As subseções a seguir apresentam a seleção do modelo (Seção 5.1.1), a seleção da base de dados (Seção 5.1.2) e a implementação das métricas (Seção 5.1.3).

5.1.1 Seleção do Modelo

Os critérios C1 a C6 (Seção 4.1.1) foram aplicados a nove candidatos: as quatro variantes do GPT-2 (*small*, *medium*, *large* e XL), o DistilGPT2 (SANH et al., 2019), o OPT-125M, o Pythia-160M (BIDERMAN et al., 2023), o BERT-base (DEVLIN et al., 2019) e o Llama 2 7B. Os candidatos compreendem a família GPT-2; três modelos abertos de porte próximo ao do GPT-2 *small*; e dois modelos de classes distintas, incluídos para verificar os limites dos critérios: um codificador e um modelo com bilhões de parâmetros. A Tabela 2 resume a verificação. As colunas C1 a C6 correspondem aos critérios de seleção do modelo, e a coluna D4, ao critério da base de dados que depende do modelo (Seção 4.1.2), verificado em relação ao WikiText-2 (MERITY et al., 2017), já que a seleção é conjunta; o traço indica critério não verificado, por ter sido o candidato eliminado por outro. A coluna Mem. FT traz a memória mínima para o *fine-tuning* completo, a 16 bytes por parâmetro (critério C3), e a coluna Downl., os *downloads* acumulados na plataforma Hugging Face, em milhões, indicador complementar de adoção que não integra os critérios.

Avaliar a inclusão de modelos pequenos posteriores a 2023 (por exemplo, SmolLM2, Qwen2.5-0.5B, Llama 3.2 1B), que a banca pode questionar.

Tabela 2 – Candidatos ao modelo base e verificação dos critérios de seleção.

Modelo	**Parâm.**	**Mem. FT (GB)**	**Downl. (mi)**	**C1**	**C2**	**C3**	**C4**	**C5**	**C6**	**D4**
GPT-2 *small*	124M	2,0	930,1	✓	✓	✓	✓	✓	✓	✓
GPT-2 *medium*	355M	5,7	29,8	✓	✓	✓	✓	✓	✓	✓
GPT-2 *large*	774M	12,4	61,0	✓	✓	✓	✓	✓	✓	✓
GPT-2 XL	1,56B	24,9	17,0	✓	✓	×	✓	✓	✓	✓
DistilGPT2	82M	1,3	256,8	✓	✓	✓	×	—	✓	—
OPT-125M	125M	2,0	215,8	✓	✓	✓	✓	✓	✓	×
Pythia-160M	162M	2,6	27,2	✓	✓	✓	✓	—	✓	×
BERT-base	110M	1,8	3.246,2	×	✓	✓	✓	×	✓	×
Llama 2 7B	6,74B	107,8	34,6	✓	×	×	✓	✓	✓	—

Fonte: Elaboração própria, com dados de Radford et al. (2019), Rajbhandari et al. (2020), Frantar e Alistarh (2023), Sun et al. (2024), Gao et al. (2020) e da plataforma Hugging Face (acesso em 28 set. 2026).

As quatro variantes do GPT-2 atendem a C5 porque Radford et al. (2019) publicam, na Tabela 3 do artigo original, a *perplexity* de cada uma no WikiText-2: 29,41 (*small*), 22,76 (*medium*), 19,93 (*large*) e 18,34 (XL). A relação entre o valor publicado para o *small* e o medido neste trabalho, obtidos por protocolos distintos, é discutida na Seção 6.1.

Seis candidatos não atendem a todos os critérios. O BERT-base é apenas codificador, pré-treinado com modelagem de linguagem mascarada sobre textos que incluem a Wikipédia em inglês (DEVLIN et al., 2019): não atende a C1; não atende a C5, pois a *perplexity* autorregressiva não é bem definida para esse tipo de modelo (Hugging Face, 2026b); e não atende a D4. O Llama 2 7B não atende a C2, pois o acesso a seus pesos na plataforma Hugging Face depende de aprovação manual (verificado em 28 set. 2026), nem a C3, pois seus 6,74 bilhões de parâmetros exigiriam 107,8 GB para o *fine-tuning* completo; atende, porém, a C5, com *perplexity* de 5,12 no WikiText-2 reportada por Sun et al. (2024). O GPT-2 XL não atende apenas a C3: um modelo de 1,5 bilhão de parâmetros exige, no mínimo, 24 GB para o treinamento em precisão mista com Adam (RAJBHANDARI et al., 2020). O DistilGPT2 não atende a C4, por ser obtido por destilação de conhecimento supervisionada pelo GPT-2 de 124M (SANH et al., 2019).

O OPT-125M e o Pythia-160M atendem a C1–C4 e a C6, e o OPT-125M também a C5, com *perplexity* de 27,66 no WikiText-2 reportada por Frantar e Alistarh (2023). Ambos, porém, não atendem a D4 em relação ao WikiText-2, pois incluem a Wikipédia no pré-treinamento: o OPT foi pré-treinado sobre uma seleção de componentes do The Pile que inclui a Wikipédia (ZHANG et al., 2022), e o Pythia, sobre o The Pile completo (BIDERMAN et al., 2023), cujo componente Wikipedia (en) soma 6,38 GiB (GAO et al., 2020). O GPT-2, ao contrário, teve os documentos da Wikipédia removidos de sua base de dados de pré-treinamento (RADFORD et al., 2019). Nenhum dos dois modelos forma par admissível com as demais bases de dados candidatas (Tabela 4): o WikiText-103 e The

Pile também não atendem a D4 em relação a eles, e o Penn Treebank e o OpenWebText não atendem a critérios que independem do modelo (D2 e D3, respectivamente).

Restam três candidatos que atendem a todos os critérios: as variantes *small*, *medium* e *large* do GPT-2, esta última com margem estreita em C3 — dos 12,4 GB estimados, restariam cerca de 3,6 GB para as ativações. Pela regra de desempate (Seção 4.1.1), adotou-se o de menor número de parâmetros. A diferença entre eles é relevante para o custo do experimento: o *medium* e o *large* têm, respectivamente, cerca de 2,9 e 6,2 vezes mais parâmetros que o *small*, e a campanha experimental registrou 195 execuções completas de avaliação — 20 da poda por magnitude, 60 da poda estruturada, 48 do perfil de sensibilidade nas duas taxas-sonda, 60 do esquema de poda com análise de sensibilidade e 7 de *fine-tuning* de recuperação. Uma época de *fine-tuning* do GPT-2 *small* sobre o conjunto de treino do WikiText-2 (2.428.416 *tokens* na tokenização do GPT-2) levou cerca de 4,6 minutos no ambiente adotado, e os sete *fine-tunings* da campanha somaram cerca de 31 minutos. Supondo custo proporcional ao número de parâmetros, coerente com a contagem de FLOPs do modelo (cerca de duas operações por parâmetro por *token*), o mesmo conjunto levaria cerca de 1,5 hora com o *medium* e 3,2 horas com o *large*, estimativas, e não medições. A escolha recaiu, portanto, sobre o GPT-2 *small*.

Cabe observar que, embora seja a menor variante da família GPT-2, o modelo é cerca de cinco vezes maior que a ResNet-50, arquitetura tradicionalmente usada em estudos de poda em visão computacional. Essa diferença de escala é justamente o que torna o trabalho pertinente: as conclusões da literatura de poda foram estabelecidas majoritariamente em redes convolucionais, e não é evidente que se transfiram para modelos com arquitetura *Transformer* e com esse perfil de parâmetros.

O GPT-2 *small*, descrito na Seção 2.3.5, é um modelo autorregressivo composto exclusivamente pelo componente decodificador da arquitetura *Transformer*. Os pesos pré-treinados foram obtidos pela plataforma Hugging Face, que os disponibiliza publicamente, e carregados por meio da biblioteca PyTorch (PASZKE et al., 2019). A Tabela 3 resume a configuração da variante utilizada e as métricas do modelo base, avaliado sem nenhuma modificação no conjunto de teste do WikiText-2, base de dados selecionada na Seção 5.1.2. A *perplexity* obtida, 25,17, é o valor contra o qual são comparadas todas as configurações podadas das etapas seguintes.

5.1.2 Seleção da Base de Dados

Cinco candidatos foram verificados quanto aos critérios D1 a D4 (Seção 4.1.2): o WikiText-2, o WikiText-103, o Penn Treebank, o OpenWebText (GOKASLAN; COHEN, 2019) e The Pile. A Tabela 4 resume a verificação. Como D1 e D4 dependem do modelo, a tabela os verifica para a família GPT-2, e a verificação recíproca, para os demais candidatos a modelo, está na Tabela 2. A coluna Treino traz o tamanho da partição de treino,

Tabela 3 – Configuração do GPT-2 *small* utilizado como modelo base.

Característica	**Valor**
Blocos *Transformer* empilhados (camadas)	12
Cabeças de atenção por camada	12
Dimensão do modelo (d_{model})	768
Dimensão intermediária do MLP (d_{ff})	3.072
Tamanho do vocabulário	50.257
Janela de contexto máxima	1.024 *tokens*
Total de parâmetros	124.439.808
Tamanho em memória	474,70 MB
FLOPs por janela de 512 *tokens*	126,6 G
Perplexity no WikiText-2 (teste)	25,17

Fonte: Elaboração própria.

que define o desempate, na unidade em que cada base o publica: nas três primeiras, *tokens* de palavra, contagem distinta da de subpalavras produzida pelo tokenizador do GPT-2 (Seção 5.4.4). O traço indica critério não verificado, por ter sido o candidato eliminado por outro.

Tabela 4 – Candidatas a base de dados de avaliação e verificação dos critérios de seleção.

Base de dados	**Treino**	**D1**	**D2**	**D3**	**D4**
WikiText-2	2,1 mi *tokens*	✓	✓	✓	✓
WikiText-103	103,2 mi *tokens*	✓	✓	✓	✓
Penn Treebank	0,93 mi *tokens*	✓	×	✓	✓
OpenWebText	39,8 GB	—	✓	×	×
The Pile	825 GiB	✓	✓	✓	×

Fonte: Elaboração própria, com dados de Merity et al. (2017), Radford et al. (2019), Gao et al. (2020) e da plataforma Hugging Face (acesso em 28 set. 2026).

Três candidatos não atendem a todos os critérios. O Penn Treebank não atende a D2: na versão usual, as palavras estão em caixa baixa, a pontuação foi removida e o vocabulário foi limitado a 10 mil palavras (MERITY et al., 2017). O OpenWebText não atende a D3, pois é distribuído com uma única partição, de treino (8.013.769 documentos, 39,8 GB, verificado na plataforma Hugging Face), nem a D4, por ser uma reprodução aberta do WebText, a base de dados de pré-treinamento do GPT-2 (GOKASLAN; COHEN, 2019). The Pile atende a D1, pois Gao et al. (2020) avaliam as quatro variantes do GPT-2 em seu conjunto de teste, e a D3, por dispor de partições oficiais; não atende a D4 porque inclui o OpenWebText2, construído pelo mesmo método do WebText (GAO et al., 2020). Em sentido oposto, os autores do GPT-2 removeram do WebText os documentos da Wikipédia para evitar sobreposição com conjuntos de avaliação; a sobreposição de 8-gramas medida por eles entre o treino do WebText e os conjuntos de teste

é de 1,63% no WikiText-2, 2,42% no WikiText-103 e 0,88% no Penn Treebank (Tabela 6 de Radford et al. (2019)).

Restam dois candidatos que atendem a todos os critérios: o WikiText-2 e o WikiText-103, que compartilham os mesmos conjuntos de validação e teste, com 217.646 e 245.569 *tokens* de palavra, respectivamente (MERITY et al., 2017), e diferem apenas no conjunto de treino, cerca de 49 vezes maior no WikiText-103. A avaliação de *perplexity* é, portanto, idêntica nos dois, e a diferença está no custo do *fine-tuning*: pelo tempo de uma época medido na Seção 5.1.1, uma época sobre o WikiText-103 levaria cerca de 3,8 horas por *fine-tuning*, e os sete *fine-tunings* da campanha passariam de cerca de 31 minutos para cerca de 25 horas (estimativa proporcional ao número de *tokens* de treino). Pela regra de desempate (Seção 4.1.2), adotou-se o de menor conjunto de treino, o WikiText-2.

O WikiText-2 é uma base de dados de língua inglesa extraída de artigos verificados da Wikipédia. Diferentemente de bases de dados construídas a partir de frases isoladas, ele preserva a estrutura original dos artigos — parágrafos completos, pontuação e capitalização mantidas —, o que o torna adequado para medir a capacidade de um modelo de acompanhar contexto ao longo de textos extensos. Empregou-se a variante `wikitext-2-raw-v1`, na qual o texto não sofre substituição de palavras raras por um marcador genérico. Essa escolha decorre de D2: o GPT-2 utiliza tokenização por *subwords*, capaz de representar palavras fora do vocabulário decompondo-as em fragmentos, e a variante processada anularia essa capacidade e distorceria a *perplexity* medida. Na implementação, a base de dados é um argumento de linha de comando, o que permite repetir o experimento com o WikiText-103 sem alterar o código.

5.1.2.1 Pré-processamento e Tokenização

O pré-processamento foi intencionalmente mínimo, para que a perplexity medida refletisse a base de dados e não decisões de limpeza. As linhas do conjunto foram unidas por quebra dupla, preservando a separação entre parágrafos tal como na base de dados original, e o texto resultante foi tokenizado como um fluxo contínuo único.

A tokenização empregou o tokenizador original do GPT-2, baseado no algoritmo BPE (*Byte Pair Encoding*), com vocabulário de 50.257 *tokens*. Manter o tokenizador original é indispensável: os *embeddings* do modelo foram aprendidos para esse vocabulário específico, e qualquer substituição invalidaria os pesos pré-treinados.

5.1.2.2 Divisão dos Dados

Adotou-se a divisão oficial da base de dados. O **conjunto de teste** foi utilizado em todas as medições de perplexity reportadas, tanto do modelo base quanto de todas as configurações comprimidas, e também nas medições do perfil de sensibilidade (Seção 5.3.1). O **conjunto de treinamento** foi utilizado exclusivamente na etapa de

fine-tuning de recuperação (Seção 5.4). O **conjunto de validação** não foi utilizado nos experimentos reportados: a implementação o emprega apenas para acompanhar, opcionalmente, a evolução do treinamento ao longo dos passos, recurso que não foi acionado nas execuções deste trabalho.

Essa separação estrita importa: como as varreduras *one-shot* não realizaram treinamento algum, elas não tiveram contato com o conjunto de treinamento, e a perplexity de teste é uma medida limpa do dano causado pela poda. A ressalva cabe ao esquema de poda com análise de sensibilidade, cujas taxas por camada derivam de medições feitas nessa mesma partição (Seção 5.9).

5.1.3 Implementação e Parâmetros das Métricas de Avaliação

Todas as configurações, do modelo base aos modelos podados com e sem análise de sensibilidade, foram avaliadas pelo mesmo conjunto de métricas, organizado nas três dimensões definidas na Seção 4.1.3. As subseções seguintes, uma por dimensão, descrevem a implementação e os parâmetros adotados; ao final, as métricas são reunidas em uma tabela de síntese.

5.1.3.1 Perplexity

A *perplexity* foi calculada pelo método da janela deslizante, padrão para modelos autorregressivos: o fluxo de *tokens* do conjunto de teste foi percorrido em janelas de 1.024 *tokens*, o tamanho máximo de contexto do modelo, com passo de 512. Em cada janela, apenas os *tokens* ainda não avaliados contribuíram para a perda, e os demais entraram como contexto, de modo que cada *token* foi previsto dispondo de contexto anterior substancial. Passos menores produzem estimativas mais precisas ao custo de mais computação; o passo adotado é o compromisso usual na literatura (Hugging Face, 2026b) e foi mantido idêntico em todas as configurações.

5.1.3.2 Custo Computacional

O número de parâmetros remanescentes foi reportado em duas contagens: o total, que reflete o que existe na arquitetura, e a de não nulos, que reflete o que sobrou após a poda. Na poda não estruturada, as duas divergem, pois os pesos anulados continuam na arquitetura; na poda estruturada, as duas contagens do modelo podado coincidem e ficam abaixo das do modelo base, pois as estruturas removidas deixam de existir. Essa divergência evidencia a diferença entre compressão contábil e compressão física. A esparsidade real, derivada dessas contagens, foi calculada sobre o conjunto podável: na poda por magnitude, é a fração de pesos anulados nas 48 matrizes podáveis; na poda estruturada, a fração de parâmetros que deixaram de existir nas mesmas projeções, vieses incluídos,

já que a remoção de uma estrutura elimina também o viés que lhe corresponde. O tamanho do modelo foi calculado pela soma dos *bytes* ocupados pelos parâmetros, todos em precisão simples, com 4 *bytes* cada; por isso, na poda por magnitude, ele permanece em 474,70 MB, pois os pesos anulados ocupam o mesmo espaço que os demais.

Os FLOPs foram contabilizados pela biblioteca `thop`, sobre uma entrada de 512 *tokens*. A ferramenta percorre os módulos do modelo e soma o custo de cada um, segundo uma regra associada ao tipo do módulo: em um módulo linear, duas operações, uma multiplicação e uma soma, por peso, para cada *token* processado. No GPT-2, as projeções das camadas são implementadas por um tipo de módulo que a ferramenta não reconhece e ao qual atribui custo nulo. Sem tratamento, a contagem ignoraria as matrizes que a poda modifica, responsáveis por 87,0 dos 126,6 GFLOPs do modelo base, e permaneceria constante sob qualquer poda; por isso, aplicou-se a essas projeções a mesma regra dos módulos lineares. As multiplicações de matriz internas ao cálculo da atenção, que não pertencem a nenhum módulo, não foram contabilizadas em nenhuma configuração. Elas representam cerca de 7% do custo do modelo base: são constantes na poda por magnitude e diminuem com a remoção de cabeças na poda estruturada, cuja redução de FLOPs reportada é, por isso, ligeiramente inferior à real. A implementação prevê ainda um estimador analítico de reserva, acionado apenas se a contagem por módulos falhar; todos os valores reportados no Capítulo 6 provêm da contagem por módulos.

O tempo de inferência foi medido sobre uma entrada de 512 *tokens*, com o temporizador `perf_counter` da linguagem Python e sincronização da GPU antes de cada leitura do temporizador: três execuções de aquecimento, descartadas para mitigar efeitos de inicialização da GPU, seguidas de 20 execuções cronometradas, das quais se reportam a média e o desvio-padrão.

5.1.3.3 Consumo de Energia

A energia foi medida com a biblioteca `CodeCarbon`, que lê a potência do *hardware* a intervalos de 1 s durante toda a avaliação de uma configuração — o cálculo da *perplexity* e as medições de custo — e registra a energia consumida em kWh, apresentada neste trabalho em Wh. A biblioteca também estima as emissões de CO_2, que foram registradas, mas não analisadas, pelas razões expostas na Seção 4.1.3. Os experimentos confirmaram essa decisão: o fator de intensidade carbônica, obtido pela razão entre as emissões e a energia registradas, variou de 0,139 a 0,453 kg de CO_2 por kWh entre as varreduras. O modelo base, avaliado sem nenhuma poda em duas varreduras, teve energia cerca de 6% menor na segunda, de 1,572 para 1,481 Wh, e emissões estimadas cerca de 71% menores, de 0,711 para 0,205 g, embora o modelo fosse o mesmo.

A Tabela 5 sintetiza as métricas das três dimensões tratadas na Seção 5.1.3: relaciona, para cada dimensão, as métricas coletadas, o que cada uma mede, a unidade

em que é reportada e a ferramenta empregada na medição.

Tabela 5 – Métricas de avaliação coletadas para cada configuração.

Métrica	**O que mede**	**Unidade**	**Ferramenta**
Desempenho preditivo (Seção 5.1.3.1)			
Perplexity	Qualidade da modelagem da linguagem; quanto menor, melhor	—	PyTorch
Custo computacional (Seção 5.1.3.2)			
Parâmetros totais	Parâmetros existentes na arquitetura	unidades	PyTorch
Parâmetros não nulos	Parâmetros efetivamente diferentes de zero	unidades	PyTorch
Esparsidade real	Fração de parâmetros podáveis eliminados	%	PyTorch
Tamanho do modelo	Memória ocupada pelos parâmetros	MB	PyTorch
FLOPs	Operações de ponto flutuante por inferência	FLOPs	`thop`
Tempo de inferência	Tempo médio de uma passagem direta	ms	`perf_counter`
Consumo de energia (Seção 5.1.3.3)			
Energia consumida	Energia gasta durante a avaliação	Wh	`CodeCarbon`

Fonte: Elaboração própria.

5.2 Poda sem Otimização

Após a seleção do modelo base, da base de dados e das métricas de avaliação (Seção 5.1), seguiu-se a escolha das estratégias de poda que poderiam ser aplicadas a esse modelo.

5.2.1 Seleção das Estratégias

Na seleção das estratégias, aplicaram-se os critérios P1 a P3 (Seção 4.2.3) a seis estratégias candidatas, pré-selecionadas na revisão de Cheng, Zhang e Shi (2024), que organiza em uma taxonomia as estratégias de poda de redes neurais profundas: as duas estratégias clássicas, a poda por magnitude e a poda estruturada por norma L_1, e quatro alternativas propostas para modelos *Transformer*, o SparseGPT, o Wanda, a *movement pruning* (SANH; WOLF; RUSH, 2020) e a poda de cabeças por gradiente. A Tabela 6 resume a verificação de P1 e P2, que se aplicam a cada estratégia; P3 refere-se ao conjunto e é tratado na conclusão. A tabela registra também o ano de publicação do artigo que propôs cada estratégia e o número de citações desse artigo na base Semantic Scholar, indicador complementar de representatividade que não integra os critérios.

Tabela 6 – Estratégias de poda candidatas e verificação dos critérios de seleção.

Estratégia	**Granularidade**	**Critério de poda**	**Ano**	**Citações**	**P1**	**P2**
Magnitude	fina	$\|w\|$ de cada peso	2015	8.047	✓	✓
Norma L_1	grossa	norma L_1 dos pesos da unidade	2017	4.214	✓	✓
SparseGPT	fina	reconstrução com dados de calibração	2023	1.552	×	✓
Wanda	fina	$\|w\|$ × norma das ativações	2024	999	×	✓
Movement pruning	fina	movimento dos pesos no *fine-tuning*	2020	654	×	×
Cabeças por gradiente	grossa	gradiente da perda em relação à máscara	2019	1.513	×	✓

Fonte: Elaboração própria, com base em Han et al. (2015), Li et al. (2017), Frantar e Alistarh (2023), Sun et al. (2024), Sanh, Wolf e Rush (2020) e Michel, Levy e Neubig (2019), e com dados da base Semantic Scholar (acesso em 3 out. 2026).

As quatro alternativas não atendem a P1, pois todas estimam a importância com base em dados. O SparseGPT e o Wanda, referências recentes da poda não estruturada de LLMs, utilizam 128 sequências de calibração extraídas da base de dados C4: o primeiro para reconstruir, com informação de segunda ordem, os pesos remanescentes de cada camada (FRANTAR; ALISTARH, 2023); o segundo para ponderar a magnitude de cada peso pela norma da ativação de entrada correspondente (SUN et al., 2024). A importância das cabeças em Michel, Levy e Neubig (2019) é estimada pelo gradiente da perda sobre uma base de dados. A *movement pruning* também não atende a P2, pois a importância de cada peso é aprendida ao longo do *fine-tuning*. A exclusão dessas estratégias não decorre de desempenho inferior: elas já incorporam o tipo de informação que a análise de sensibilidade introduz, e sua combinação com o esquema de poda com análise de sensibilidade fica fora do escopo deste trabalho.

Portanto, conclui-se que, entre as seis candidatas, apenas a poda por magnitude e a poda estruturada por norma L_1 atendem aos critérios estabelecidos: cada uma atende a P1 e a P2, e as duas, juntas, atendem a P3, pois a primeira opera sobre pesos individuais e a segunda, sobre unidades arquiteturais inteiras. Os artigos que as propuseram são também os mais citados entre as seis candidatas (Tabela 6), diferença que reflete em parte a antiguidade dos trabalhos, e a meta-análise de Blalock et al. (2020), que reuniu os resultados de 81 trabalhos de poda, aponta como um dos resultados consistentes da área que a poda pela magnitude dos pesos comprime substancialmente as redes sem reduzir a acurácia.

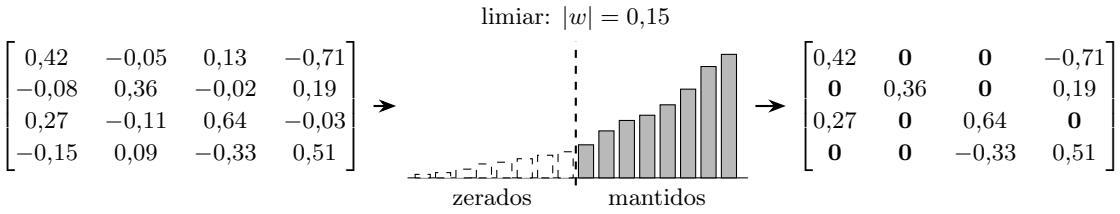
A seguir, cada uma das estratégias selecionadas é detalhada em uma subseção própria; depois delas, são tratados os aspectos comuns às duas: a distribuição da poda entre as camadas e as taxas de poda avaliadas.

5.2.2 Poda por Magnitude

O primeiro método, de poda não estruturada, segue o método resenhado na Seção 3.1. O critério de poda é o valor absoluto do peso: assume-se que pesos próximos

de zero contribuem pouco para a saída e podem ser anulados. Dada uma taxa de poda, ordenaram-se os pesos por $|w|$, tomou-se como limiar o valor de $|w|$ na posição correspondente à taxa e zeraram-se os pesos com $|w|$ igual ou inferior a ele. A Figura 8 ilustra o procedimento em uma matriz hipotética de 16 pesos, à taxa de poda de 50%: o limiar é o oitavo menor valor de $|w|$, 0,15, e os oito pesos com $|w|$ até esse valor são anulados. A matriz conserva suas dimensões, e os zeros ficam espalhados pelas posições em que estavam os pesos de menor valor absoluto.

Figura 8 – Poda por magnitude aplicada a uma matriz de pesos hipotética.



(a) matriz de pesos 4×4 — 16 pesos

(b) pesos ordenados por $|w|$ taxa de 50%: saem os 8 menores

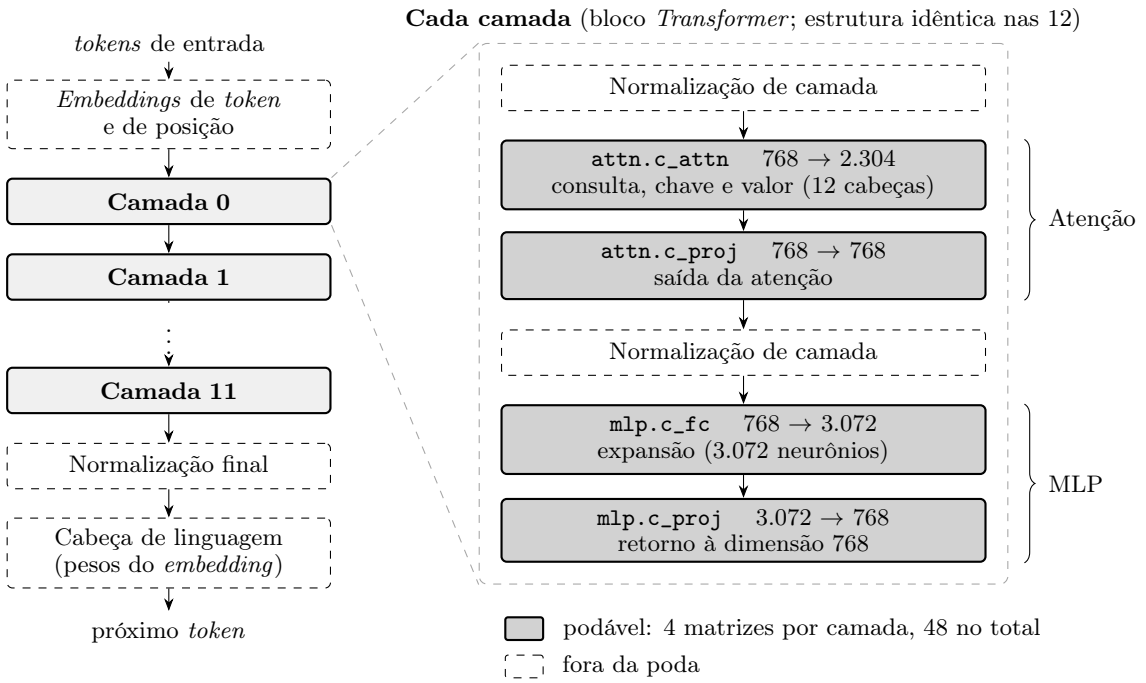
(c) matriz podada continua 4×4 — 8 pesos zerados

Fonte: Elaboração própria.

Essa taxa, como toda taxa de poda deste trabalho, é relativa aos parâmetros podáveis, e não ao total de parâmetros do modelo (Seção 4.2.1): o limiar é calculado e aplicado apenas sobre eles. No GPT-2 *small*, os parâmetros podáveis são as matrizes de peso das quatro projeções lineares de cada camada, duas na atenção — `attn.c_attn`, que produz consulta, chave e valor, e `attn.c_proj`, que projeta a saída das cabeças — e duas no MLP, `mlp.c_fc` e `mlp.c_proj`. Nas 12 camadas, são 48 matrizes e 84.934.656 pesos, 68,3% dos 124.439.808 parâmetros do modelo (Figura 9). Ficam fora da poda, pelas razões expostas na Metodologia, as matrizes de *embedding* de *token* (`wte`) e de posição (`wpe`); a cabeça de linguagem (`lm_head`), que no GPT-2 é o mesmo tensor de `wte` e cujo custo, 39,5 dos 126,6 GFLOPs do modelo base, permanece constante em todas as configurações; e os vieses e parâmetros de normalização de camada.

Na Figura 9, o modelo aparece à esquerda, com as 12 camadas, de estrutura idêntica e numeradas de 0 a 11, entre os *embeddings* e a cabeça de linguagem; à direita, detalha-se a estrutura de cada camada, com as quatro matrizes de projeção podáveis em destaque, o que totaliza 48 matrizes, e os componentes fora da poda tracejados. As cabeças de atenção e os neurônios do MLP não são matrizes à parte, e sim fatias que atravessam duas projeções: cada uma das 12 cabeças de uma camada ocupa 192 colunas de `attn.c_attn`, 64 para consulta, 64 para chave e 64 para valor, e as 64 linhas de `attn.c_proj` que recebem sua saída; cada um dos 3.072 neurônios ocupa uma coluna de `mlp.c_fc`, que o produz, e uma linha de `mlp.c_proj`, que o consome. A poda estruturada remove a fatia inteira, nas duas matrizes. No modelo inteiro, são 144 cabeças e 36.864 neurônios.

Figura 9 – Localização dos parâmetros podáveis no GPT-2 *small*.



A implementação preserva a característica essencial do método: os pesos são **zerados**, não removidos. As matrizes conservam suas dimensões originais e a arquitetura permanece intacta — o que, como discutido na Seção 2.6, é precisamente a razão pela qual esse método não reduz o custo de inferência em *hardware* convencional.

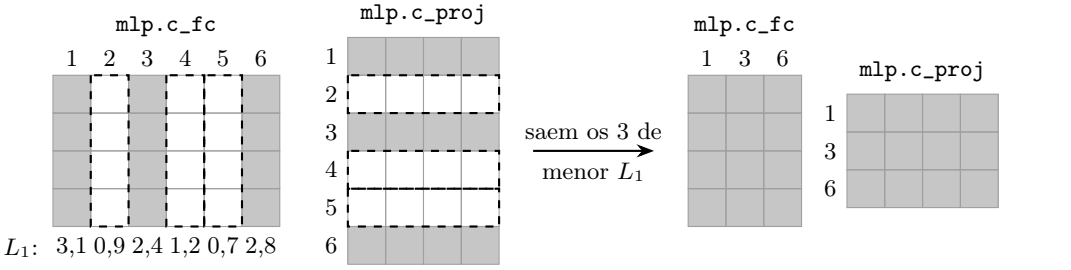
5.2.3 Poda por Norma L_1

O segundo método, de poda estruturada, adapta ao *Transformer* o método resenhado na Seção 3.2. O trabalho original poda filtros convolucionais ordenados por norma L_1; aqui, as unidades estruturais equivalentes são as **cabeças de atenção** e os **neurônios do MLP**, os 3.072 de sua dimensão intermediária. Para cada uma dessas estruturas calculou-se a norma L_1 dos pesos que lhe correspondem, e removeram-se as de menor norma. Também aqui a poda se restringe aos parâmetros podáveis (Seção 4.2.1): cada cabeça e cada neurônio é uma fatia de duas das matrizes podáveis (Figura 9), e os *embeddings*, a cabeça de linguagem e os parâmetros de normalização permanecem intactos em todas as taxas de poda.

A Figura 10 ilustra o procedimento para os neurônios do MLP, em uma camada hipotética com seis neurônios, à taxa de poda de 50%. Cada neurônio ocupa uma coluna de `mlp.c_fc`, que o produz, e uma linha de `mlp.c_proj`, que o consome, e sua norma L_1 é a soma dos valores absolutos dos pesos dessas duas fatias, acrescida do viés correspondente em `mlp.c_fc`. Os três neurônios de menor norma, 2, 4 e 5, saem inteiros: a coluna de `mlp.c_fc` e a linha de `mlp.c_proj` de cada um são eliminadas, e as duas matrizes passam a ter três neurônios. As cabeças de atenção seguem o mesmo procedimento, com fatias

maiores: cada cabeça ocupa 192 colunas de `attn.c_attn` e 64 linhas de `attn.c_proj`.

Figura 10 – Poda por norma L_1 dos neurônios do MLP em uma camada hipotética.



(a) antes da poda
seis neurônios e a norma L_1 de cada um

(b) após a poda de 50%
três neurônios; as duas matrizes encolhem

Fonte: Elaboração própria.

A diferença decisiva em relação ao método anterior é que a remoção é **física**. As matrizes foram efetivamente reconstruídas com dimensões menores: as colunas de consulta, chave e valor associadas às cabeças removidas foram eliminadas de `c_attn`, as linhas correspondentes saíram de `c_proj`, e a contabilidade interna do módulo de atenção — número de cabeças e dimensão de divisão — foi atualizada de acordo. O resultado não é um modelo esparso, mas uma arquitetura menor e densa.

A implementação admite três alvos: apenas cabeças, apenas neurônios do MLP, ou ambos simultaneamente. Os resultados do Capítulo 6 empregam os dois alvos simultâneos, com a mesma taxa aplicada a cada tipo de estrutura; os alvos isolados aparecem apenas na comparação entre cabeças e neurônios da Seção 6.4.2. Uma salvaguarda impede que qualquer camada fique sem nenhuma cabeça ou sem nenhum neurônio: sem ela, a poda deixaria de remover estruturas e passaria a remover subcamadas inteiras — a atenção ou o MLP de uma camada —, granularidade mais grossa que a adotada e fora do escopo deste trabalho.

Convém registrar que a taxa de poda, neste método, é expressa como fração de **estruturas** removidas, não de parâmetros. Duas razões fazem a fração de parâmetros efetivamente eliminada divergir da taxa solicitada, e por isso ela é reportada separadamente em todas as tabelas. A primeira é que cabeças e neurônios têm custos diferentes em parâmetros, já que cada estrutura é removida com todos os pesos que lhe pertencem, e uma cabeça de atenção reúne muito mais pesos que um neurônio do MLP (Figura 9). Cada cabeça ocupa 192 colunas de `attn.c_attn` e 64 linhas de `attn.c_proj`, todas com 768 pesos, o que soma 196.608 pesos; cada neurônio ocupa apenas uma coluna de `mlp.c_fc` e uma linha de `mlp.c_proj`, também com 768 pesos cada, o que soma 1.536. Remover uma única cabeça equivale, portanto, a remover 128 neurônios. Como a taxa conta estruturas, e não pesos, a fração de parâmetros eliminada depende de quais estruturas são removidas: as 12 cabeças de uma camada concentram um terço de seus pesos podáveis (2.359.296 de 7.077.888), e os 3.072 neurônios, os outros dois terços. Uma taxa de 50% aplicada apenas

às cabeças elimina, assim, cerca de 16,7% dos parâmetros podáveis, um terço da taxa; aplicada apenas aos neurônios, cerca de 33,3%, dois terços dela. Quando os dois alvos recebem a mesma taxa, como nos resultados do Capítulo 6, as duas parcelas somam a própria taxa, e a divergência passa a vir da segunda razão, descrita a seguir, cujo efeito a diferença de tamanho amplia: cada cabeça que o arredondamento deixa de remover conserva tantos pesos quanto 128 neurônios.

A segunda razão é a granularidade discreta das estruturas: o número de unidades a remover é a parte inteira do produto entre a taxa e a contagem disponível, o que nos neurônios do MLP — 3.072 por camada — é praticamente imperceptível, mas nas cabeças de atenção, que são apenas 12 por camada, produz arredondamentos visíveis. Uma taxa de 90%, por exemplo, remove 10 das 12 cabeças, ou seja, 83%.

5.2.4 Distribuição de Poda entre as Camadas

Os dois métodos foram avaliados com os esquemas de poda uniforme e não uniforme definidos na Seção 4.2. No esquema uniforme, a poda por magnitude calcula um limiar para cada uma das 48 matrizes podáveis, e a poda estruturada ordena as cabeças e os neurônios de cada camada separadamente. No esquema não uniforme, a poda por magnitude aplica um único limiar às 48 matrizes, e a poda estruturada ordena as estruturas das doze camadas em conjunto, em duas ordenações independentes, uma de cabeças e outra de neurônios. As ordenações são separadas porque as normas L_1 dos dois tipos de estrutura não são comparáveis: uma cabeça soma 196.608 pesos, 128 vezes os 1.536 de um neurônio, e, no GPT-2, a cabeça de menor norma tem norma 14 vezes maior que a do neurônio de maior norma. Em uma ordenação conjunta, todos os neurônios viriam antes da primeira cabeça, e nenhuma cabeça seria removida até a taxa de 90%; com as duas ordenações, a taxa de poda incide igualmente sobre os dois tipos, e a 50%, por exemplo, removem-se metade das cabeças e metade dos neurônios.

5.2.5 Taxas de Poda Avaliadas

As dez taxas de poda definidas na Metodologia (Seção 4.2.2), de 0% a 90%, foram aplicadas às duas estratégias selecionadas, cada uma com os dois esquemas de poda, o que resultou em quatro varreduras: a uniforme e a não uniforme da poda por magnitude e a uniforme e a não uniforme da poda estruturada. Nesta última, a taxa conta cabeças de atenção e neurônios do MLP, e não pesos (Seção 5.2.3), e as mesmas taxas foram aplicadas também aos alvos isolados, apenas cabeças ou apenas neurônios, usados na comparação entre os dois tipos de estrutura. A verificação prevista para a taxa de 0% foi satisfeita em todas as varreduras, inclusive nas da Etapa 3: o modelo base reproduziu, com todos os dígitos registrados, a *perplexity* de 25,17 medida na Etapa 1 (Seção 5.1).

5.3 Poda com Otimização

Na terceira etapa, aplicou-se a técnica descrita na Seção 4.3 às duas estratégias de poda, nas mesmas taxas de poda da Etapa 2 (Seção 5.2.5). As subseções a seguir apresentam os valores adotados em cada fase: a medida da degradação e as taxas-sonda da fase 1, e a regra que define o esquema de poda na fase 2, com o teto e o parâmetro de intensidade.

5.3.1 Fase 1 — Perfil de Sensibilidade

Cada camada ℓ foi podada isoladamente, à taxa-sonda, e a degradação foi registrada em escala logarítmica:

$$\Delta_\ell = \ln\left(\frac{\mathrm{PPL}_\ell}{\mathrm{PPL}_{\mathrm{base}}}\right), \tag{5.1}$$

em que PPL_ℓ é a *perplexity* do modelo com apenas a camada ℓ podada. O logaritmo é necessário porque a *perplexity* de modelos podados sem recuperação varia por ordens de grandeza; em escala linear, uma única camada catastrófica tornaria todas as demais indistinguíveis entre si. As medições do perfil foram feitas na partição de teste, a mesma em que os modelos podados são avaliados (Seção 5.1.2.2), ponto tratado entre as limitações (Seção 5.9).

Adotou-se a taxa-sonda de 50%, o limite superior admitido pelo procedimento (Seção 4.3), e o perfil foi levantado separadamente para cada estratégia. À mesma taxa-sonda, as duas estratégias causam a uma camada danos de natureza diferente. A poda por magnitude anula metade dos pesos podáveis da camada, escolhidos um a um entre os de menor valor absoluto, de modo que as 12 cabeças e os 3.072 neurônios continuam em operação, cada um com parte de seus pesos. A poda estruturada remove 6 das 12 cabeças e 1.536 dos 3.072 neurônios, cada um com todos os seus pesos, e mantém intactos os demais. A degradação medida reflete, portanto, propriedades diferentes da camada em cada estratégia, e o perfil de cada estratégia orientou apenas o seu próprio esquema de poda; a Seção 6.3.3 mostra que os dois perfis de fato divergem.

Para verificar a estabilidade do perfil, como prevê o passo 1 do procedimento, o perfil completo foi medido novamente, nas duas estratégias, à taxa-sonda de 30%. O valor fica 20 pontos percentuais abaixo de 50%, distância suficiente para que a coincidência das ordenações seja informativa, e acima das taxas baixas, como 10%, em que as degradações ficariam pequenas e próximas entre si. A segunda medição serviu apenas como verificação: todas as taxas de poda por camada reportadas derivam do perfil de 50%. O resultado da comparação é apresentado na Seção 6.3.5.

Os perfis foram comparados em duas situações: entre as duas taxas-sonda, na verificação de estabilidade, e entre as duas estratégias de poda. Em ambas, a comparação

refere-se à ordenação das camadas, e não aos valores de Δ_ℓ, que dependem da taxa-sonda e da estratégia. A concordância entre duas ordenações foi medida pelo coeficiente de correlação de postos proposto por Spearman (1904), representado por ρ. Para o cálculo, as 12 camadas de cada perfil recebem postos de 1 a 12, em ordem decrescente de degradação, de modo que o posto 1 corresponde à camada mais sensível. O coeficiente é obtido das diferenças entre os postos que cada camada ocupa nos dois perfis:

$$\rho = 1 - \frac{6 \sum_\ell d_\ell^2}{n\,(n^2 - 1)}, \tag{5.2}$$

em que d_ℓ é a diferença entre os postos da camada ℓ nos dois perfis e $n = 12$ é o número de camadas. Por exemplo, uma camada que ocupa o 2º posto em um perfil e o 5º no outro tem $d_\ell = 3$. Essa expressão é válida na ausência de empates, e nenhum dos perfis medidos apresentou duas camadas com a mesma degradação. O coeficiente varia de -1 a 1. O valor 1 indica que as duas ordenações são idênticas, e o valor -1, que uma é o inverso da outra. Valores próximos de zero indicam ausência de associação entre as ordenações. Como o cálculo utiliza exclusivamente os postos, uma camada com degradação muito superior às demais ocupa uma única posição na ordenação e não determina, isoladamente, o resultado. Segundo o autor, essa é a principal vantagem do uso de postos em relação à correlação calculada sobre os valores, na qual os casos extremos têm influência desproporcional.

5.3.2 Fase 2 — Definição do Esquema de Poda

A fase 2 converteu o perfil de sensibilidade em um esquema de poda, segundo os passos 2 e 3 do procedimento (Seção 4.3), pela regra da Equação 5.3:

$$r_\ell = \min\!\left(\alpha \cdot \Delta_\ell^{-\beta},\ r_{\max}\right), \qquad \frac{1}{12}\sum_{\ell=0}^{11} r_\ell = s, \tag{5.3}$$

em que r_ℓ é a taxa de poda da camada ℓ; Δ_ℓ é a sua degradação (Equação 5.1), cujo inverso é a tolerância; α é o fator; β, o parâmetro de intensidade; $r_{\max} = 0{,}95$, o teto; e s, a taxa de poda. A segunda parte da equação exige que a média das taxas das 12 camadas seja igual à taxa de poda. Na implementação, essa média é ponderada pelo número de parâmetros podáveis de cada camada, o que permite aplicar a regra a modelos com camadas de tamanhos diferentes; no GPT-2, cujas camadas têm o mesmo tamanho, ela coincide com a média simples. O teto de 95% impede que uma camada perca todos os seus elementos. O fator foi sempre obtido por tentativas sucessivas, ajustado até que a média das taxas coincidisse com a taxa de poda; quando nenhuma camada atinge o teto, o procedimento chega ao mesmo valor da divisão descrita no passo 3.

O parâmetro de intensidade foi implementado como o expoente β, aplicado a cada tolerância antes da multiplicação pelo fator. Enquanto nenhuma camada atinge o teto, a

razão entre as taxas de duas camadas é igual à razão entre suas tolerâncias elevada a β: com $\beta = 1$, as taxas seguem as tolerâncias na proporção exata; com $\beta = 0{,}5$, seguem as raízes quadradas das tolerâncias; e, com $\beta = 0$, todas as camadas recebem a mesma taxa, como no esquema de poda uniforme. O parâmetro não altera a taxa de poda, que o fator mantém igual à média das taxas por camada, nem a ordem das camadas: com qualquer β maior que zero, a camada mais tolerante continua a ser a mais podada. A Tabela 7 mostra esse efeito no exemplo da Tabela 1, à taxa de poda de 30%. Em todas as linhas, a média é 30% e nenhuma camada atinge o teto; a passagem de $\beta = 1$ para $\beta = 0{,}5$ retira 9,7 pontos percentuais da camada 1, a mais tolerante, e acrescenta 8 pontos percentuais à camada 2, a mais sensível.

Tabela 7 – Taxas de poda por camada no exemplo da Tabela 1 para três valores do parâmetro de intensidade.

β	**Fator**	**Camada 0**	**Camada 1**	**Camada 2**	**Camada 3**	**Média**
1,0	0,100	20,0%	50,0%	10,0%	40,0%	30%
0,5	0,180	25,5%	40,3%	18,0%	36,1%	30%
0	0,300	30,0%	30,0%	30,0%	30,0%	30%

No GPT-2, com $\beta = 1$, a camada mais tolerante recebe uma taxa 42,9 vezes maior que a da mais sensível na poda estruturada e 11,3 vezes maior na poda por magnitude, as mesmas razões do perfil de sensibilidade (Tabela 13); com $\beta = 0{,}5$, essas razões caem para as suas raízes quadradas, 6,5 e 3,4. À taxa de poda de 10%, por exemplo, a poda estruturada com $\beta = 1$ atribuiu 0,4% à camada 0, a mais sensível, e 16,9% à camada 9, a mais tolerante; com $\beta = 0{,}5$, as mesmas camadas receberam 2,1% e 13,7%.

O parâmetro é necessário porque seguir integralmente o perfil, o que equivale a fixar $\beta = 1$, traz dois problemas. O primeiro é que não há limite para a diferença de tolerância entre duas camadas: uma camada que quase não degrada o modelo teria tolerância muito alta e concentraria praticamente toda a poda. O segundo é que o perfil é uma medição aproximada. Obtido com cada camada podada isoladamente, a uma única taxa-sonda, ele é mais confiável quanto à ordem das camadas do que quanto à proporção exata entre elas, pois a degradação de camadas podadas em conjunto supera a soma de suas degradações individuais (Seção 6.3.6). Valores de β entre 0 e 1 preservam a ordem medida e atenuam a proporção. Avaliaram-se $\beta = 1$ e $\beta = 0{,}5$ sobre o mesmo perfil, sem repetir a fase 1, a de maior custo. O valor 0,5 é o ponto intermediário entre os dois extremos em escala multiplicativa: com ele, a razão entre as taxas extremas é a média geométrica entre a do esquema de poda uniforme, igual a 1, e a do perfil seguido integralmente; na poda estruturada, 6,5 está para 1 assim como 42,9 está para 6,5. A comparação entre as duas intensidades é apresentada na Seção 6.4.4.

O valor $\beta = 0$ também foi avaliado, mas como verificação da implementação, e não como esquema de poda candidato: com ele, a regra deve reproduzir a varredura

uniforme da Etapa 2. Na poda estruturada, a reprodução deve ser exata. Na poda por magnitude, não: a regra usa um único limiar para as quatro matrizes de cada camada, enquanto a varredura uniforme usa um limiar para cada matriz, e as duas removem a mesma fração de cada camada, mas não exatamente os mesmos pesos. A verificação é apresentada na Seção 6.4.3.

Por fim, se a poda de uma camada na fase 1 não aumentasse a *perplexity* — o que pode ocorrer, pois há casos em que remover cabeças de atenção a *melhora* (MICHEL; LEVY; NEUBIG, 2019) —, a degradação dessa camada seria nula ou negativa, e a tolerância, que é o seu inverso, não poderia ser calculada. Por isso, no cálculo da tolerância, a degradação nunca fica abaixo de um valor mínimo positivo, 0,0001, e uma camada nessa situação entra no esquema de poda como a mais tolerante. No perfil de 50%, do qual derivam os esquemas de poda, essa precaução não chegou a ser acionada: a menor degradação foi de 0,0346, mais de 300 vezes esse valor mínimo.

5.4 *Fine-tuning* de Recuperação

As Etapas 2 e 3 operaram em regime *one-shot*, isto é, podaram e avaliaram sem retreinamento, de modo a isolar o dano causado pela poda. Esse regime, porém, não corresponde ao procedimento da literatura: tanto o método da Seção 3.1 quanto o da Seção 3.2 retreinam o modelo após podá-lo. Na quarta etapa, esse retreinamento foi incorporado ao experimento: sete configurações foram submetidas ao *fine-tuning* de recuperação, e a *perplexity* de cada uma foi medida antes e depois do retreinamento, o que indica quanto da degradação é recuperável. As subseções a seguir apresentam as configurações retreinadas, o procedimento aplicado a cada uma, os cuidados que condicionam a validade da medição e os parâmetros do treinamento.

5.4.1 Configurações Retreinadas

O retreinamento foi aplicado às seguintes configurações das Etapas 2 e 3:

- **Modelo base retreinado:** o modelo base, sem poda, cuja função é descrita na Seção 5.4.3;
- **Poda não estruturada — poda por magnitude**, com esquema de poda uniforme, a 30% e a 50% de esparsidade;
- **Poda estruturada — poda de cabeças de atenção e de neurônios do MLP**, com os dois alvos simultâneos e esquema de poda não uniforme, a 10% e a 30% de estruturas removidas;

- **Poda estruturada com análise de sensibilidade**, com o esquema de poda definido pelo perfil da própria poda estruturada, com $\beta = 1$, nas mesmas duas taxas.

As configurações da lista foram escolhidas a partir dos resultados das Etapas 2 e 3 em regime *one-shot*, apresentados no Capítulo 6. A escolha respondeu a três perguntas.

A primeira é qual dos dois esquemas de poda sem análise de sensibilidade, uniforme ou não uniforme, retreinar em cada estratégia. Escolheu-se, em cada uma, o de menor *perplexity* nas taxas de poda em que a escolha faz diferença. Na poda por magnitude, foi o uniforme, que supera o não uniforme entre 30% e 70%. Fora dessa faixa, a escolha pouco importa: até 20%, os dois quase não degradam o modelo, e a partir de 80% ambos já o destroem (Seção 6.4.1). Na poda estruturada, foi o não uniforme, que supera o uniforme em todas as taxas, exceto 20% (Seção 6.4.2).

A segunda é em qual estratégia retreinar também o esquema de poda com análise de sensibilidade. Escolheu-se apenas a poda estruturada, por ter sido a única a reduzir o tempo de inferência e o consumo de energia (Seção 6.2). Na poda por magnitude, os pesos são zerados, mas não removidos (Seção 5.2.2), e o custo não cai. Usou-se o parâmetro de intensidade $\beta = 1$, o de melhor desempenho na poda estruturada (Seção 6.4.4).

A terceira é em quais taxas de poda retreinar. Na poda estruturada, escolheram-se 10% e 30%. São as mesmas taxas nos dois esquemas de poda, não uniforme e com análise de sensibilidade, o que permite compará-los também depois do retreinamento, e ambas estão na faixa de 10% a 50% em que, em regime *one-shot*, o esquema de poda com análise de sensibilidade obteve *perplexity* menor que o não uniforme (Seção 6.4.2). Na poda por magnitude, escolheram-se duas taxas com danos de intensidades diferentes: 30%, a primeira taxa acima das de até 20%, em que o acréscimo de *perplexity* é de até 2,1 pontos, e 50%, em que a degradação já é severa, com *perplexity* de 473,28 (Seção 6.4.1). Com isso, o retreinamento é observado tanto sobre um dano moderado quanto sobre um dano severo.

5.4.2 Procedimento

Cada configuração passou pelo mesmo procedimento, implementado no módulo `finetune` (Tabela 9):

1. o modelo base foi recarregado a partir dos pesos pré-treinados e podado pelas mesmas funções das Etapas 2 e 3, de modo que o modelo retreinado fosse idêntico ao avaliado em regime *one-shot*;
2. mediu-se a *perplexity* do modelo podado, ponto de partida da recuperação; nas sete configurações, o valor obtido reproduziu, em todos os dígitos registrados, o das varreduras *one-shot*;

3. na poda não estruturada, registrou-se a **máscara binária** do modelo podado, que marca com 1 os pesos que permaneceram e com 0 os que foram anulados, para mantê-los em zero durante o retreinamento (Seção 5.4.3);

4. o modelo foi retreinado sobre a partição de treino do WikiText-2, com os parâmetros da Tabela 8;

5. o modelo retreinado foi avaliado na partição de teste por todas as métricas da Etapa 1 (Tabela 5), pelo mesmo procedimento aplicado aos modelos não retreinados.

O retreinamento atualizou todos os parâmetros do modelo podado — inclusive *embeddings*, vieses e parâmetros de normalização —, e não apenas as matrizes podáveis (Seção 4.2.1), já que o efeito da poda não fica restrito às matrizes de onde os elementos foram removidos. As camadas do GPT-2 são empilhadas (Seção 2.3.4): a saída de cada uma é a entrada da seguinte, e a saída da última alimenta a normalização final e a cabeça de linguagem. Ao podar uma camada, altera-se a representação que ela entrega, e todos os componentes posteriores, podados ou não, passam a receber entradas diferentes daquelas para as quais foram pré-treinados. Recuperar o modelo exige, portanto, não só reajustar os pesos que restaram nas matrizes podadas, mas também readaptar esses componentes às novas entradas. Os parâmetros de normalização, por exemplo, controlam a escala de cada dimensão da entrada da atenção e do MLP e podem reajustá-la às representações alteradas; restringir o retreinamento às matrizes podáveis os manteria fixos nos valores do pré-treinamento, calibrados para o modelo sem poda. Do mesmo modo, a cabeça de linguagem, que no GPT-2 compartilha os pesos com o *embedding* de *token*, precisa readaptar-se às representações que recebe da última camada, o que só é possível se esse *embedding* também for atualizado. Atualizar todos os parâmetros, por outro lado, não desfaz a poda: na poda estruturada, as estruturas removidas deixaram de existir, e, na poda por magnitude, os pesos anulados foram mantidos em zero por uma máscara (Seção 5.4.3).

O modelo avaliado é o obtido ao final do último passo do otimizador: a partição de validação não foi utilizada, e não houve parada antecipada nem seleção do melhor ponto do treinamento. Com isso, a quantidade de retreinamento é idêntica em todas as configurações, condição para que suas *perplexities* pós-retreinamento sejam comparáveis entre si.

5.4.3 Modelo Base Retreinado e Preservação da Esparsidade

Duas decisões de implementação merecem registro por afetarem a validade dos resultados.

A primeira é o **modelo base retreinado**. O GPT-2 não foi pré-treinado no WikiText-2, e sim no WebText, composto por páginas da web obtidas de *links* publicados no Reddit e do qual os documentos da Wikipédia foram removidos (Seção 5.1.2). Por isso, o simples *fine-tuning* do modelo base no WikiText-2 já reduz a *perplexity* por **adaptação de domínio**, sem poda alguma envolvida. A adaptação de domínio é o ganho de desempenho que um modelo pré-treinado obtém quando seu treinamento continua sobre textos do domínio em que ele será aplicado (GURURANGAN et al., 2020). Aqui, esse domínio são os artigos da Wikipédia, com vocabulário, estilo e estrutura próprios: retreinado sobre eles, o modelo aprende essas regularidades e passa a prever melhor o conjunto de teste, formado pelo mesmo tipo de texto. Sem essa linha de controle, seria impossível separar a recuperação do dano da poda da mera adaptação à base de dados. O modelo base foi, portanto, retreinado pelo mesmo número de passos e avaliado junto às demais configurações.

A segunda é a **preservação da esparsidade**. Na poda não estruturada, os pesos podados não são removidos, e sim zerados: continuam ocupando suas posições nas matrizes (Seção 5.2.2). Para o otimizador, um peso zerado é um parâmetro como outro qualquer. Seu gradiente, que indica em que direção alterar o peso para reduzir a perda, em geral não é nulo, e o otimizador o alteraria já no primeiro passo. Ao final do retreinamento, os pesos anulados teriam voltado a valores diferentes de zero, o modelo estaria novamente denso, e o resultado deixaria de medir a recuperação de um modelo podado.

Para evitar isso, usou-se uma **máscara binária fixa**, como em Han et al. (2015). A máscara é uma matriz do mesmo tamanho de cada matriz de pesos podável, com valor 1 nas posições dos pesos que permaneceram após a poda e 0 nas posições dos pesos anulados. Ela é capturada uma única vez, logo após a poda, e não muda durante o treinamento, razão pela qual é chamada de fixa. Depois de cada passo do otimizador, cada matriz de pesos é multiplicada, elemento a elemento, por sua máscara: os pesos nas posições de valor 1 conservam o valor que o otimizador acabou de lhes atribuir, e os pesos nas posições de valor 0 voltam a zero. Por exemplo, com valores ilustrativos, em uma linha de quatro pesos com máscara (1; 0; 1; 0), se um passo do otimizador levar os pesos a (0,80; 0,02; −0,30; 0,01), a multiplicação os devolve a (0,80; 0; −0,30; 0). Assim, os pesos que permaneceram são retreinados normalmente, enquanto os podados ficam em zero do início ao fim do treinamento. Como a máscara age sobre os próprios pesos, depois de cada passo, esse resultado não depende da forma como o otimizador calcula a atualização.

A esparsidade medida ao final do treinamento foi registrada como verificação de integridade e coincidiu, nas duas configurações de magnitude, com a medida logo após a poda (Seção 6.5.4). Na poda estruturada não houve máscara, pois as estruturas já haviam

sido fisicamente removidas: as matrizes ficaram menores, e não restou nenhuma posição zerada a preservar.

5.4.4 Parâmetros do Treinamento

Para o treinamento, as linhas da partição de treino do WikiText-2 foram concatenadas em um único texto, tokenizado e dividido em blocos contíguos e disjuntos de 512 *tokens*, isto é, sem sobreposição entre blocos; o resto final, menor que um bloco, foi descartado. Resultaram 4.743 blocos, ou 2.428.416 *tokens*. É o procedimento indicado na documentação da biblioteca `transformers` para o *fine-tuning* de modelos de linguagem causais, como o GPT-2 (Hugging Face, 2026a).

Essa divisão difere da adotada na avaliação, que percorre o texto por janelas deslizantes de 1.024 *tokens* com passo de 512, sobrepostas em metade de seu comprimento (Seção 5.1.3.1), porque as duas etapas têm objetivos diferentes. Em ambas, uma única passagem do modelo sobre um trecho de texto produz uma previsão para cada *token* do trecho, feita a partir dos *tokens* que o antecedem dentro dele. O primeiro *token* de um trecho é, portanto, previsto quase sem contexto, e o último, com o trecho inteiro como contexto. Na avaliação, o objetivo é medir o desempenho do modelo, e previsões feitas com pouco contexto o subestimariam. A sobreposição evita isso: cada janela repete os 512 *tokens* finais da anterior apenas como contexto, sem contabilizá-los na perda, de modo que, a partir da segunda janela, cada *token* avaliado dispõe de pelo menos 512 *tokens* anteriores e entra no cálculo uma única vez.

No treinamento, o objetivo é ajustar os pesos a partir do texto, usando cada *token* como alvo de previsão uma vez por época. Previsões com pouco contexto não prejudicam esse objetivo, pois também são exemplos legítimos da tarefa de prever o próximo *token*, e os blocos disjuntos o atingem sem desperdício: cada *token* é alvo exatamente uma vez, e cada passagem do modelo aproveita todas as previsões que produz. Usar no treinamento as janelas da avaliação levaria a uma de duas situações, ambas desvantajosas. Se todos os *tokens* de cada janela contassem na perda, cada *token* seria usado duas vezes por época, já que cada janela repete metade da anterior: a época equivaleria a duas passagens pelo texto, com cerca do dobro do custo, e janelas consecutivas seriam amostras parcialmente repetidas, e não independentes. Se apenas os *tokens* novos contassem, como na avaliação, metade de cada janela serviria só de contexto: o modelo processaria dois *tokens* para cada *token* de treino, o que dobraria, aproximadamente, o tempo e o consumo de energia do retreinamento sem acrescentar nenhum *token* de treino.

A diferença entre as duas divisões não afeta a comparação entre os modelos: o treinamento usa apenas a partição de treino, e todas as *perplexities*, antes e depois do retreinamento, foram medidas na partição de teste pelo mesmo procedimento de janelas deslizantes. A Tabela 8 reúne os parâmetros adotados, idênticos nas sete configurações.

Tabela 8 – Parâmetros do *fine-tuning* de recuperação.

Parâmetro	**Valor**
Blocos de treino (512 *tokens* cada)	4.743
Blocos por lote	4
Lotes acumulados por passo do otimizador	4
Lote efetivo por passo	16 blocos (8.192 *tokens*)
Passos do otimizador	297 (uma época)
Otimizador	AdamW
Taxa de aprendizado máxima	5×10^{-5}
Aquecimento linear da taxa de aprendizado	17 passos (6% do total, truncado)
Decaimento da taxa de aprendizado	linear, até zero no passo 297
Decaimento de pesos	0,01
Limite da norma do gradiente	1,0
Precisão numérica	mista (FP16 e FP32)
Semente aleatória	42

Fonte: Elaboração própria.

Os gradientes de quatro lotes consecutivos, de quatro blocos cada, foram acumulados antes de cada passo do otimizador, o que equivale a um lote efetivo de 16 blocos; a perda de cada lote foi dividida por quatro, de modo que o gradiente acumulado correspondesse à média dos lotes, e não à soma. O número de passos foi calculado para corresponder a uma época, isto é, a uma passagem pelo conjunto de treino: os 4.743 blocos, divididos em lotes efetivos de 16, resultam em 297 passos, com arredondamento para cima.

O otimizador AdamW (LOSHCHILOV; HUTTER, 2019) aplica o decaimento de pesos separadamente da atualização pelo gradiente. O decaimento foi aplicado apenas às matrizes de peso, deixando de fora vieses e parâmetros de normalização, conforme prática consolidada. A taxa de aprendizado cresceu linearmente durante os 17 primeiros passos e decresceu linearmente até zero no último, e a norma do gradiente foi limitada a 1,0 antes de cada passo. A taxa de aprendizado e o lote efetivo estão entre os valores recomendados por Devlin et al. (2019) para o *fine-tuning* de *Transformers* pré-treinados — taxa de 2×10^{-5} a 5×10^{-5} e lotes de 16 ou 32 exemplos —; o decaimento de pesos de 0,01 e o aquecimento seguido de decaimento linear são os adotados por esses autores no pré-treinamento do BERT. Nenhum desses valores foi ajustado para este experimento (Seção 5.9). A quantidade de retreinamento, uma época, fica abaixo das duas a quatro épocas recomendadas pelos mesmos autores; o que essa limitação impede de concluir é discutido na Seção 6.5.4.

A passagem direta e a retropropagação executaram em **precisão mista** (MICIKEVICIUS et al., 2018): as operações foram realizadas em meia precisão (FP16), com os pesos mantidos em precisão simples (FP32) e escalonamento dinâmico da perda, que evita o subtransbordamento dos gradientes em meia precisão. A escolha aproveita as uni-

dades de cálculo especializadas da GPU Tesla T4, que alcança 65 TFLOPS em precisão mista contra 8,1 TFLOPS em precisão simples (NVIDIA Corporation, 2019), e corresponde à configuração pressuposta na estimativa de memória do critério C3 (Seção 4.1.1). Ela introduz, porém, uma fonte adicional de variação numérica que não existe nas etapas *one-shot*, todas executadas em precisão simples. O consumo de energia do treinamento foi medido pela biblioteca CodeCarbon separadamente do consumo da avaliação: trata-se de um custo pago uma única vez, que precisa ser amortizado contra a economia por inferência para que o balanço energético do trabalho feche corretamente.

Diferentemente de todas as etapas anteriores, esta é estocástica — a ordem dos lotes e o *dropout* introduzem variabilidade — e, portanto, é a única em que a semente aleatória efetivamente altera o resultado. Cada configuração foi retreinada uma única vez, com a semente 42, reiniciada antes de cada configuração. A ordem dos lotes, sorteada por um gerador próprio iniciado com essa semente, foi a mesma nas sete configurações, de modo que as diferenças entre elas não decorrem da ordem em que os dados foram apresentados. A ausência de repetições com sementes distintas é discutida entre as limitações (Seção 5.9).

5.5 *Framework* de Implementação

A implementação foi feita em Python, com a biblioteca PyTorch para construção, manipulação e treinamento dos modelos. O PyTorch foi escolhido por seu sistema de diferenciação automática, que constrói dinamicamente o grafo computacional e calcula os gradientes da retropropagação durante o *fine-tuning*, e por seu suporte nativo à aceleração em GPU.

Os pesos pré-treinados foram obtidos na plataforma Hugging Face por meio da biblioteca `transformers`, que implementa sobre o PyTorch as arquiteturas dos modelos candidatos. A Hugging Face é uma plataforma colaborativa de hospedagem e distribuição de modelos, e não uma biblioteca: em 28 set. 2026, hospedava mais de 3,1 milhões de modelos, entre eles diversas famílias de modelos de linguagem, como GPT-2, OPT, Pythia, Llama e BERT.

As demais bibliotecas empregadas são:

- `datasets` (LHOEST et al., 2021): carregamento do WikiText-2 a partir da plataforma Hugging Face, já dividido em suas partições oficiais de treino, validação e teste;
- `thop` (ZHU, 2022): contagem do custo computacional de uma inferência, módulo a módulo. A biblioteca conta multiplicações seguidas de soma (MACs, do inglês *multiply-accumulate operations*), e cada uma foi convertida em duas operações de

ponto flutuante (FLOPs). Como a biblioteca não reconhece o módulo `Conv1D`, com que a `transformers` implementa as projeções lineares do GPT-2, acrescentou-se uma regra de contagem para ele;

- `CodeCarbon` (COURTY et al., 2023): medição empírica do consumo de energia, a partir do monitoramento do *hardware*.

As duas estratégias de poda foram implementadas diretamente sobre as matrizes de pesos, sem bibliotecas dedicadas à poda. Na poda estruturada, essa implementação própria foi necessária: a versão 5 da biblioteca `transformers` removeu a interface de poda de cabeças de atenção que existia nas versões anteriores. A remoção física das estruturas foi, portanto, feita no próprio código: as matrizes foram reconstruídas sem as linhas e colunas das estruturas removidas, e os atributos do módulo de atenção que registram o número de cabeças e a largura das projeções foram atualizados, para que o modelo continue dividindo as matrizes corretamente entre as cabeças restantes.

A implementação da poda é verificada por um conjunto de 15 testes automatizados (`test_poda`), executados sobre um GPT-2 de dimensões reduzidas e pesos aleatórios: o que se verifica é o funcionamento da poda, que não depende dos pesos pré-treinados. Os 15 testes foram aprovados. O teste central é o de **equivalência numérica**. Remover fisicamente um conjunto de cabeças de atenção, ou de neurônios do MLP, deve produzir as mesmas saídas que apenas anular os pesos correspondentes no modelo de dimensões originais, a menos de diferenças de arredondamento (tolerância absoluta de 10^{-5} e relativa de 10^{-4}). As duas operações têm de ser equivalentes: anular as fatias de consulta, chave e valor de uma cabeça zera seu vetor de valores, de modo que sua contribuição à saída da atenção se anula qualquer que seja a distribuição de atenção calculada; e anular os pesos de entrada e o viés de um neurônio do MLP zera sua ativação, pois a função GELU vale zero em zero. Por isso, qualquer erro de indexação na reconstrução das matrizes, como remover a linha ou a coluna errada, quebraria a equivalência e seria detectado pelo teste.

Os demais testes verificam que os atributos do módulo de atenção permanecem coerentes com as dimensões das matrizes; que a poda a taxa zero não altera o modelo; que a salvaguarda da poda estruturada (Seção 5.2.3) preserva ao menos uma cabeça e um neurônio por camada, mesmo com taxa de poda de 100%; que as estruturas removidas são as de menor norma L_1; que a poda por magnitude atinge a esparsidade pedida, zera os pesos de menor valor absoluto e preserva as dimensões das matrizes; que a regra do esquema de poda mantém a média das taxas por camada igual à taxa de poda e poda menos a camada mais sensível; que, com $\beta = 0$, o esquema de poda reproduz a varredura uniforme na poda estruturada e difere dela na poda por magnitude (Seção 5.3.2); e que a máscara binária mantém os pesos podados em zero após um passo do otimizador (Seção 5.4.3).

5.6 *Pipeline* Experimental

O código está organizado em módulos, cada um responsável por uma única etapa do experimento, o que permite que todas as estratégias de poda reutilizem exatamente a mesma rotina de avaliação. Essa é a condição para que as diferenças medidas entre as configurações se devam à poda, e não a diferenças na forma de medir. A Tabela 9 descreve os módulos e a Figura 11 representa o fluxo experimental, no qual todas as estratégias convergem para a mesma rotina de avaliação, o que garante que todas as configurações sejam medidas da mesma forma.

Tabela 9 – Organização modular da implementação.

Módulo	**Responsabilidade**
`config`	Configuração central: modelo, base de dados, parâmetros de avaliação, semente e ambiente.
`utils`	Fixação das sementes e escrita incremental dos resultados em arquivo.
`data`	Carregamento e tokenização da base de dados.
`model`	Carregamento do GPT-2 pré-treinado.
`eval`	*Perplexity* por janela deslizante e métricas de custo computacional.
`energy`	Medição de energia e emissões.
`prune_magnitude`	Poda não estruturada por magnitude, nos dois esquemas de poda.
`prune_structured`	Poda estruturada de cabeças e neurônios, com remoção física.
`sensitivity`	Perfil de sensibilidade (fase 1) e esquema de poda com análise de sensibilidade (fase 2).
`finetune`	*Fine-tuning* de recuperação, com preservação de esparsidade.
`run`	Orquestração da avaliação do modelo base.
`test_poda`	Testes de corretude da poda, descritos na Seção 5.5.

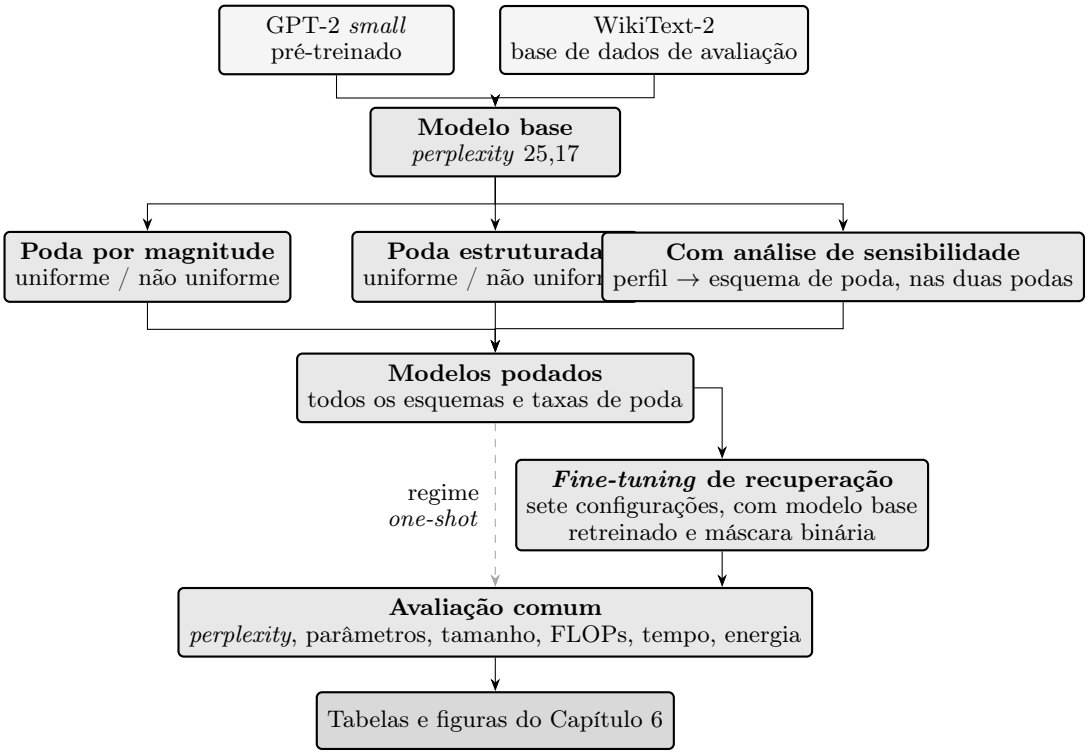
Fonte: Elaboração própria.

Como indica a linha tracejada da figura, todos os modelos podados foram avaliados em regime *one-shot*, isto é, logo após a poda, sem retreinamento; apenas as sete configurações da Seção 5.4.1 passaram também pelo *fine-tuning* antes de uma nova avaliação. Essa bifurcação é o que permite separar, nos resultados, o dano causado pela poda daquilo que o retreinamento consegue recuperar.

5.7 Ambiente de Execução e Reprodutibilidade

Os experimentos foram executados na plataforma Kaggle, escolhida por oferecer acesso gratuito a GPU dedicada, com sessões longas o bastante para as varreduras com-

Figura 11 – Fluxo experimental do trabalho.



Fonte: Elaboração própria.

pletas (Tabela 10). Todas as sessões usaram uma GPU NVIDIA Tesla T4, com 16 GB de memória (NVIDIA Corporation, 2019). A implementação detecta automaticamente o dispositivo disponível, o que permite executar o código localmente, em CPU, para testes rápidos, sem alteração de código.

Quanto à reprodutibilidade, dois aspectos merecem registro.

O primeiro é que as varreduras *one-shot* são **determinísticas**. Não há treinamento, o modelo opera em modo de avaliação, o conjunto de teste é fixo e os critérios de poda são ordenações determinísticas sobre pesos fixos. Verificou-se que execuções independentes, em sessões distintas, reproduzem a *perplexity* em todos os dígitos registrados: as sete configurações do *fine-tuning*, podadas novamente em outra sessão, partiram exatamente dos valores das varreduras *one-shot* (Seção 5.4.2). Repetições com sementes diferentes, nessas etapas, produziriam resultados idênticos e não acrescentariam informação. A semente só influencia o resultado no *fine-tuning*, a única etapa estocástica do trabalho.

O segundo é que as medições de **tempo e energia dependem da sessão**. O mesmo modelo base, sem qualquer poda, registrou tempos de inferência de 40,5, 54,0 e 46,3 ms por janela em três sessões diferentes, e consumos de energia entre 1,176 e 1,616 Wh por avaliação completa do conjunto de teste (Seção 6.1). A variação é atribuída às diferenças entre as máquinas físicas alocadas a cada sessão e à carga concorrente nelas.

Por essa razão, comparações de custo entre varreduras executadas em sessões distintas foram sempre feitas em termos relativos ao modelo base da própria sessão, nunca em valores absolutos.

5.8 Síntese das Decisões de Implementação

Todo desenho experimental implica escolhas que condicionam o escopo dos resultados. A Tabela 10 sintetiza as principais decisões adotadas ao longo deste capítulo, as alternativas consideradas e a justificativa de cada uma.

Tabela 10 – Síntese das principais decisões de implementação do trabalho.

Decisão	**Alternativas consideradas**	**Escolha**	**Justificativa**
Base de dados	WikiText-103, Penn Treebank, OpenWebText, The Pile	**WikiText-2**	Atende aos quatro critérios da Seção 4.1.2; entre os que atendem, tem o menor conjunto de treino (Tabela 4).
Modelo	GPT-2 *medium*/*large*/XL, DistilGPT2, OPT-125M, Pythia-160M, BERT, Llama 2	**GPT-2 *small***	Atende aos seis critérios da Seção 4.1.1 e forma com o WikiText-2 um par que atende a D1–D4; entre os que atendem, é o de menor número de parâmetros (Tabela 2).
Estratégias de poda	SparseGPT, Wanda, *movement pruning*, poda de cabeças por gradiente	**Duas** (magnitude e estruturada)	Únicas alternativas que atendem a P1 e P2; juntas, atendem a P3 (Tabela 6). A restrição de tempo experimental inviabilizaria uma terceira com a mesma profundidade.
Regime de poda	Iterativo (podar e retreinar em ciclos)	***One-shot* primeiro**	Isola o efeito da poda do efeito do retreinamento; o *fine-tuning* entra depois, como etapa separada e mensurável.
Estratégia otimizada	Taxas dinâmicas, variação do momento da poda, destilação de conhecimento, quantização	**Sensibilidade por camada**	Altera apenas a distribuição da poda entre as camadas, objeto da hipótese do trabalho (Seção 4.3); as alternativas exigem treinamento durante a poda, um segundo modelo ou outra forma de compressão.
Métrica principal	*Accuracy* em tarefas descendentes	**Perplexity**	Métrica intrínseca padrão para modelos de linguagem, mensurável sem *fine-tuning* em tarefa específica.
Ambiente	Máquina local, Google Colab	**Kaggle (GPU T4)**	Acesso gratuito a GPU dedicada com sessões longas o bastante para as varreduras completas.

Fonte: Elaboração própria.

5.9 Limitações Metodológicas

Reconhecem-se, desde o desenho experimental, limitações que condicionam o escopo dos resultados:

- **Um único modelo e uma única base de dados.** Os experimentos empregaram exclusivamente o GPT-2 *small* sobre o WikiText-2. Os resultados podem não se transferir diretamente para modelos de escala substancialmente maior ou para bases de dados de natureza distinta. A implementação foi mantida parametrizável justamente para viabilizar essa verificação em trabalhos futuros.

- **Duas estratégias de poda.** As estratégias foram escolhidas pelos critérios P1 a P3 (Seção 5.2.1), que excluem as que estimam a importância com dados ou ao longo do treinamento, como o SparseGPT e o Wanda, referências recentes da poda de LLMs. As conclusões referem-se, portanto, a critérios de poda calculados apenas a partir dos pesos, e não se avaliou o esquema de poda com análise de sensibilidade aplicado a essas estratégias.

- **Faixa restrita de taxas-sonda no perfil de sensibilidade.** O perfil que alimenta o esquema de poda foi levantado a uma única taxa-sonda, de 50%, e sua estabilidade foi verificada por uma réplica a 30% (Seção 6.3.5). As duas taxas pertencem, porém, ao mesmo regime intermediário de poda: nada garante que a ordenação das camadas por sensibilidade se mantenha em taxas muito menores ou muito maiores.

- **Sensibilidade medida em isolamento.** A taxa-sonda é aplicada a uma camada por vez, o que ignora as interações entre camadas. Como mostra a Seção 6.3.6, a degradação de camadas podadas em conjunto supera a soma de suas degradações individuais: na poda por magnitude, em escala logarítmica, a soma das degradações das camadas podadas uma a uma é 1,238, enquanto a poda simultânea de todas elas a 50% produz degradação de 2,934, mais que o dobro. O perfil deve, por isso, ser lido como uma ordenação das camadas por sensibilidade, e não como uma estimativa da *perplexity* do modelo com várias camadas podadas.

- **Perfil medido na partição de teste.** As degradações que compõem o perfil de sensibilidade foram medidas na mesma partição de teste em que os modelos podados são avaliados (Seção 5.3.1). O esquema de poda com análise de sensibilidade utiliza, assim, doze valores por estratégia obtidos dessa partição, informação de que os esquemas de poda uniforme e não uniforme não dispõem; também o parâmetro de intensidade e as configurações retreinadas foram escolhidos a partir de resultados medidos nela (Seções 6.4.4 e 5.4.1). Parte da vantagem observada pode, por isso, refletir ajuste a essa partição, e este trabalho não mediu quanto: levantar o perfil na partição de validação é a verificação que separaria os dois efeitos.

- **Dependência do *hardware*.** Tempo e energia dependem da máquina em que são medidos. Os valores absolutos não devem ser comparados entre ambientes distintos, nem mesmo entre sessões da mesma plataforma; a análise apoia-se em comparações relativas ao modelo base medido na mesma sessão (Seção 5.7).

- **Semente única no *fine-tuning*.** Esta é a única etapa estocástica do experimento, e todas as configurações foram executadas com uma única semente. As *perplexities* pós-retreinamento não vêm, portanto, acompanhadas de estimativa de variância, e diferenças pequenas entre configurações retreinadas devem ser lidas com a cautela correspondente. A comparação principal da etapa — esquema de poda com análise de sensibilidade contra esquema de poda não uniforme — é afetada por essa limitação justamente na taxa de poda de 10%, em que os dois praticamente empatam após o retreinamento: *perplexity* de 25,96 com análise de sensibilidade e de 25,33 no esquema de poda não uniforme (Seção 6.5).

Apesar dessas limitações, o desenho descrito é suficiente para responder às duas perguntas formuladas na abertura do Capítulo 4, que são o objeto do Capítulo 6.

6 Resultados

Este capítulo apresenta os resultados dos experimentos descritos no Capítulo 5, organizados por tipo de métrica, de modo que cada tabela compare uma única grandeza. A Seção 6.1 apresenta os valores medidos para o modelo base e as condições de leitura das tabelas. A Seção 6.2 apresenta o custo computacional e o consumo de energia dos modelos podados, que respondem à pergunta (A). A Seção 6.3 apresenta o perfil de sensibilidade por camada, e a Seção 6.4 compara o desempenho preditivo dos esquemas de poda com e sem análise de sensibilidade, o que responde à pergunta (B). A Seção 6.5 apresenta os resultados do *fine-tuning* de recuperação, e a Seção 6.6 combina desempenho preditivo e energia nos indicadores definidos na Seção 4.1.3. Por fim, a Seção 6.7 sintetiza os resultados.

6.1 Protocolo e Modelo Base

Esta seção apresenta os valores medidos para o modelo base e as condições que orientam a leitura das tabelas das seções seguintes.

O modelo base, avaliado pelo procedimento da Seção 5.1.3.1, apresentou *perplexity* de 25,17 no conjunto de teste do WikiText-2. As demais métricas do modelo base constam da Tabela 3. O valor obtido é inferior aos 29,41 publicados por Radford et al. (2019). Essa diferença é compatível com o efeito do procedimento de avaliação. Com janelas deslizantes sobrepostas, cada *token* é predito com pelo menos 512 *tokens* de contexto, o que reduz a *perplexity* em relação à avaliação em janelas disjuntas. A documentação da biblioteca `transformers` registra esse efeito no GPT-2 *large*: 19,44 com janelas disjuntas de 1.024 *tokens*, valor próximo dos 19,93 publicados, e 16,44 com passo de 512 (Hugging Face, 2026b), uma redução de cerca de 15%. Uma redução da mesma ordem, de cerca de 14%, separa os 29,41 publicados para o GPT-2 *small* dos 25,17 medidos neste trabalho, obtidos pelo mesmo procedimento daquela documentação.

Duas condições orientam a leitura das tabelas apresentadas a seguir.

A primeira é o regime de avaliação. Com exceção da Seção 6.5, todos os resultados foram obtidos em regime *one-shot*, isto é, sem retreinamento após a poda (Seção 4.4). Por essa razão, as *perplexities* reportadas não são comparáveis às de trabalhos que retreinam o modelo após a poda. Elas também não devem ser lidas como proposta de uso dos modelos podados, mas como medida do dano que cada estratégia, com cada esquema de poda, causa por si só. A Seção 6.5 mostra que esse dano é parcialmente reversível.

A segunda é a dependência entre as medições de custo e a sessão de execução. As

varreduras foram executadas em sessões distintas do ambiente Kaggle, e o mesmo modelo base registrou tempos de inferência de 40,5 ms, 54,0 ms e 46,3 ms por janela em três delas. O consumo de energia do modelo base variou de forma análoga, entre 1,176 e 1,616 Wh por avaliação completa do conjunto de teste. Por essa razão, a energia é sempre expressa pela redução relativa ΔE (Equação 4.1), calculada em relação ao modelo base medido na mesma sessão, e nunca em valores absolutos. O tempo de inferência segue a mesma regra.

O resumo por varredura (Seção 4.1.3) considera as taxas de poda de 10% a 50%. Esse limite decorre do critério definido naquela seção: 50% foi a maior taxa em que nenhuma camada atingiu o teto $r_{\max} = 0{,}95$ em nenhum dos esquemas de poda com análise de sensibilidade. Com $\beta = 1{,}0$, a 60%, uma camada já atingiu o teto na poda por magnitude e duas na poda estruturada. A 90%, foram 10 e 11 das 12 camadas, respectivamente.

Esse limite não exclui configurações de interesse prático. A 60%, a menor *perplexity* entre as oito configurações avaliadas, quatro esquemas de poda em cada uma das duas estratégias, foi de 1.524,55, cerca de 61 vezes a do modelo base. As taxas de 60% a 90% são reportadas nas tabelas, mas não entram nas médias.

6.2 Custo Computacional e Consumo de Energia

Esta seção responde à pergunta (A), formulada no Capítulo 4: em que medida a redução de parâmetros se converte em redução de tempo de inferência e de consumo de energia. A Tabela 11 apresenta as grandezas calculadas a partir da arquitetura: parâmetros, tamanho e FLOPs. A Tabela 12 apresenta a grandeza medida durante a execução, a redução relativa de energia.

Na Tabela 11, os parâmetros da poda por magnitude correspondem aos não nulos, e os valores referem-se ao esquema de poda uniforme. O esquema de poda não uniforme produz os mesmos valores, pois nenhum dos dois altera a arquitetura. Na poda estruturada, os valores referem-se ao esquema de poda não uniforme, e os demais esquemas removem frações de parâmetros que diferem dele em até 2,1 pontos percentuais. Na Tabela 12, ΔE (Equação 4.1) é calculado em relação ao modelo base medido na mesma sessão. Nas duas tabelas, β corresponde ao parâmetro de intensidade (Seção 5.3.2).

Na poda por magnitude, o custo calculado não foi reduzido. Entre as taxas de 0% e 90%, o número de parâmetros não nulos passou de 124,4 M para 48,0 M, uma redução de 61%. O tamanho em memória e os FLOPs, entretanto, permaneceram em 474,70 MB e 126,58 G nas 10 taxas. Esse resultado decorre de os pesos serem anulados, e não removidos (Seção 5.2.2): as matrizes conservam suas dimensões, e as operações executadas são as mesmas. Assim, a redução de 61% aparece na contagem de parâmetros, mas não na memória ocupada nem nas operações executadas.

Tabela 11 – Custo calculado dos modelos podados, por taxa de poda.

	Parâmetros (M)		**Tamanho (MB)**		**FLOPs (G)**	
Taxa de poda	**Magn.**	**Estrut.**	**Magn.**	**Estrut.**	**Magn.**	**Estrut.**
0%	124,4	124,4	474,7	474,7	126,6	126,6
10%	115,9	116,0	474,7	442,6	126,6	118,0
20%	107,5	107,6	474,7	410,5	126,6	109,3
30%	99,0	99,0	474,7	377,6	126,6	100,5
40%	90,5	90,6	474,7	345,5	126,6	91,9
50%	82,0	81,9	474,7	312,6	126,6	83,1
60%	73,5	73,5	474,7	280,5	126,6	74,5
70%	65,0	65,1	474,7	248,3	126,6	65,9
80%	56,5	56,5	474,7	215,5	126,6	57,0
90%	48,0	48,1	474,7	183,3	126,6	48,4

Fonte: Elaboração própria.

Tabela 12 – Redução relativa de energia, por taxa de poda (%).

		Taxa de poda								
Análise	**Esquema de poda**	10%	20%	30%	40%	50%	60%	70%	80%	90%
Poda por magnitude										
Sem análise de sensibilidade	Não uniforme	1,7	2,6	2,8	3,7	4,1	5,3	6,2	7,7	9,0
	Uniforme	2,1	3,0	2,4	3,1	4,4	5,0	5,9	7,4	8,9
Com análise de sensibilidade	$\beta = 1{,}0$	3,8	3,8	3,9	4,7	5,8	5,8	6,4	6,9	6,8
	$\beta = 0{,}5$	3,5	3,5	3,8	4,3	5,0	5,6	6,3	6,9	7,1
Poda estruturada										
Sem análise de sensibilidade	Não uniforme	2,5	5,5	13,5	21,4	28,6	36,2	44,2	51,1	58,7
	Uniforme	5,3	11,8	17,6	24,8	34,3	40,9	45,8	54,5	60,0
Com análise de sensibilidade	$\beta = 1{,}0$	3,4	11,0	18,1	25,0	32,6	39,9	47,1	54,4	61,6
	$\beta = 0{,}5$	2,6	10,0	17,3	24,5	33,1	40,5	46,9	55,0	61,6

Fonte: Elaboração própria.

A Tabela 12 registra, entretanto, reduções de energia de 1,7% a 9,0% na poda por magnitude. Esse resultado não contradiz o anterior, pois as duas tabelas apresentam grandezas de naturezas diferentes. O tamanho e os FLOPs são calculados a partir das dimensões das matrizes e, portanto, não dependem dos valores dos pesos: um peso anulado é contado como qualquer outro. A energia e o tempo de inferência, por sua vez, são medidos durante a execução e podem variar mesmo quando a arquitetura é a mesma.

Nas varreduras da poda por magnitude, ΔE cresceu lentamente com a taxa de poda, até cerca de 9% na taxa de 90%. O tempo de inferência também diminuiu: 5,4% entre os extremos, de 53,96 para 51,04 ms no esquema de poda uniforme. Essas reduções foram menores que a variação do próprio modelo base entre sessões (Seção 6.1). Além disso, elas não se acumularam de uma varredura para a seguinte: a varredura não uniforme

terminou com 1,429 Wh, a 90%, e a varredura uniforme, executada em seguida, registrou 1,572 Wh a 0%.

A causa dessas reduções não pode ser determinada por este protocolo, pois há duas explicações possíveis, que ele não separa. A primeira é um efeito dos próprios pesos nulos. Gregersen, Patel e Choukse (2024) mediram, em outras GPUs, que a potência consumida na multiplicação de matrizes depende dos valores dos operandos e diminui com a proporção de zeros, embora o número de operações e o tempo de execução permaneçam os mesmos. Esse efeito, entretanto, não explicaria a redução do tempo de inferência. A segunda é um efeito da ordem de execução. Em cada varredura, as taxas foram executadas em ordem crescente, de modo que qualquer variação ao longo da execução se confunde com o efeito da taxa. Verificar qual das duas ocorreu exigiria executar as taxas em outra ordem, o que não foi feito.

Qualquer que seja a causa, essas reduções fornecem uma referência para o restante da análise. Como ocorreram sem que nenhuma estrutura fosse removida, reduções de até cerca de 10% em tempo ou energia não são consideradas evidência de ganho da poda. Esse valor, definido a partir desses resultados, é adotado como **limiar de detecção** do protocolo. Segundo esse critério, os valores da poda por magnitude na Tabela 12 são reportados, mas não são interpretados como redução de consumo decorrente da poda: na GPU Tesla T4 utilizada, sem suporte de *hardware* ou de *kernels* para esparsidade, a poda não estruturada não apresentou ganho de eficiência detectável.

Na poda estruturada, o custo foi reduzido de forma proporcional ao número de parâmetros. Com 90% das estruturas removidas, no esquema de poda não uniforme, o tamanho em memória diminuiu 61%, os FLOPs, 62%, a energia, 59%, e o tempo de inferência, 56%. Essas reduções acompanharam a redução de 61% no número de parâmetros. Nesse esquema de poda, a redução de energia superou o limiar de detecção a 30% (13,5%) e, a 90%, foi quase seis vezes maior que ele. Os resultados das duas estratégias respondem, em conjunto, à pergunta (A): na GPU Tesla T4 utilizada, a redução de parâmetros só se converteu em redução detectável de consumo quando a poda removeu estruturas inteiras da arquitetura, e não quando apenas anulou pesos.

Os esquemas de poda com análise de sensibilidade apresentaram o mesmo comportamento, o que era esperado, pois alteram a distribuição da poda entre as camadas, e não o tipo de elemento removido. Na poda por magnitude, ΔE permaneceu abaixo do limiar de detecção em todos os esquemas de poda. Na poda estruturada, os quatro esquemas de poda apresentaram valores próximos de ΔE em cada taxa. A 50%, por exemplo, os valores foram de 28,6% (não uniforme), 34,3% (uniforme), 32,6% ($\beta = 1{,}0$) e 33,1% ($\beta = 0{,}5$). A maior diferença entre os esquemas, de 6,3 pontos percentuais, ocorreu a 20%. Essa diferença é da ordem da variação do próprio modelo base dentro de um mesmo arquivo de resultados: 1,176 Wh na varredura não uniforme e 1,261 Wh na uniforme,

uma diferença de 7%. O esquema de poda não uniforme registrou o menor ΔE em todas as taxas. Entretanto, como essa diferença não se separa da variação de medição, não é possível atribuir vantagem de energia a nenhum esquema de poda.

6.3 Perfil de Sensibilidade por Camada

Esta seção apresenta o perfil de sensibilidade, medido nas duas taxas-sonda definidas na Seção 5.3.1, e o esquema de poda que dele resulta.

6.3.1 Perfil Obtido

A Tabela 13 apresenta os perfis das duas taxas-sonda, e a Figura 12 representa graficamente o de 50%, do qual derivam os esquemas de poda. Na tabela, cada valor corresponde à degradação Δ_ℓ (Equação 5.1) da camada indicada. Entre parênteses, consta o posto da camada na ordenação de sensibilidade, em que 1 indica a mais sensível. Na figura, as escalas verticais diferem entre os dois gráficos. Além disso, na poda estruturada, a barra da camada 0 excede o eixo e está truncada.

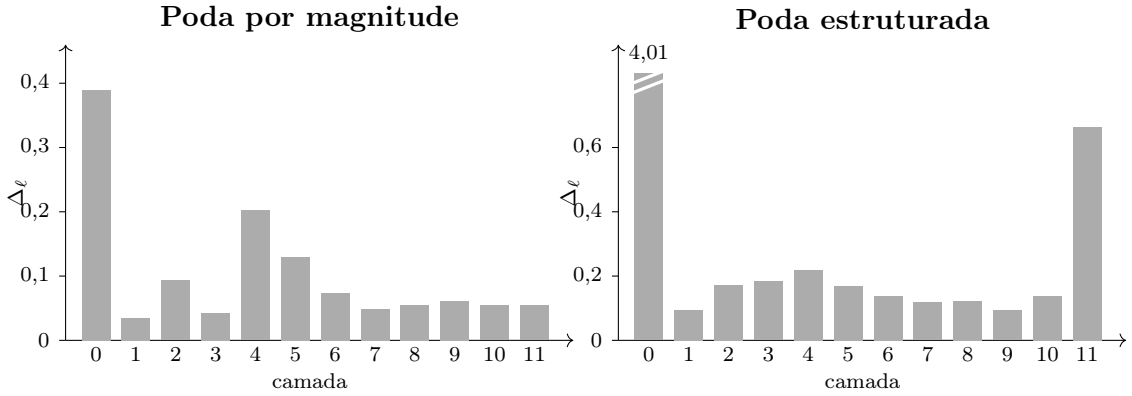
Tabela 13 – Perfil de sensibilidade por camada do GPT-2 *small* no WikiText-2, nas duas taxas-sonda.

	Poda por magnitude		**Poda estruturada**	
Camada	**Sonda 30%**	**Sonda 50%**	**Sonda 30%**	**Sonda 50%**
0	0,041 (1)	0,390 (1)	3,423 (1)	4,009 (1)
1	0,007 (12)	0,035 (12)	0,159 (3)	0,094 (11)
2	0,012 (7)	0,093 (4)	0,130 (7)	0,173 (5)
3	0,007 (11)	0,043 (11)	0,143 (5)	0,183 (4)
4	0,039 (2)	0,202 (2)	0,156 (4)	0,218 (3)
5	0,035 (3)	0,130 (3)	0,093 (8)	0,169 (6)
6	0,016 (4)	0,074 (5)	0,132 (6)	0,138 (7)
7	0,008 (10)	0,049 (10)	0,053 (12)	0,118 (10)
8	0,011 (8)	0,055 (7)	0,065 (10)	0,122 (9)
9	0,014 (5)	0,061 (6)	0,060 (11)	0,093 (12)
10	0,008 (9)	0,054 (9)	0,088 (9)	0,138 (8)
11	0,013 (6)	0,054 (8)	0,539 (2)	0,663 (2)
Razão máx./mín.	6,1×	11,3×	65,1×	42,9×

Fonte: Elaboração própria.

O primeiro resultado é que a sensibilidade não foi uniforme entre as camadas, o que confirma a premissa da técnica. À taxa-sonda de 50%, na poda por magnitude, a camada mais sensível degradou o modelo 11,3 vezes mais que a mais tolerante. Na poda estruturada, essa razão foi de 42,9. Portanto, o esquema de poda uniforme, que aplica a

Figura 12 – Degradação causada pela poda isolada de cada camada, à taxa-sonda de 50%.



Fonte: Elaboração própria.

mesma taxa às 12 camadas, desconsidera uma diferença de uma ordem de grandeza na degradação causada pela poda de cada uma delas.

6.3.2 Camadas Mais Sensíveis

O segundo resultado refere-se à camada 0, a mais sensível nas duas estratégias. Na poda estruturada, o efeito foi mais acentuado: a remoção de metade das cabeças de atenção e dos neurônios do MLP apenas da camada 0 elevou a *perplexity* de 25,17 para 1.386,43, ou seja, 55 vezes o valor do modelo base. Como comparação, a poda estruturada com esquema de poda uniforme a 50%, que poda as 12 camadas, resultou em *perplexity* de 4.573,00. Em escala logarítmica, a camada 0 respondeu, isoladamente, por $4{,}01/5{,}20 \approx 77\%$ da degradação produzida pela poda uniforme de todo o modelo.

Uma hipótese compatível com esse resultado decorre da posição da camada 0. Nessa camada, as representações deixam de ser a soma direta dos *embeddings* de *token* e de posição e passam a ser contextualizadas pela primeira vez. Dessa forma, um dano nessa camada alcança as 11 camadas seguintes. Essa hipótese é coerente com os resultados de Lad et al. (2025), que removeram camadas inteiras, ou trocaram a ordem de camadas adjacentes, em modelos de cinco famílias, entre eles o GPT-2 *small*. Em todos os modelos avaliados, a intervenção na primeira camada, que corresponde à camada 0 na numeração adotada neste trabalho, esteve entre as que mais degradaram o desempenho, enquanto a remoção de camadas intermediárias foi tolerada. Os autores interpretam a camada 0 como uma extensão da camada de *embedding* e atribuem às camadas iniciais a integração do contexto local às representações dos *tokens*. Segundo essa interpretação, sem a camada 0, as camadas seguintes passam a operar sobre representações fora da distribuição para a qual foram treinadas. Como o perfil mede a degradação, e não o seu mecanismo, essa hipótese não foi testada neste trabalho.

Na poda estruturada, a segunda camada mais sensível foi a camada 11, imediata-

mente anterior à cabeça de linguagem, com $\Delta_{11} = 0{,}663$, quase cinco vezes a mediana das camadas intermediárias. O perfil da poda estruturada apresentou, assim, formato aproximado de "U": as camadas das extremidades foram as mais sensíveis, e as intermediárias degradaram comparativamente pouco o modelo. Esse padrão não se repetiu na poda por magnitude, em que as camadas mais sensíveis foram a 0 e a 4, e a camada 11 ocupou o oitavo posto.

6.3.3 Comparação dos Perfis entre as Estratégias

O terceiro resultado é que os perfis das duas estratégias não coincidiram. A correlação de postos de Spearman (Equação 5.2), que vale 1 quando duas ordenações são idênticas e se aproxima de zero quando não há associação entre elas, foi de $\rho = 0{,}50$ entre os dois perfis, o que indica concordância parcial. As duas estratégias coincidiram em apontar a camada 0 como a mais sensível e as camadas 1 e 7 entre as mais tolerantes. Nas demais camadas, entretanto, as ordenações divergiram. A camada 11 foi a segunda mais sensível na poda estruturada e apenas a oitava na poda por magnitude. A camada 3, por sua vez, foi a segunda mais tolerante na poda por magnitude e a quarta mais sensível na poda estruturada.

Portanto, o GPT-2 *small* não possui um perfil de sensibilidade único. A sensibilidade corresponde a uma propriedade do par formado pela camada e pela estratégia de poda, e não da camada isoladamente. Esse resultado confirma a necessidade, prevista na Seção 4.3, de medir o perfil separadamente para cada estratégia.

6.3.4 Esquema de Poda Resultante

O perfil de sensibilidade foi convertido em taxas de poda por camada pela Equação 5.3 (Seção 5.3.2). Esta subseção apresenta as taxas resultantes dessa conversão.

A Tabela 14 apresenta, para cada estratégia de poda, as taxas de poda por camada do esquema de poda com análise de sensibilidade, à taxa de poda de 50% e com $\beta = 1{,}0$. Os valores correspondem às taxas efetivamente aplicadas, registradas nos resultados dos experimentos. Nas duas estratégias, o esquema de poda uniforme aplica 50% a cada uma das 12 camadas. No esquema de poda com análise de sensibilidade, por sua vez, as taxas variaram entre as camadas. Na poda estruturada, a camada 0 recebeu taxa de 2%, e a camada 1, de 84%. Na poda por magnitude, essas mesmas camadas receberam 8% e 94%.

Tabela 14 – Taxas de poda por camada aplicadas à taxa de poda de 50%, com $\beta = 1{,}0$.

Camada	0	1	2	3	4	5	6	7	8	9	10	11
Magnitude (%)	8	94	35	75	16	25	44	67	59	53	61	61
Estruturada (%)	2	84	46	43	36	47	57	67	64	85	57	12

Fonte: Elaboração própria.

6.3.5 Robustez do Perfil à Taxa-Sonda

Esta subseção apresenta a verificação prevista no procedimento (Seção 4.3): se a ordenação das camadas depende da taxa-sonda adotada. Para isso, os dois perfis da Tabela 13 foram comparados.

Os valores de Δ_ℓ diferem entre as duas taxas-sonda, pois a quantidade de elementos removidos de cada camada é diferente. Por isso, a comparação refere-se à ordenação das camadas, e não aos valores. A correlação de postos de Spearman entre os dois perfis foi de $\rho = 0{,}944$ na poda por magnitude e de $\rho = 0{,}713$ na poda estruturada. As ordenações, portanto, não foram idênticas, e a da poda estruturada foi a menos estável.

Na poda por magnitude, a ordenação praticamente se manteve. As três camadas mais sensíveis (0, 4 e 5) e as três mais tolerantes (1, 3 e 7) foram as mesmas nos dois perfis, e nenhuma camada mudou mais de três postos.

Na poda estruturada, as camadas 0 e 11 ocuparam os dois primeiros postos nos dois perfis, de modo que o formato em "U" (Seção 6.3.2) não dependeu da taxa-sonda. Além disso, a camada 0 respondeu por 65,5% da soma dos Δ_ℓ a 50% e por 67,9% a 30%. A divergência concentrou-se na camada 1, que passou do 11º posto, à taxa-sonda de 50%, para o 3º, à de 30%. Essa camada foi a única cuja degradação diminuiu com o aumento da taxa-sonda: de 0,159, a 30%, para 0,094, a 50%. As demais camadas mudaram, no máximo, dois postos. Essa mudança de posto é grande, mas corresponde a uma diferença pequena de degradação. Podada isoladamente, a camada 1 resultou em *perplexity* de 29,52 a 30% e de 27,66 a 50%, uma diferença inferior a dois pontos. As camadas 1 a 10 apresentam sensibilidades próximas entre si: nas duas taxas-sonda, a poda isolada de cada uma delas resultou em *perplexity* entre 26,53 e 31,31, enquanto a da camada 0 resultou em 772,11 e 1.386,43. Dessa forma, pequenas diferenças de degradação são suficientes para alterar os postos das camadas intermediárias.

A mudança da camada 1 tem consequência para o esquema de poda. Com o perfil de 50%, essa camada está entre as que recebem as maiores taxas de poda: 84% à taxa de poda de 50% (Tabela 14). Com o perfil de 30%, estaria entre as que recebem as menores. Portanto, na poda estruturada, a verificação confirma a posição das duas camadas mais sensíveis, mas não a ordenação completa. O esquema de poda derivado do perfil de 30% não foi avaliado, de modo que os resultados da Seção 6.4 referem-se ao perfil de 50%. Segundo a análise da Seção 6.4.4, o fator determinante nessa estratégia é a preservação da camada 0, cuja posição foi a mesma nos dois perfis.

6.3.6 Limitações do Perfil de Sensibilidade

Três limitações da medição do perfil condicionam a leitura dos resultados do esquema de poda com análise de sensibilidade.

A primeira é a faixa restrita de taxas-sonda, registrada entre as limitações do trabalho (Seção 5.9): a verificação da subseção anterior não abrange taxas-sonda muito menores ou muito maiores que as duas adotadas.

A segunda, e mais relevante, é que o perfil mede o efeito de cada camada isoladamente e, portanto, não considera as interações entre elas. Na poda por magnitude, a soma dos Δ_ℓ individuais foi de 1,238, enquanto a poda simultânea de todas as camadas a 50%, no esquema de poda uniforme, produziu degradação logarítmica de 2,934, mais que o dobro. A degradação foi, portanto, superaditiva: a degradação de camadas podadas em conjunto supera a soma de suas degradações individuais. Por essa razão, o perfil deve ser lido como uma ordenação das camadas por sensibilidade, e não como uma estimativa da degradação de um modelo com várias camadas podadas.

A terceira é que todas as medições do perfil foram realizadas em regime *one-shot*. Camadas sensíveis nesse regime poderiam ser recuperadas pelo retreinamento, o que alteraria o esquema de poda mais adequado. A adoção do regime *one-shot* é coerente com o objetivo de avaliar o efeito da poda separadamente daquele do retreinamento (critério P2, Seção 4.2.3), mas restringe a generalidade das conclusões.

Nenhuma dessas limitações altera o principal resultado desta seção: as camadas do GPT-2 *small* diferiram entre si em mais de uma ordem de grandeza quanto à degradação causada por sua poda. Os esquemas de poda uniforme e não uniforme não utilizam essa informação. A seção seguinte verifica se ela se converte em menor perda de desempenho preditivo para uma mesma taxa de poda.

6.4 Desempenho Preditivo

Esta seção responde à pergunta (B): se o esquema de poda com análise de sensibilidade melhora o compromisso entre compressão e qualidade em relação aos esquemas de poda sem essa análise. As Tabelas 15 e 16 apresentam a *perplexity* dos quatro esquemas de poda em cada taxa de poda. O esquema de poda com análise de sensibilidade foi avaliado com as duas intensidades, $\beta = 1{,}0$ e $\beta = 0{,}5$, sobre o mesmo perfil. As Seções 6.4.1 e 6.4.2 apresentam os resultados de cada estratégia. Em seguida, a Seção 6.4.3 apresenta a verificação com $\beta = 0$, e a Seção 6.4.4, o efeito do parâmetro de intensidade. Por fim, a Seção 6.4.5 analisa os casos em que a *perplexity* diminuiu com o aumento da taxa de poda.

6.4.1 Poda por Magnitude

A Tabela 15 apresenta a *perplexity* dos quatro esquemas de poda na poda por magnitude, sem retreinamento. O menor valor de cada taxa está em negrito. No esquema

de poda não uniforme, a *perplexity* diminuiu de 70% a 90%, comportamento analisado na Seção 6.4.5.

Tabela 15 – *Perplexity* da poda por magnitude, sem retreinamento, nos quatro esquemas de poda.

	Sem análise de sensibilidade		**Com análise de sensibilidade**	
Taxa de poda	**Não uniforme**	**Uniforme**	$\boldsymbol{\beta = 1{,}0}$	$\boldsymbol{\beta = 0{,}5}$
0%	25,17 (modelo base)			
10%	**25,40**	25,47	25,44	25,44
20%	26,87	27,22	26,79	**26,57**
30%	40,23	36,42	34,62	**31,20**
40%	165,20	104,82	193,83	**57,30**
50%	1.096,50	473,28	1.026,09	**447,57**
60%	5.121,25	2.004,20	**1.524,55**	2.171,79
70%	9.371,16	6.891,85	**1.853,30**	2.805,39
80%	8.847,27	10.871,00	**2.747,56**	3.715,68
90%	**4.612,69**	13.996,88	10.398,10	7.616,63

Fonte: Elaboração própria.

Nos esquemas de poda sem análise de sensibilidade, o crescimento da *perplexity* acentuou-se em torno da taxa de 30%. Até 20%, o acréscimo foi de 0,2 a 2,1 pontos. A 40%, a *perplexity* atingiu 165,20 no esquema de poda não uniforme e 104,82 no uniforme. A 50%, atingiu 1.096,50 e 473,28, respectivamente. Pelo limite de 5% (Seção 4.1.3), que corresponde a uma *perplexity* de até 26,43, o desempenho preditivo foi preservado apenas a 10%. A 20%, os dois esquemas de poda já excederam esse limite, com 26,87 e 27,22.

A comparação entre os dois esquemas de poda sem análise de sensibilidade dependeu da taxa de poda. O esquema de poda não uniforme apresentou valores ligeiramente menores até 20%, e o uniforme, de 30% a 70%. Em 80% e 90%, o não uniforme voltou a apresentar valores menores. Nessas duas taxas, entretanto, ambos os modelos apresentaram *perplexity* acima de 4.600, de modo que a comparação não tem consequência prática. Portanto, de 30% a 50%, o esquema de poda uniforme foi superior ao não uniforme. Esse resultado é contraintuitivo, pois o esquema de poda não uniforme distribui a poda com base na magnitude dos pesos, enquanto o uniforme não utiliza nenhuma medida de importância. Trata-se do primeiro indício de que a magnitude do peso não é, isoladamente, uma medida suficiente da importância de um parâmetro.

O esquema de poda com análise de sensibilidade obteve *perplexity* menor que a dos dois esquemas de poda sem essa análise em sete das nove taxas, de 20% a 80%. A 30%, a *perplexity* diminuiu de 36,42, valor do melhor esquema de poda sem análise de sensibilidade, para 31,20, uma redução de 14%. A 40%, diminuiu de 104,82 para 57,30, quase a metade, para a mesma taxa de poda. A 50%, a vantagem se manteve, embora pequena: 447,57 contra 473,28. De 60% a 80%, o esquema de poda com $\beta = 1{,}0$ produziu

perplexities de 1,3 a 3,7 vezes menores que as do melhor esquema de poda sem análise de sensibilidade. Nessas taxas, entretanto, todos os modelos apresentaram *perplexity* acima de 1.500, de modo que a comparação não tem consequência prática. Na média geométrica de 10% a 50% (Seção 4.1.3), a melhor varredura com análise de sensibilidade ($\beta = 0{,}5$, 55,80) ficou cerca de 15% abaixo da melhor varredura sem essa análise (uniforme, 66,00).

As duas exceções ocorreram nas taxas extremas. A 10%, o esquema de poda não uniforme (25,40) ficou 0,04 abaixo das duas configurações com análise de sensibilidade (25,44), diferença sem consequência prática. A 90%, o esquema de poda não uniforme apresentou o menor valor. Entretanto, como a *perplexity* de todas as configurações ultrapassou 4.600, a ordenação entre elas também não tem consequência prática.

6.4.2 Poda Estruturada

A Tabela 16 apresenta a *perplexity* dos quatro esquemas de poda na poda estruturada, sem retreinamento. O menor valor de cada taxa está em negrito. Nos quatro esquemas de poda, a *perplexity* diminuiu entre algumas taxas de poda consecutivas, comportamento analisado na Seção 6.4.5.

Tabela 16 – *Perplexity* da poda estruturada, sem retreinamento, nos quatro esquemas de poda.

	Sem análise de sensibilidade		**Com análise de sensibilidade**	
Taxa de poda	**Não uniforme**	**Uniforme**	**$\beta = 1{,}0$**	**$\beta = 0{,}5$**
0%	25,17 (modelo base)			
10%	356,20	481,13	**124,29**	215,83
20%	2.102,06	1.484,89	**371,07**	717,19
30%	2.413,52	5.467,56	**724,53**	923,65
40%	2.434,92	10.486,71	**1.301,15**	9.489,87
50%	2.402,58	4.573,00	**2.327,59**	7.922,81
60%	**2.957,26**	5.799,76	5.393,15	12.322,05
70%	**4.635,36**	12.249,58	10.130,33	103.463,60
80%	**7.620,02**	53.313,38	8.891,77	203.995,84
90%	5.822,89	827.316,19	5.353,85	**4.830,24**

Fonte: Elaboração própria.

Na poda estruturada, a redução de custo foi acompanhada de elevada perda de desempenho preditivo em regime *one-shot*. Com 10% das estruturas removidas, a *perplexity* do esquema de poda não uniforme passou de 25,17 para 356,20, enquanto a da poda por magnitude, à mesma taxa, foi de 25,40. Nenhuma taxa de poda ficou dentro do limite de 5% na poda estruturada sem retreinamento. Esse resultado é coerente com o método original da poda por norma L_1, em que a poda é seguida de retreinamento (Seção 3.2).

Entre os esquemas de poda sem análise de sensibilidade, o resultado foi inverso ao da poda por magnitude: o esquema de poda não uniforme apresentou os menores valores em todas as taxas, com exceção da de 20%. O esquema de poda uniforme, por sua vez, degradou-se fortemente em taxas altas, com *perplexity* de 53.313 a 80% e de 827.316 a 90%, contra 7.620 e 5.823 do não uniforme. Considerados em conjunto, os resultados das duas estratégias mostram que nenhum dos dois esquemas de poda sem análise de sensibilidade foi superior em ambas, o que motiva a análise de sensibilidade. Uma explicação possível, não testada neste trabalho, decorre da Seção 6.3.3: como a magnitude do peso e a norma da estrutura refletem aspectos diferentes da importância de uma camada, o esquema de poda não uniforme, que se baseia nessas grandezas, pode obter bons resultados em uma estratégia e não na outra.

A comparação entre os dois tipos de estrutura, realizada com os alvos isolados, indicou que a remoção de cabeças de atenção degradou menos o modelo que a remoção de neurônios do MLP. A única exceção foi a taxa de 10% no esquema de poda não uniforme, com *perplexity* de 237 contra 139. Com 129 das 144 cabeças removidas (90%), a *perplexity* foi de 781, enquanto a remoção de neurônios do MLP à mesma taxa produziu valores acima de 3.900. Além disso, no esquema de poda não uniforme, a curva das cabeças não foi monotônica: apresentou mínimo local em torno de 30%, com *perplexity* de 162, inferior aos 237 obtidos com 10% das cabeças removidas. Esse resultado é compatível com a redundância das cabeças de atenção observada por Michel, Levy e Neubig (2019). Por outro lado, os neurônios do MLP concentram cerca de dois terços dos parâmetros podáveis (Seção 5.2.3), de modo que a poda restrita às cabeças tem alcance limitado como estratégia de compressão.

O esquema de poda com análise de sensibilidade, com $\beta = 1{,}0$, obteve a menor *perplexity* em todas as taxas de 10% a 50%. A vantagem foi máxima entre 20% e 30%. A 20%, a *perplexity* diminuiu de 1.484,89, valor do melhor esquema de poda sem análise de sensibilidade, para 371,07, um fator de quatro. A 30%, diminuiu de 2.413,52 para 724,53, um fator de 3,3. A 10%, o fator foi de 2,9 (356,20 contra 124,29). A partir de 40%, a vantagem diminuiu: o fator foi de 1,9 a 40% (2.434,92 contra 1.301,15), e a diferença foi marginal a 50% (2.402,58 contra 2.327,59). Na média geométrica de 10% a 50%, a *perplexity* diminuiu de 1.602,62, no esquema de poda não uniforme, para 632,46, um fator de 2,5. A redução da *perplexity* a um quarto, para a mesma taxa de poda, foi a maior vantagem observada para o esquema de poda com análise de sensibilidade.

Esse resultado exige uma ressalva. Como a remoção de estruturas é discreta, as taxas de poda por camada não são atingidas exatamente (Seção 5.2.3), e o esquema de poda com análise de sensibilidade removeu uma fração de parâmetros ligeiramente menor que a do não uniforme na mesma taxa de poda. A 10%, a esparsidade real foi de 9,20% contra 9,90%. A 30%, de 29,01% contra 29,95%. Portanto, parte da vantagem observada

pode decorrer dessa remoção menor. Entretanto, a diferença de esparsidade foi inferior a um ponto percentual, enquanto a diferença de *perplexity* chegou a um fator de quatro, de modo que essa remoção menor não constitui a causa principal da vantagem.

De 60% a 80%, o esquema de poda não uniforme voltou a obter a menor *perplexity*. A 90%, a *perplexity* de todas as configurações superou 4.800, e a ordenação entre elas não tem consequência prática. A causa provável dessa inversão está nas taxas de poda por camada. A partir de 60%, um número crescente de camadas atingiu o teto $r_{\max} = 0{,}95$: duas camadas a 60%, quatro a 70%, oito a 80% e 11 a 90%. Nessas camadas, o esquema de poda deixa de diferenciar as taxas e, ao mesmo tempo, concentra mais poda nas camadas que permanecem abaixo do teto. Portanto, o esquema de poda com análise de sensibilidade só diferencia as camadas enquanto a taxa de poda puder ser atingida sem que elas alcancem o teto, o que justifica o limite de 50% adotado nas médias (Seção 6.1). Essa limitação decorre da própria regra, e não de um erro de implementação.

6.4.3 Verificação com $\beta = 0$

Esta subseção apresenta a verificação da implementação prevista na Seção 5.3.2: com $\beta = 0$, a regra da Equação 5.3 atribui a mesma taxa a todas as camadas e deve reproduzir a varredura uniforme na poda estruturada, mas não na poda por magnitude.

Na poda estruturada, a reprodução foi exata. Nas 10 taxas de poda, o esquema de poda com $\beta = 0$ reproduziu a *perplexity* da varredura uniforme em todos os dígitos registrados: 5.467,5635 a 30%, 4.573,0020 a 50% e 827.316,1875 a 90%, por exemplo. Esse resultado indica que a conversão do perfil em taxas, a determinação do fator e a aplicação das taxas por camada estão implementadas corretamente. Assim, a vantagem obtida pelo esquema de poda com análise de sensibilidade nas seções anteriores decorre do perfil, e não de diferenças de procedimento.

Na poda por magnitude, como previsto, a reprodução não ocorreu, pois a regra utiliza um limiar por camada, e a varredura uniforme, um limiar por matriz (Seção 5.3.2). Os dois procedimentos produziram a mesma esparsidade em cada camada, mas distribuída de forma diferente entre as quatro matrizes, e as *perplexities* divergiram: 473,28 na varredura uniforme contra 590,25 com $\beta = 0$, a 50%, e 13.996,88 contra 8.610,96, a 90%. Dessa forma, na poda por magnitude, o esquema de poda com $\beta = 0$ não corresponde ao esquema de poda uniforme da Seção 6.4.1, e sim a um terceiro esquema de poda, cuja comparação com os demais está fora do escopo deste trabalho. Na poda estruturada, única estratégia que reduziu o consumo de energia, a verificação vale integralmente.

6.4.4 Efeito do Parâmetro de Intensidade β

Esta subseção compara as duas intensidades avaliadas, $\beta = 1{,}0$ e $\beta = 0{,}5$ (Seção 5.3.2). O principal resultado é que a intensidade mais adequada dependeu da estratégia de poda, e de forma oposta nas duas.

Na poda estruturada, cujo perfil apresenta razão de 42,9 entre a degradação da camada mais sensível e a da mais tolerante (Tabela 13), $\beta = 1$ obteve *perplexity* menor que $\beta = 0{,}5$ em todas as taxas de poda até 80%. Nessa estratégia, a atenuação do perfil foi prejudicial: a 70% e a 80%, o esquema de poda com $\beta = 0{,}5$ produziu *perplexities* de 103.463 e 203.996, respectivamente 8,4 e 3,8 vezes as do esquema de poda uniforme, o de pior resultado entre os demais nessas taxas. Na poda por magnitude, cujo perfil apresenta razão de 11,3, ocorreu o inverso: $\beta = 0{,}5$ foi melhor de 20% a 50%. De 60% a 80%, $\beta = 1$ obteve os menores valores, e, a 90%, $\beta = 0{,}5$ voltou a ser melhor (7.616,63 contra 10.398,10).

Na poda por magnitude, a intensidade determinou o resultado da comparação com os esquemas de poda sem análise de sensibilidade. A 40%, por exemplo, a *perplexity* foi de 57,30 com $\beta = 0{,}5$, de 104,82 com o esquema de poda uniforme, o melhor sem análise de sensibilidade nessa taxa, e de 193,83 com $\beta = 1$. Se apenas $\beta = 1$ tivesse sido avaliado, o esquema de poda com análise de sensibilidade teria sido superior aos esquemas de poda sem essa análise em cinco das nove taxas de poda, e não em sete (Seção 6.4.1). A 40% e a 50%, teria resultado em *perplexity* maior, e não menor.

Na poda estruturada, a adoção exclusiva de $\beta = 0{,}5$ teria produzido o efeito inverso. Com $\beta = 1$, o esquema de poda com análise de sensibilidade obteve a menor *perplexity* em todas as taxas de poda de 10% a 50% (Seção 6.4.2). Com $\beta = 0{,}5$, teria sido superado pelo esquema de poda não uniforme a 40% e a 50%, com *perplexity* de 9.489,87 e 7.922,81, contra 2.434,92 e 2.402,58. Essas são as mesmas duas taxas de poda em que $\beta = 1$ foi superado na poda por magnitude. Portanto, entre os valores avaliados, nenhuma intensidade única teria sido adequada às duas estratégias.

A *perplexity* variou com β, embora a quantidade de elementos removidos praticamente não tenha variado. Na poda por magnitude a 40%, os modelos obtidos com $\beta = 1$ e com $\beta = 0{,}5$ conservaram os mesmos 90,5 milhões de parâmetros não nulos. Na poda estruturada, à mesma taxa, $\beta = 1$ removeu 54 cabeças e 14.739 neurônios, e $\beta = 0{,}5$, 51 cabeças e 14.740 neurônios. Como o parâmetro de intensidade altera apenas a distribuição da poda entre as camadas (Seção 5.3.2), a variação da *perplexity* decorre dessa distribuição: a mesma quantidade de elementos degrada mais o modelo quando removida de uma camada sensível do que de uma camada tolerante. Além disso, a própria métrica amplia a diferença. Como a *perplexity* é a exponencial da perda média por *token* (Hugging Face, 2026b), diferenças moderadas na perda tornam-se multiplicativas. A 40%, na poda por

magnitude, a *perplexity* com $\beta = 1$, de 193,83, foi 3,4 vezes a obtida com $\beta = 0{,}5$, de 57,30, o que corresponde a uma diferença de 1,22 na escala logarítmica da Equação 5.1. Na poda estruturada, a razão de 7,3 entre $\beta = 0{,}5$ e $\beta = 1$ corresponde a uma diferença de 1,99.

As taxas de poda por camada indicam por que cada intensidade apresentou pior resultado em uma das estratégias. Na poda por magnitude a 40%, $\beta = 1$ atribuiu taxa de 75,5% à camada 1, a mais tolerante. Essa taxa é 1,5 vez maior que a taxa-sonda de 50% com que a tolerância da camada foi medida (Seção 5.3.1), e, acima da taxa-sonda, o perfil não fornece informação sobre o comportamento da camada. Com $\beta = 0{,}5$, a mesma camada recebeu 57,1%, valor mais próximo da taxa-sonda. Na poda estruturada, o desvio ocorreu no sentido oposto: $\beta = 0{,}5$ elevou a taxa da camada 0, a mais sensível (Seção 6.3.2), de 1,6% para 8,4% a 40% e de 3,0% para 14,6% a 70%. Portanto, uma intensidade alta demais leva as camadas tolerantes a taxas de poda que o perfil não mediu, e uma intensidade baixa demais aumenta a poda da camada que o perfil indicou como a mais sensível.

O efeito de cada desvio depende de como a degradação de uma camada cresce com a sua taxa de poda. Os perfis medidos a 30% e a 50% (Seção 6.3.5) indicam que esse crescimento difere entre as estratégias. Na poda por magnitude, a passagem da taxa-sonda de 30% para 50%, um aumento de 1,7 vez, multiplicou a degradação de cada camada por valores entre 3,8 e 9,4, ou seja, a degradação cresceu muito mais depressa que a taxa. Assim, a concentração da poda nas camadas tolerantes, produzida por $\beta = 1$, leva essas camadas à faixa em que a degradação mais cresce, enquanto a atenuação das diferenças, produzida por $\beta = 0{,}5$, distribui a poda e reduz a degradação total. Essa atenuação, entretanto, tem limite: a 40%, o esquema de poda uniforme, em que todas as camadas recebem a mesma taxa, resultou em *perplexity* de 104,82, valor situado entre os 57,30 de $\beta = 0{,}5$ e os 193,83 de $\beta = 1$. Portanto, na poda por magnitude, o melhor resultado ocorreu na intensidade intermediária.

Na poda estruturada, o mesmo aumento da taxa-sonda multiplicou a degradação das camadas por valores entre 0,6 e 2,2. Além disso, a camada 0 já degradava fortemente o modelo a 30%: podada isoladamente, elevou a *perplexity* de 25,17 para 772,11. Nessa estratégia, a degradação depende mais de qual camada é podada do que da taxa aplicada, e a preservação da camada 0, assegurada por um β alto, é o fator determinante. A 40%, a *perplexity* diminuiu a cada aumento da intensidade: 10.486,71 com $\beta = 0$, 9.489,87 com $\beta = 0{,}5$ e 1.301,15 com $\beta = 1$. As duas explicações baseiam-se em camadas podadas isoladamente e em apenas duas taxas-sonda, e a degradação de camadas podadas em conjunto é superaditiva (Seção 6.3.6). Portanto, elas são compatíveis com os resultados, mas não constituem uma demonstração.

Esses resultados indicam que o parâmetro de intensidade é parte relevante da

regra que define o esquema de poda. O parâmetro expressa o grau em que o perfil é seguido, e o grau adequado dependeu da estratégia: foi maior na poda estruturada, cujo perfil apresenta razão de 42,9 e é dominado pela camada 0, e menor na poda por magnitude, cujo perfil apresenta razão de 11,3 e várias camadas de sensibilidade intermediária. Esse grau só pôde ser determinado experimentalmente. Como apenas dois valores foram avaliados, a intensidade ótima de cada estratégia não foi determinada, e sua busca constitui uma extensão deste trabalho.

6.4.5 Redução da *Perplexity* com o Aumento da Taxa de Poda

Esta subseção analisa um comportamento das Tabelas 15 e 16 contrário ao esperado. Em nove ocasiões, a *perplexity* de um esquema de poda diminuiu entre duas taxas de poda consecutivas. Na poda por magnitude, as reduções ocorreram exclusivamente no esquema de poda não uniforme, cuja *perplexity* passou de 9.371,16, a 70%, para 8.847,27, a 80%, e para 4.612,69, a 90%. Na poda estruturada, ocorreram nos quatro esquemas de poda. A maior delas foi a do esquema de poda com $\beta = 0{,}5$, cuja *perplexity* passou de 203.995,84, a 80%, para 4.830,24, a 90%, valor inferior aos 9.489,87 obtidos a 40%.

Essas reduções não decorrem de erro na aplicação da poda nem de variação entre execuções. Em todas as varreduras, a esparsidade real cresceu com a taxa de poda. Além disso, cada taxa parte do modelo base (Seção 4.2.2) e seleciona os elementos pela mesma ordenação, de modo que os elementos removidos a uma taxa de poda incluem os removidos nas taxas menores do mesmo esquema de poda. Por sua vez, a avaliação é determinística. A verificação com $\beta = 0$ (Seção 6.4.3) reproduziu, em execução separada, todos os valores da varredura uniforme da poda estruturada, incluída a redução de 10.486,71, a 40%, para 4.573,00, a 50%.

As reduções ocorreram exclusivamente em modelos cujo desempenho preditivo já estava comprometido. Todas partiram de *perplexity* igual ou superior a 2.434,92, valor do esquema de poda não uniforme a 40% na poda estruturada, que corresponde a cerca de 97 vezes a do modelo base. Abaixo desse valor, a *perplexity* cresceu com a taxa de poda em todos os esquemas de poda das duas estratégias.

Nessa faixa de valores, a *perplexity* deixa de indicar quanto do modelo foi preservado. A métrica corresponde ao inverso da probabilidade atribuída ao conjunto de teste, normalizado pelo número de *tokens*, ou seja, ao inverso da média geométrica das probabilidades atribuídas aos *tokens* corretos (JURAFSKY; MARTIN, 2025). Assim, um modelo que atribuísse a mesma probabilidade a cada um dos 50.257 *tokens* do vocabulário (Seção 5.1.2.1), sem utilizar o contexto, teria *perplexity* de 50.257. Quatro configurações da Tabela 16 excederam essa referência: o esquema de poda uniforme a 80% e a 90% e o esquema de poda com $\beta = 0{,}5$ a 70% e a 80%. Um valor acima dela só é possível quando o modelo atribui aos *tokens* corretos, em média, probabilidade inferior à da distribuição

uniforme, ou seja, quando concentra a probabilidade em *tokens* incorretos. Portanto, em modelos nessa condição, a *perplexity* depende do grau de concentração das predições em *tokens* incorretos, e não apenas da quantidade de elementos preservados.

O esquema de poda com $\beta = 0{,}5$, na poda estruturada, ilustra esse comportamento. A 80%, seis camadas receberam taxas de poda iguais ou superiores a 94,9%, as camadas 2 a 5, taxas de 75,6% a 86,0%, a camada 11, de 43,4%, e a camada 0, de 17,6%. A 90%, o teto $r_{\max} = 0{,}95$ determinou quase todo o esquema de poda. Dez camadas receberam 95%, a camada 11, 92,4%, e a camada 0, 37,6%. Em uma camada no teto, restam uma das 12 cabeças de atenção e 154 dos 3.072 neurônios do MLP. Portanto, o modelo a 90% não conservou, em nenhuma camada, mais estruturas que o modelo a 80%, e sua *perplexity* foi 42 vezes menor. Em relação à referência de 50.257, a média geométrica das probabilidades atribuídas aos *tokens* corretos passou de cerca de um quarto da probabilidade uniforme, a 80%, para cerca de dez vezes essa probabilidade, a 90%. Com $\beta = 1{,}0$, o esquema de poda a 90% foi praticamente o mesmo, com 11 camadas no teto e 35,0% na camada 0, e a *perplexity* foi de 5.353,85.

A comparação com o esquema de poda uniforme indica o papel da camada 0. A 90%, os dois esquemas de poda com análise de sensibilidade conservaram oito das 12 cabeças dessa camada e uma em cada uma das demais, enquanto o esquema de poda uniforme conservou duas cabeças em todas as camadas. A *perplexity* deste último foi de 827.316,19. Esse resultado é coerente com a Seção 6.4.4, segundo a qual a preservação da camada 0 é o fator determinante na poda estruturada.

A diferença entre 80% e 90%, por sua vez, não se explica pela camada 0, que foi mais podada a 90%. Uma hipótese compatível com esses valores, não testada neste trabalho, refere-se às camadas parcialmente removidas. No *Transformer*, a saída de cada camada é somada à representação que ela recebe (VASWANI et al., 2017). Dessa forma, uma camada quase inteiramente removida acrescenta pouco a essa soma, enquanto uma camada parcialmente removida continua a alterar a representação, a partir de entradas diferentes daquelas para as quais foi treinada. A 80%, as camadas 2 a 5 e a camada 11 estavam nessa condição. A 90%, apenas a camada 0 estava. Essa hipótese é coerente com a tolerância à remoção de camadas intermediárias inteiras observada por Lad et al. (2025). Sua verificação exigiria medir as distribuições de probabilidade produzidas por cada modelo, o que não foi realizado.

Em consequência, uma *perplexity* menor em uma taxa de poda maior não indica recuperação de desempenho preditivo. De 60% a 90%, todas as configurações apresentaram *perplexity* superior a 1.500, e as diferenças entre elas não têm consequência prática, como indicado nas subseções anteriores. Essa leitura é coerente com o limite de 50% adotado no resumo por varredura (Seção 6.1).

6.5 *Fine-tuning* de Recuperação

Esta seção apresenta os resultados da Etapa 4, em que as sete configurações definidas na Seção 5.4.1 foram submetidas ao *fine-tuning* de recuperação, com os parâmetros da Tabela 8. A *perplexity* de cada configuração foi medida antes e depois do retreinamento (Tabela 17). Diferentemente das varreduras, esse experimento foi executado integralmente em uma única sessão, de modo que suas medições de tempo e energia são diretamente comparáveis entre si.

Na Tabela 17, as configurações sem análise de sensibilidade utilizam o esquema de poda uniforme, na poda por magnitude, e o não uniforme, na poda estruturada. As configurações com análise de sensibilidade utilizam $\beta = 1{,}0$. A coluna Pós-FT apresenta a *perplexity* após o *fine-tuning*, e a coluna Razão expressa esse valor como múltiplo da *perplexity* do modelo base retreinado, e não do modelo base original.

Tabela 17 – *Perplexity* antes (*one-shot*) e depois do *fine-tuning* de recuperação.

	Sem análise de sensibilidade			**Com análise de sensibilidade**		
Configuração	***One-shot***	**Pós-FT**	**Razão**	***One-shot***	**Pós-FT**	**Razão**
Modelo base	25,17 *one-shot*; 18,68 pós-FT (referência da razão)					
Magnitude 30%	36,42	20,54	1,10	—	—	—
Magnitude 50%	473,28	30,27	1,62	—	—	—
Estruturada 10%	356,20	25,33	1,36	124,29	25,96	1,39
Estruturada 30%	2.413,52	104,78	5,61	724,53	56,08	3,00

Fonte: Elaboração própria.

6.5.1 Efeito do Retreinamento sobre o Modelo Base

O primeiro resultado refere-se ao modelo base, sem poda. Submetido ao mesmo retreinamento, o modelo base teve sua *perplexity* reduzida de 25,17 para 18,68, uma redução de 26% obtida sem a remoção de nenhum parâmetro. Esse resultado é compatível com a adaptação de domínio descrita na Seção 5.4.3.

Esse resultado confirma a necessidade do modelo base retreinado como referência. Sem ele, a poda por magnitude a 30% pareceria ter superado o modelo base original, com *perplexity* de 20,54 contra 25,17, o que sugeriria que a poda melhora o modelo. Em relação ao modelo base retreinado (18,68), entretanto, a mesma configuração ficou 10% acima. Por essa razão, as colunas de razão da Tabela 17 adotam essa referência.

6.5.2 Recuperação por Configuração

O retreinamento recuperou parte da degradação em todas as configurações, e a recuperação tendeu a ser maior quanto maior era a *perplexity* antes do retreinamento,

embora essa relação não tenha sido estrita. A *perplexity* ficou 1,8 vez menor na poda por magnitude a 30% e 23,0 vezes menor na poda estruturada a 30%. Nessa última configuração, com esquema de poda não uniforme, a *perplexity* passou de 2.413,52 para 104,78. Na poda por magnitude a 50%, passou de 473,28 para 30,27, isto é, 1,62 vez o valor do modelo base retreinado.

Esse resultado condiciona a leitura das seções anteriores. A *perplexity* em regime *one-shot* mede a degradação imediata causada pela poda, mas superestima a degradação que permanece após o retreinamento, pois parte dela foi revertida por uma época de retreinamento. A recuperação, entretanto, não foi completa: nenhuma configuração ficou dentro de 5% do modelo base retreinado, e a mais próxima, a poda por magnitude a 30%, ficou 10% acima dele (Seção 6.6).

A relação entre as duas granularidades também se manteve após o retreinamento. A poda por magnitude a 30% atingiu a menor *perplexity* entre as configurações podadas, 20,54. Entretanto, como estabelecido na Seção 6.2, essa estratégia não apresentou redução de memória nem redução detectável de tempo e energia. A poda estruturada a 30%, com esquema de poda não uniforme, registrou 40,82 ms por janela e 1,285 Wh por avaliação completa, contra 50,21 ms e 1,588 Wh do modelo base retreinado, uma redução de aproximadamente 19% nas duas grandezas.

6.5.3 Efeito do *Fine-tuning* sobre a Vantagem da Análise de Sensibilidade

Esta subseção compara, nas duas taxas de poda da poda estruturada submetidas ao retreinamento, o esquema de poda não uniforme e o esquema de poda com análise de sensibilidade, antes e depois do retreinamento (Tabela 17). O resultado refere-se diretamente à contribuição central do trabalho.

A 10%, a vantagem do esquema de poda com análise de sensibilidade desapareceu. Em regime *one-shot*, esse esquema registrou *perplexity* de 124,29, contra 356,20 do esquema de poda não uniforme, uma vantagem de 2,87 vezes. Após o retreinamento, a relação se inverteu: 25,96 contra 25,33, com ligeira vantagem do esquema de poda não uniforme. A 30%, a vantagem se manteve, mas foi reduzida quase à metade: o fator de 3,33 do regime *one-shot* (724,53 contra 2.413,52) passou a 1,87 (56,08 contra 104,78).

Uma interpretação compatível com esses resultados é que o esquema de poda com análise de sensibilidade reduz a degradação imediata causada pela poda, e o retreinamento recupera justamente esse tipo de degradação. Segundo essa interpretação, quanto maior a quantidade de retreinamento, menor a vantagem de uma melhor distribuição da poda entre as camadas, e o perfil de sensibilidade é mais útil quanto menor for a quantidade de retreinamento disponível. Após uma época de retreinamento, a vantagem foi maior a 30% que a 10%. Entretanto, duas taxas de poda, avaliadas com uma única semente, não

permitem estabelecer uma tendência.

Esse resultado não invalida a contribuição, mas delimita o seu alcance. A 30%, o esquema de poda com análise de sensibilidade manteve *perplexity* aproximadamente duas vezes menor, com o mesmo custo computacional: 40,28 ms e 1,307 Wh, contra 40,82 ms e 1,285 Wh do esquema de poda não uniforme, valores equivalentes dentro da variação de medição. A 10%, com retreinamento, o esquema de poda sem análise de sensibilidade foi suficiente.

6.5.4 Verificações de Integridade e Custo

Esta subseção apresenta duas verificações do procedimento de retreinamento e o seu custo energético. A primeira verificação refere-se à preservação da esparsidade (Seção 5.4.3). A esparsidade das configurações de poda por magnitude, medida ao final do treinamento, permaneceu em 30,0000% e 50,0000%. Esse resultado confirma que a máscara binária foi reaplicada corretamente após cada passo do otimizador. A segunda refere-se à esparsidade real das configurações de poda estruturada. Nas configurações com análise de sensibilidade, ela ficou ligeiramente abaixo da obtida com o esquema de poda não uniforme: 9,20% e 29,01%, contra 9,90% e 29,95%. Essa diferença, inferior a um ponto percentual, decorre da remoção discreta das estruturas e favorece marginalmente o esquema de poda com análise de sensibilidade (Seção 6.4.2).

Quanto ao custo, o retreinamento do modelo base consumiu 7,33 Wh, contra 1,59 Wh de uma avaliação completa do mesmo modelo, ou seja, aproximadamente 4,6 vezes. Esse custo, entretanto, ocorre uma única vez, enquanto a economia por inferência da poda estruturada se acumula a cada uso. A poda estruturada a 30%, com esquema de poda não uniforme, economizou 0,303 Wh por avaliação completa do conjunto de teste (1,285 contra 1,588 Wh). Assim, o custo de seu próprio retreinamento, de 6,34 Wh, é compensado em aproximadamente 21 avaliações completas. A economia é expressa por avaliação completa, e não por janela, porque o consumo foi medido durante toda a avaliação de uma configuração (Seção 5.1.3.3), e o número de janelas de cada avaliação não foi registrado.

A principal limitação desta etapa é a quantidade de retreinamento, de uma única época. Com esses resultados, não é possível afirmar se a redução da vantagem do esquema de poda com análise de sensibilidade se estabiliza no valor observado a 30% ou se prossegue, com mais épocas, até desaparecer também nessa taxa. Essa é a verificação mais imediata que este trabalho deixa em aberto.

6.6 Compromisso entre Desempenho Preditivo e Energia

As seções anteriores trataram o custo e o desempenho preditivo separadamente. Esta seção combina as duas dimensões por meio dos indicadores definidos na Seção 4.1.3, apresentados na Tabela 18, em que PPL corresponde à *perplexity*. Sem retreinamento, cada linha corresponde ao resumo de uma varredura nas taxas de 10% a 50%, e o limite de 5% do ponto de eficiência corresponde a uma *perplexity* de 26,43. Após o *fine-tuning*, cada linha corresponde a uma configuração, a referência é o modelo base retreinado, e o limite de 5% corresponde a 19,62.

Tabela 18 – Indicadores de compromisso entre desempenho preditivo e energia.

Varredura ou configuração	**PPL**	**ΔE (%)**	**η**	**S_E^***
Sem retreinamento (médias de 10% a 50%)				
Modelo base	25,17	0,0	1,000	—
Poda por magnitude				
Sem análise de sensibilidade — não uniforme	86,97	3,0	0,298	10%
Sem análise de sensibilidade — uniforme	66,00	3,0	0,393	10%
Com análise de sensibilidade — $\beta = 1{,}0$	85,96	4,4	0,306	10%
Com análise de sensibilidade — $\beta = 0{,}5$	55,80	4,0	0,470	10%
Poda estruturada				
Sem análise de sensibilidade — não uniforme	1.602,62	14,3	0,018	—
Sem análise de sensibilidade — uniforme	2.847,86	18,7	0,011	—
Com análise de sensibilidade — $\beta = 1{,}0$	632,46	18,0	0,049	—
Com análise de sensibilidade — $\beta = 0{,}5$	1.607,97	17,5	0,019	—
Após o *fine-tuning* de recuperação				
Modelo base retreinado	18,68	0,0	1,000	—
Poda por magnitude				
Sem análise de sensibilidade — uniforme, 30%	20,54	0,8	0,917	—
Sem análise de sensibilidade — uniforme, 50%	30,27	2,2	0,631	—
Poda estruturada				
Sem análise de sensibilidade — não uniforme, 10%	25,33	4,6	0,773	—
Com análise de sensibilidade — $\beta = 1{,}0$, 10%	25,96	3,2	0,744	—
Sem análise de sensibilidade — não uniforme, 30%	104,78	19,1	0,220	—
Com análise de sensibilidade — $\beta = 1{,}0$, 30%	56,08	17,7	0,405	—

Fonte: Elaboração própria.

O primeiro resultado é que nenhuma varredura e nenhuma configuração retreinada superou o modelo base em eficiência relativa. O indicador η ficou abaixo de 1 em todas as linhas da tabela, com e sem retreinamento, porque a *perplexity* cresceu mais depressa do que a energia diminuiu. Na poda estruturada sem retreinamento, a varredura de melhor resultado, com análise de sensibilidade e $\beta = 1{,}0$, economizou em média 18,0% de energia, com *perplexity* 25 vezes maior que a do modelo base, e seu η foi de 0,049. Após o *fine-tuning*, o maior η da poda estruturada foi o do esquema de poda não

uniforme a 10% (0,773). Essa configuração economizou 4,6% de energia, valor abaixo do limiar de detecção, com *perplexity* 36% acima da do modelo base retreinado. Na poda por magnitude, os valores de η foram mais altos, mas não em razão de maior eficiência: como ΔE ficou abaixo do limiar de detecção (Seção 6.2), η refletiu apenas a *perplexity* e variações abaixo desse limiar.

Em cada estratégia, o esquema de poda com análise de sensibilidade apresentou o maior η: 0,470 contra 0,393 na poda por magnitude ($\beta = 0{,}5$ contra o esquema de poda uniforme) e 0,049 contra 0,018 na poda estruturada ($\beta = 1{,}0$ contra o não uniforme). Como ΔE foi equivalente entre os esquemas de poda (Seção 6.2), essa diferença decorreu da *perplexity*. Portanto, a redução da perda de desempenho preditivo obtida com a análise de sensibilidade não teve custo adicional de energia. Após o *fine-tuning*, a vantagem em η se manteve a 30% (0,405 contra 0,220) e desapareceu a 10% (0,744 contra 0,773), acompanhando a *perplexity* (Seção 6.5.3).

O ponto de eficiência foi o indicador mais restritivo. Sem retreinamento, apenas a poda por magnitude apresentou S_E^*, de 10% em todos os esquemas de poda. A 20%, mesmo a melhor configuração (26,57, com $\beta = 0{,}5$) excedeu o limite de 26,43. Na poda estruturada, nenhuma taxa ficou dentro do limite: a menor *perplexity*, 124,29, foi quase cinco vezes a do modelo base (Seção 6.4.2). Após o *fine-tuning*, nenhuma configuração ficou dentro de 5% do modelo base retreinado, e a mais próxima, a poda por magnitude a 30%, ficou 10% acima (20,54 contra 18,68). Portanto, com o limite de 5%, mantido sem ajuste em função dos valores obtidos, o trabalho não identificou uma taxa de poda estruturada que preserve o desempenho preditivo. O que o trabalho identificou foi quanto desse desempenho a análise de sensibilidade preserva em cada taxa.

6.7 Síntese dos Resultados

Esta seção sintetiza os resultados do capítulo em relação às perguntas (A) e (B), formuladas no Capítulo 4.

Quanto à pergunta (A), a conversão da redução de parâmetros em redução de consumo dependeu da granularidade da poda. A poda por magnitude reduziu em 61% os parâmetros não nulos sem alterar o tamanho em memória nem os FLOPs, e sem produzir, em tempo ou energia, efeito acima do limiar de detecção do protocolo. A poda estruturada, ao remover cabeças de atenção e neurônios do MLP, converteu a mesma redução de 61% dos parâmetros em reduções de 56% no tempo de inferência, 59% na energia e 62% nos FLOPs. Portanto, na GPU Tesla T4 utilizada, entre as estratégias avaliadas, apenas a poda estruturada reduziu o consumo de energia acima do limiar de detecção.

Quanto à pergunta (B), a resposta é afirmativa, com alcance delimitado. As camadas do GPT-2 *small* diferiram entre si em mais de uma ordem de grandeza quanto

à degradação causada por sua poda, e a camada 0 respondeu, isoladamente, por cerca de três quartos da degradação da poda estruturada com esquema de poda uniforme a 50%. Essa informação, que os esquemas de poda sem análise de sensibilidade não utilizam, resultou em menor perda de desempenho preditivo quando incorporada à distribuição da poda entre as camadas. Na poda estruturada, a estratégia que reduziu o consumo, o esquema de poda com análise de sensibilidade obteve a menor *perplexity* em todas as taxas de 10% a 50%: por fatores de 2,9 a 4 entre 10% e 30%, de 1,9 a 40% e de forma marginal a 50%. Na média geométrica dessa faixa, o fator foi de 2,5, sem diferença de energia distinguível da variação de medição. Três limites delimitam essa conclusão: o esquema de poda deixa de diferenciar as camadas em taxas de poda a partir de 60%; o perfil de sensibilidade precisa ser medido separadamente para cada estratégia de poda; e a vantagem diminui com o retreinamento.

Os indicadores combinados delimitam esse resultado. Nenhuma varredura e nenhuma configuração retreinada superou o modelo base em eficiência relativa, e nenhuma configuração de poda estruturada ficou dentro de 5% da *perplexity* do modelo base, com ou sem retreinamento. Portanto, a análise de sensibilidade não produziu uma taxa de poda estruturada sem perda de desempenho preditivo. Ela reduziu essa perda nas taxas de 10% a 50%, o que tornou o esquema de poda com análise de sensibilidade o de maior eficiência relativa em cada estratégia.

O retreinamento é o limite que mais condiciona a contribuição. O *fine-tuning* de recuperação (Seção 6.5) reduziu a vantagem do esquema de poda com análise de sensibilidade: a 10%, a vantagem desapareceu após uma época de retreinamento, e, a 30%, reduziu-se de um fator de 3,33 para 1,87. Portanto, a utilidade da análise de sensibilidade depende da taxa de poda e da quantidade de retreinamento. Sem retreinamento, a vantagem foi máxima entre 20% e 30% e marginal a 50%. Após uma época, manteve-se a 30% e desapareceu a 10%. Assim, a análise é útil, sobretudo, quando o retreinamento é caro ou indisponível. Por fim, a leitura desses resultados depende do modelo base retreinado: o retreinamento do modelo sem poda reduziu a *perplexity* de 25,17 para 18,68, e é em relação a esse valor, e não ao do modelo base original, que a recuperação deve ser avaliada.

7 Considerações Finais

Referências

BAHDANAU, D.; CHO, K.; BENGIO, Y. Neural machine translation by jointly learning to align and translate. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2015. Citado na página 11.

BENGIO, Y.; SIMARD, P.; FRASCONI, P. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, v. 5, n. 2, p. 157–166, 1994. Citado 2 vezes nas páginas 6 e 11.

BERGSTRA, J.; BENGIO, Y. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, v. 13, p. 281–305, 2012. Citado na página 8.

BIDERMAN, S. et al. Pythia: A suite for analyzing large language models across training and scaling. In: *Proceedings of the 40th International Conference on Machine Learning*. [S.l.: s.n.], 2023. (PMLR, v. 202), p. 2397–2430. Citado 2 vezes nas páginas 42 e 43.

BLALOCK, D. et al. What is the state of neural network pruning? In: *Proceedings of Machine Learning and Systems (MLSys)*. [S.l.: s.n.], 2020. Citado 8 vezes nas páginas 17, 18, 21, 25, 26, 31, 36 e 50.

CHENG, H.; ZHANG, M.; SHI, J. Q. A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, v. 46, n. 12, p. 10558–10578, 2024. Citado na página 49.

COURTY, B. et al. *CodeCarbon: estimate and track carbon emissions from your computing*. 2023. Disponível em: <https://github.com/mlco2/codecarbon>. Acesso em: 17 abr. 2026. Citado na página 65.

DENIL, M. et al. Predicting parameters in deep learning. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2013. Citado 2 vezes nas páginas 16 e 21.

DENTON, E. L. et al. Exploiting linear structure within convolutional networks for efficient evaluation. In: *Advances in Neural Information Processing Systems (NIPS)*. [S.l.: s.n.], 2014. v. 27, p. 1269–1277. Citado na página 17.

DEVLIN, J. et al. BERT: Pre-training of deep bidirectional transformers for language understanding. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. [S.l.]: Association for Computational Linguistics, 2019. p. 4171–4186. Citado 3 vezes nas páginas 42, 43 e 63.

FRANKLE, J.; CARBIN, M. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2019. Citado 2 vezes nas páginas 22 e 25.

FRANTAR, E.; ALISTARH, D. SparseGPT: Massive language models can be accurately pruned in one-shot. In: *Proceedings of the 40th International Conference on Machine Learning*. [S.l.: s.n.], 2023. (PMLR, v. 202), p. 10323–10337. Citado 3 vezes nas páginas 33, 43 e 50.

GAO, L. et al. The Pile: An 800GB dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020. Citado 2 vezes nas páginas 43 e 45.

GOKASLAN, A.; COHEN, V. *OpenWebText Corpus*. 2019. Disponível em: <http://Skylion007.github.io/OpenWebTextCorpus>. Acesso em: 28 set. 2026. Citado 2 vezes nas páginas 44 e 45.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. Cambridge, MA: MIT Press, 2016. Disponível em: <https://www.deeplearningbook.org/>. Citado 7 vezes nas páginas 2, 3, 4, 6, 8, 9 e 29.

GREGERSEN, T.; PATEL, P.; CHOUKSE, E. Input-dependent power usage in GPUs. *arXiv preprint arXiv:2409.18324*, 2024. Citado na página 75.

GURURANGAN, S. et al. Don't stop pretraining: Adapt language models to domains and tasks. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. [S.l.]: Association for Computational Linguistics, 2020. p. 8342–8360. Citado na página 61.

HAN, S. et al. Learning both weights and connections for efficient neural networks. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2015. Citado 8 vezes nas páginas 17, 19, 23, 24, 26, 41, 50 e 61.

HENDERSON, P. et al. Towards the systematic reporting of the energy and carbon footprints of machine learning. *Journal of Machine Learning Research*, v. 21, n. 248, p. 1–43, 2020. Citado na página 31.

HENDRYCKS, D.; GIMPEL, K. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016. Citado na página 14.

HINTON, G.; VINYALS, O.; DEAN, J. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. Citado na página 17.

HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. *Neural Computation*, v. 9, n. 8, p. 1735–1780, 1997. Citado na página 11.

Hugging Face. *Causal language modeling*. 2026. Documentação da biblioteca `transformers`. Disponível em: <https://huggingface.co/docs/transformers/tasks/language_modeling>. Acesso em: 26 out. 2026. Citado na página 62.

Hugging Face. *Perplexity of fixed-length models*. 2026. Documentação da biblioteca `transformers`. Disponível em: <https://huggingface.co/docs/transformers/perplexity>. Acesso em: 30 set. 2026. Citado 4 vezes nas páginas 43, 47, 72 e 85.

JACOB, B. et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [S.l.: s.n.], 2018. Citado na página 16.

JURAFSKY, D.; MARTIN, J. H. *Speech and Language Processing*. 3. ed. [S.l.]: Stanford University, 2025. Rascunho da 3.ª edição. Disponível em: <https://web.stanford.edu/~jurafsky/slp3/>. Citado 2 vezes nas páginas 9 e 87.

KOVALEVA, O. et al. BERT busters: Outlier dimensions that disrupt transformers. In: *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*. [S.l.]: Association for Computational Linguistics, 2021. p. 3392–3405. Citado na página 34.

KRIZHEVSKY, A.; SUTSKEVER, I.; HINTON, G. E. Imagenet classification with deep convolutional neural networks. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2012. Citado na página 5.

LAD, V. et al. Remarkable robustness of LLMs: Stages of inference? In: *Advances in Neural Information Processing Systems*. [S.l.]: Curran Associates, Inc., 2025. v. 38. Citado 2 vezes nas páginas 77 e 88.

LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. *Nature*, v. 521, n. 7553, p. 436–444, 2015. Citado 2 vezes nas páginas 2 e 6.

LECUN, Y.; DENKER, J. S.; SOLLA, S. A. Optimal brain damage. In: *Advances in Neural Information Processing Systems*. [S.l.]: Morgan Kaufmann, 1990. v. 2, p. 598–605. Citado na página 21.

LHOEST, Q. et al. Datasets: A community library for natural language processing. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. [S.l.]: Association for Computational Linguistics, 2021. p. 175–184. Citado na página 64.

LI, H. et al. Pruning filters for efficient convnets. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2017. Citado 7 vezes nas páginas 19, 24, 25, 26, 36, 41 e 50.

LIU, Z. et al. Rethinking the value of network pruning. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2019. Citado na página 25.

LOSHCHILOV, I.; HUTTER, F. Decoupled weight decay regularization. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2019. Citado na página 63.

LUCCIONI, A. S.; VIGUIER, S.; LIGOZAT, A.-L. Estimating the carbon footprint of BLOOM, a 176B parameter language model. *arXiv preprint arXiv:2211.02001*, 2022. Citado na página 16.

MERITY, S. et al. Pointer sentinel mixture models. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2017. Citado 3 vezes nas páginas 42, 45 e 46.

MICHEL, P.; LEVY, O.; NEUBIG, G. Are sixteen heads really better than one? In: *Advances in Neural Information Processing Systems*. [S.l.]: Curran Associates, Inc., 2019. v. 32. Citado 5 vezes nas páginas 13, 21, 50, 58 e 83.

MICIKEVICIUS, P. et al. Mixed precision training. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2018. Citado na página 63.

NAIR, V.; HINTON, G. E. Rectified linear units improve restricted Boltzmann machines. In: *Proceedings of the 27th International Conference on Machine Learning (ICML)*. [S.l.: s.n.], 2010. Citado 2 vezes nas páginas 5 e 14.

NIELSEN, M. A. *Neural Networks and Deep Learning*. [S.l.]: Determination Press, 2015. Disponível em: <http://neuralnetworksanddeeplearning.com/>. Citado 3 vezes nas páginas 3, 4 e 5.

NVIDIA Corporation. *NVIDIA T4 Tensor Core GPU: Datasheet*. 2019. Disponível em: <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/tesla-t4/t4-tensor-core-datasheet-951643.pdf>. Acesso em: 28 set. 2026. Citado 3 vezes nas páginas 27, 64 e 67.

NVIDIA Corporation. *NVIDIA A100 Tensor Core GPU Architecture*. 2020. White paper. Descreve o suporte a esparsidade estruturada 2:4 na arquitetura Ampere. Citado na página 19.

PASZKE, A. et al. Pytorch: An imperative style, high-performance deep learning library. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2019. Citado na página 44.

PATTERSON, D. et al. Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350*, 2021. Citado na página 15.

RADFORD, A. et al. Language models are unsupervised multitask learners. *OpenAI Technical Report*, 2019. Citado 6 vezes nas páginas 14, 27, 43, 45, 46 e 72.

RAJBHANDARI, S. et al. ZeRO: Memory optimizations toward training trillion parameter models. In: *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. [S.l.]: IEEE, 2020. p. 1–16. Citado 2 vezes nas páginas 27 e 43.

RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. *Nature*, v. 323, n. 6088, p. 533–536, 1986. Citado na página 7.

SANH, V. et al. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019. Citado 2 vezes nas páginas 42 e 43.

SANH, V.; WOLF, T.; RUSH, A. M. Movement pruning: Adaptive sparsity by fine-tuning. In: *Advances in Neural Information Processing Systems*. [S.l.]: Curran Associates, Inc., 2020. v. 33, p. 20378–20389. Citado 2 vezes nas páginas 49 e 50.

SCHWARTZ, R. et al. Green AI. *Communications of the ACM*, v. 63, n. 12, p. 54–63, 2020. Citado 2 vezes nas páginas 15 e 31.

SEVILLA, J. et al. Compute trends across three eras of machine learning. *arXiv preprint arXiv:2202.05924*, 2022. Citado na página 16.

SNOEK, J.; LAROCHELLE, H.; ADAMS, R. P. Practical Bayesian optimization of machine learning algorithms. In: *Advances in Neural Information Processing Systems (NIPS)*. [S.l.: s.n.], 2012. v. 25, p. 2951–2959. Citado na página 8.

SPEARMAN, C. The proof and measurement of association between two things. *The American Journal of Psychology*, v. 15, n. 1, p. 72–101, 1904. Citado na página 56.

STRUBELL, E.; GANESH, A.; MCCALLUM, A. Energy and policy considerations for deep learning in nlp. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*. [S.l.: s.n.], 2019. Citado 2 vezes nas páginas 15 e 27.

SUN, M. et al. A simple and effective pruning approach for large language models. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2024. Citado 3 vezes nas páginas 33, 43 e 50.

TOUVRON, H. et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023. Citado na página 27.

VASWANI, A. et al. Attention is all you need. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2017. p. 5998–6008. Citado 4 vezes nas páginas 10, 12, 13 e 88.

WOLF, T. et al. Transformers: State-of-the-art natural language processing. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. [S.l.]: Association for Computational Linguistics, 2020. p. 38–45. Citado na página 28.

YANG, T.-J.; CHEN, Y.-H.; SZE, V. Designing energy-efficient convolutional neural networks using energy-aware pruning. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. [S.l.: s.n.], 2017. Citado na página 25.

ZHANG, S. et al. OPT: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022. Citado 2 vezes nas páginas 27 e 43.

ZHU, L. *THOP: PyTorch-OpCounter*. 2022. Biblioteca Python `thop`, versão 0.1.1. Disponível em: <https://github.com/Lyken17/pytorch-OpCounter>. Acesso em: 26 out. 2026. Citado na página 64.

ZHU, M.; GUPTA, S. To prune, or not to prune: Exploring the efficacy of pruning for model compression. *arXiv preprint arXiv:1710.01878*, 2017. Citado na página 22.

8 Apêndice

[TO DO]